

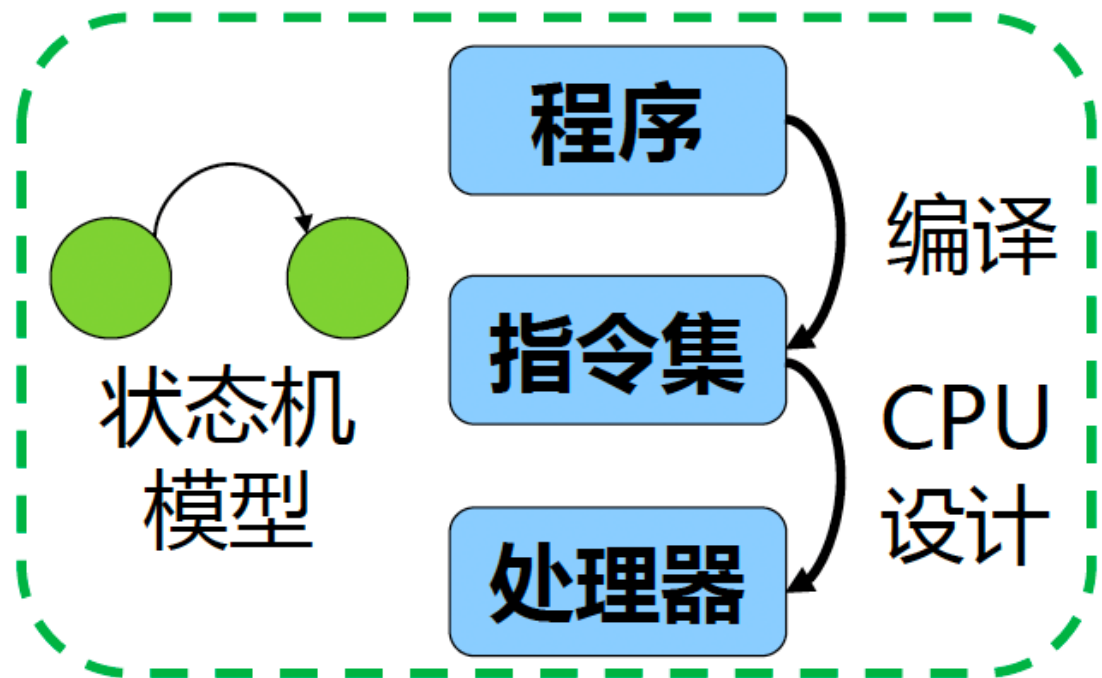
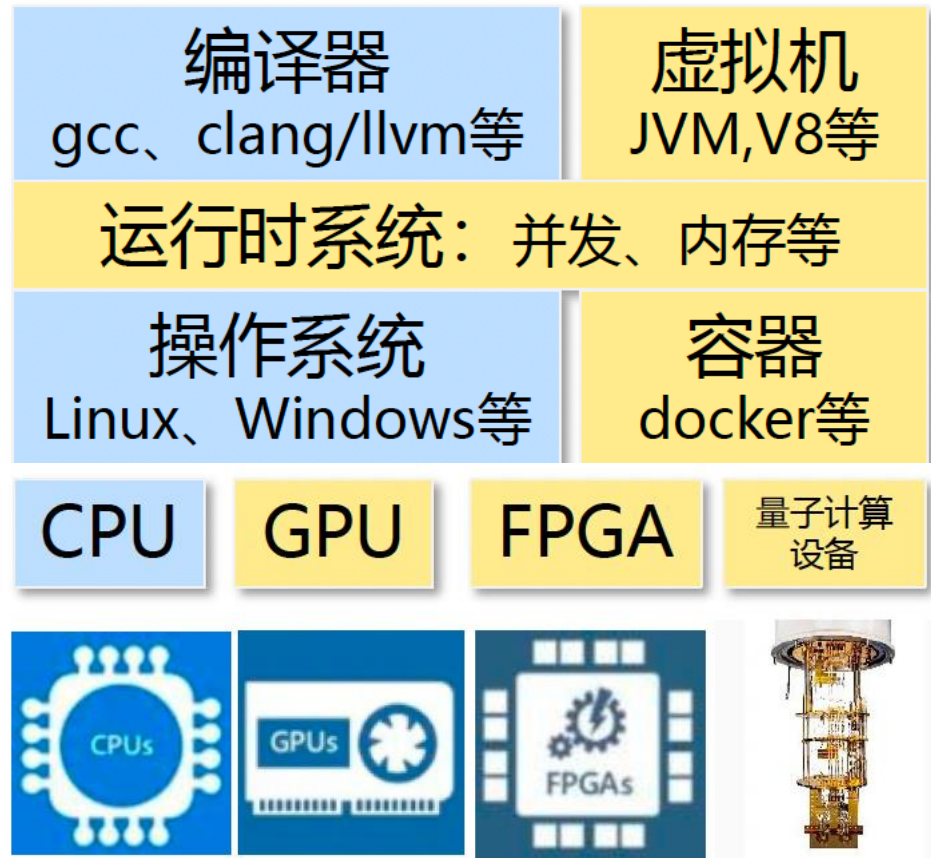
Review

- 1. Fundamentals of computer system -----
- 2. ILP-----
- 3. Memory Hierarchy (Cache)-----
- 4. Memory Hierarchy (Main Memory)-----
Main memory, Virtual memory
- 5. File System-----
Mass-storage structure, File system, IO, Virtualization
- 6. DLP and TLP-----

SysIII-Lab

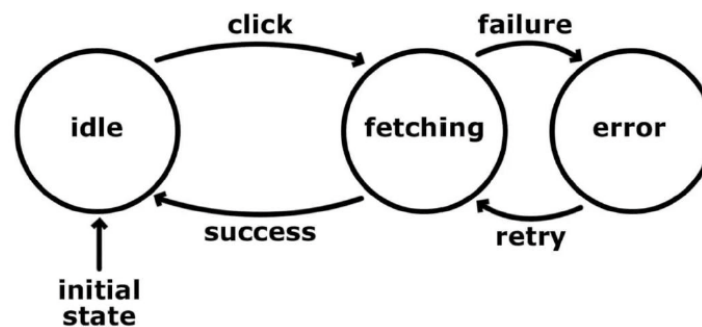
- 动态分支预测
- 带Cache的流水线CPU设计实现
- RV64 虚拟内存管理
- RV64 用户模式
- RV64 缺页异常处理以及 fork 机制
- RV64 架构软硬件贯通综合实践
- 鲲鹏Arm架构系统综合实践-Bonus

knowledge map



通过状态机建立程序-指令集-处理器的关联

- 状态集合 $S = \{S1, S2, \dots\}$
- 激励事件
- 状态转移规则
- 初始状态 $S0 \in S$

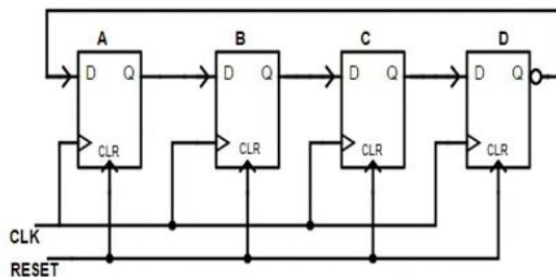


程序是个状态机

```
/* 1 */ int main() {  
/* 2 */  int x = 1;  
/* 3 */  int y = 2;  
/* 4 */  int z = x + y;  
/* 5 */  printf("z = %d\n", z);  
/* 6 */  return 0;  
/* 7 */ }
```

```
// S = <x, y, z, PC>  
S0 = <?, ?, ?, 2> // '?'表示未初始化  
S1 = <1, ?, ?, 3>  
S2 = <1, 2, ?, 4>  
S3 = <1, 2, 3, 5>  
S4 = <1, 2, 3, 6> // 输出"z = 3"  
S5 = <1, 2, 3, 结束>
```

处理器是个状态机



```
// S = <A, B, C, D>  
S0 = <0, 0, 0, 0>  
S1 = <1, 0, 0, 0>  
S2 = <1, 1, 0, 0>  
S3 = <1, 1, 1, 0>  
S4 = <1, 1, 1, 1>  
S5 = <0, 1, 1, 1>  
S6 = <0, 0, 1, 1>  
S7 = <0, 0, 0, 1>  
S8 = <0, 0, 0, 0> = S0
```

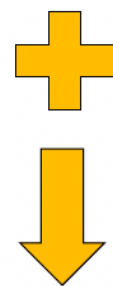
指令集是个状态机

// PC: instruction	label: statement
0: li x1, 0	pc0: x1 = 0;
1: li x2, 0	pc1: x2 = 0;
2: li x3, 100	pc2: x3 = 100;
3: addi x2, x2, 1	pc3: x2 = x2 + 1;
4: add x1, x1, x2	pc4: x1 = x1 + x2;
5: blt x2, x3, 3	pc5: if (x2 < x3) goto pc3;
6: j 6	pc6: goto pc6;

用C程序理解指令
执行指令 = 改变机器状态

	程序	指令集定义的 抽象计算机	处理器
状态	{变量, PC}	{寄存器, 内存}	{时序逻辑电路}
状态转移规则	C语句的语义	指令的语义	组合逻辑电路
依据	C语言标准手册	指令集手册	处理器设计文档

程序编译 = 将语句翻译成语义等价的指令序列



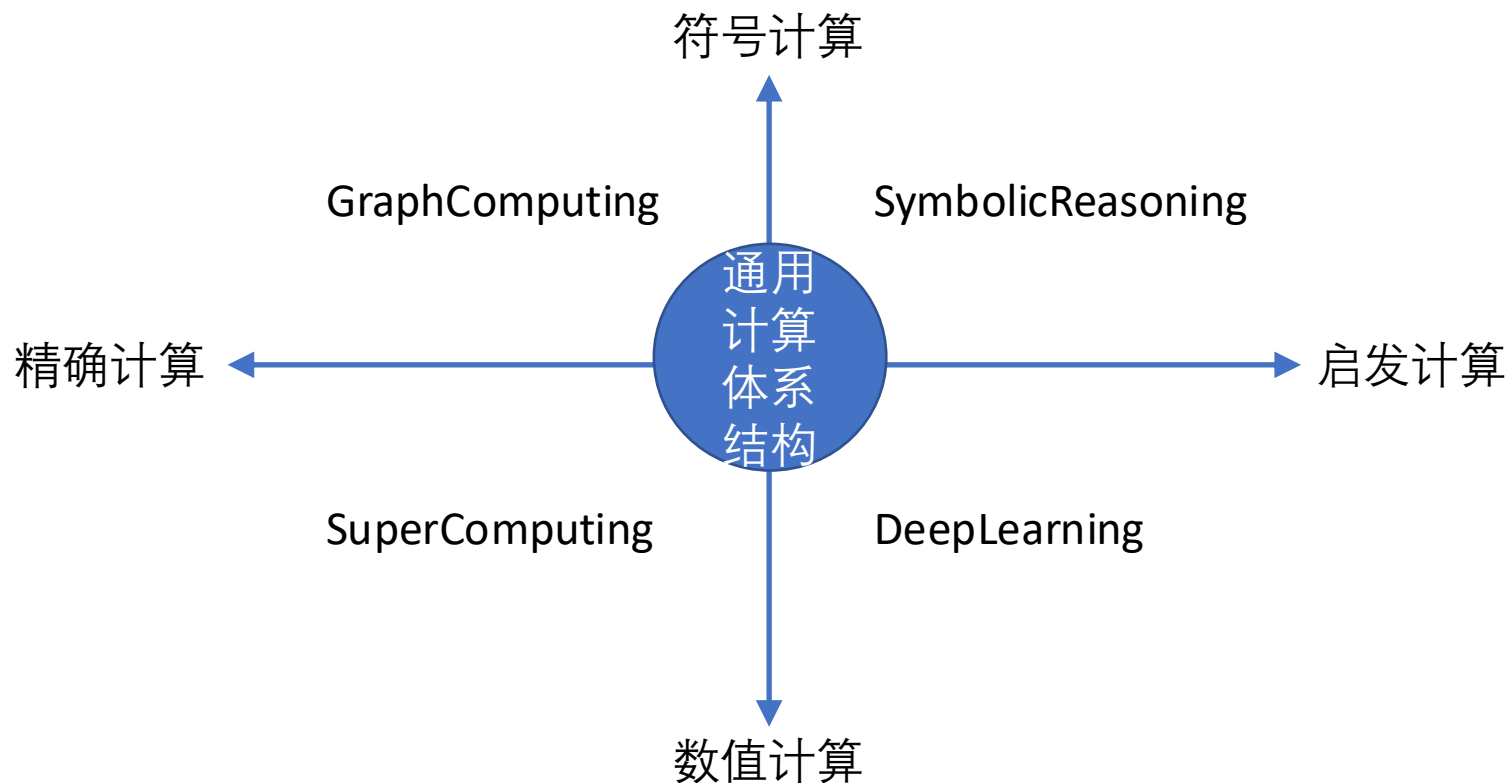
架构设计 = 按照指令语义设计行为等价的电路

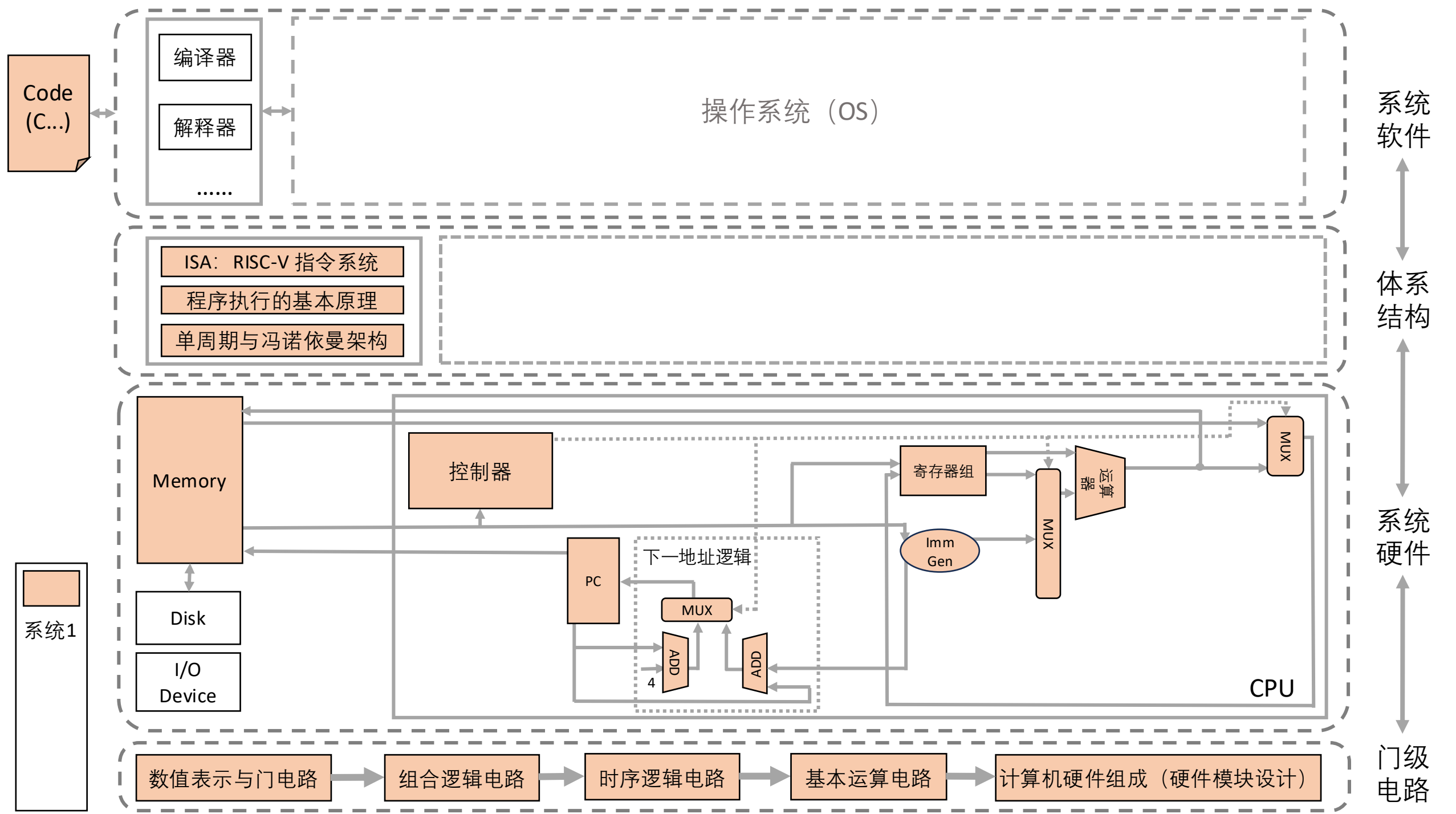
程序在处理器上运行 = 指令序列控制处理器电路进行状态转移

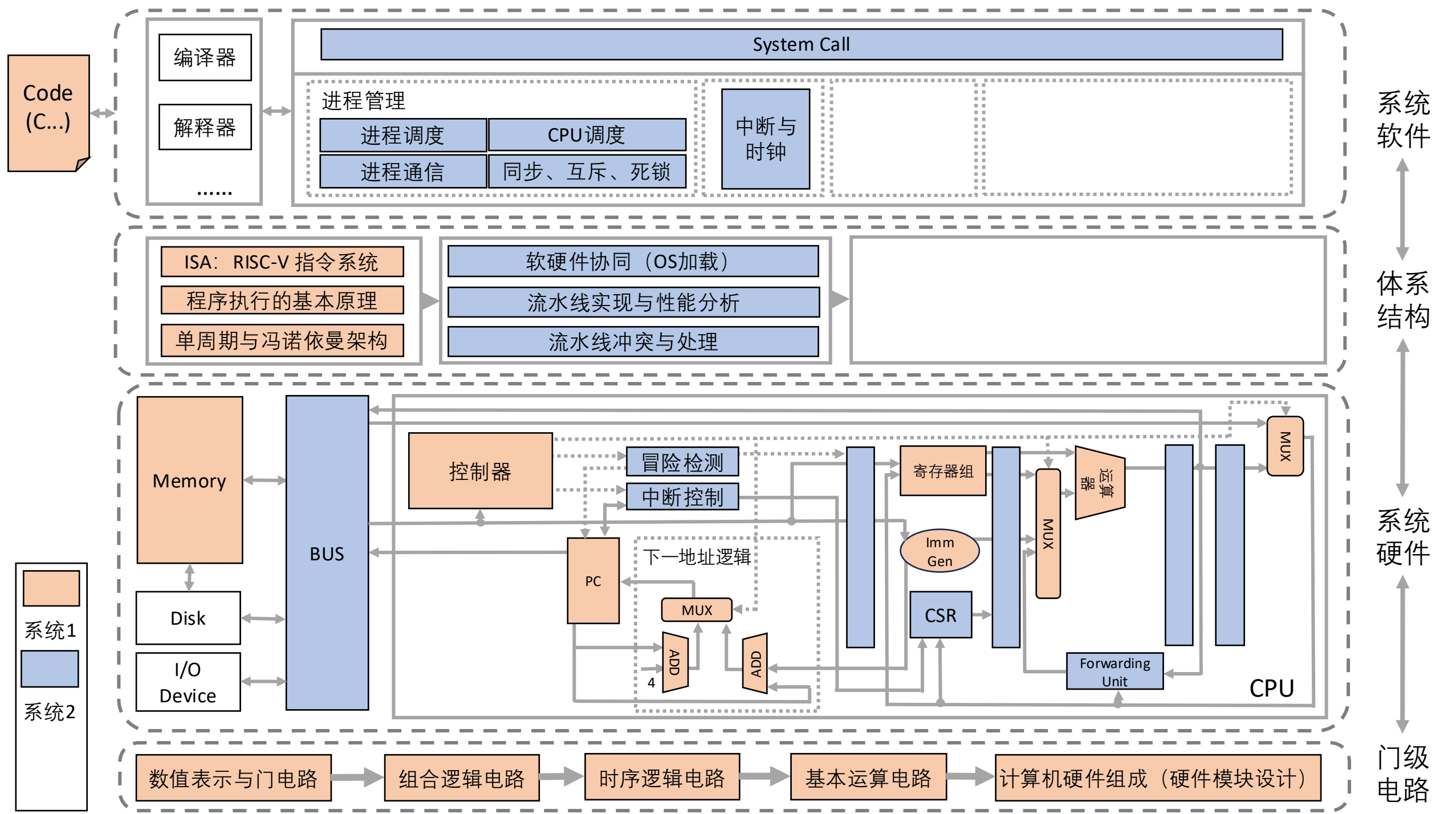
3个状态机产生关联

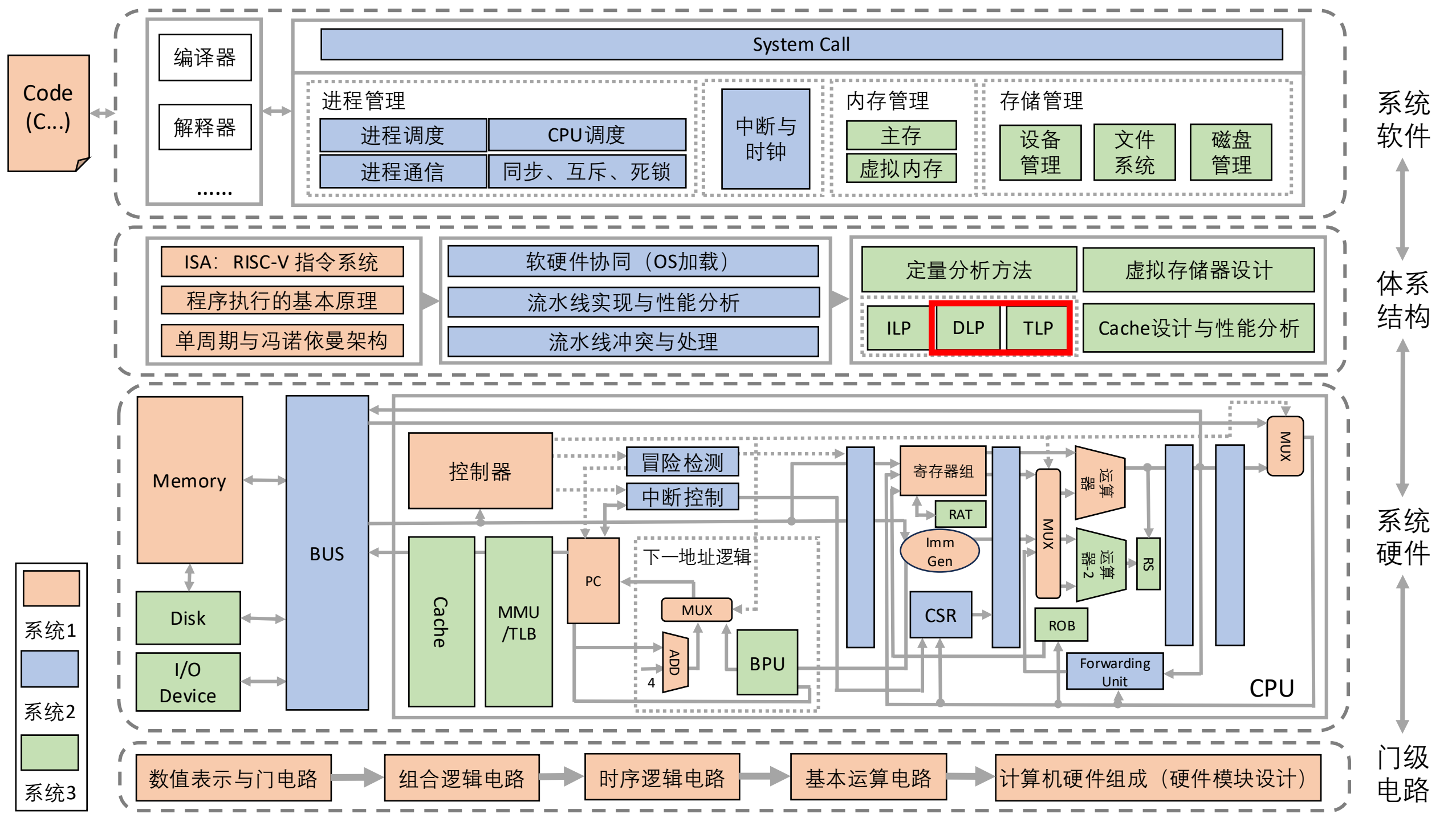
软硬件协同

应用程序	从C语言到二进制、ELF文件和链接	游戏	OS应用
运行时环境	裸机运行时环境	IO库	C库
操作系统			自制OS
指令集	RISC-V指令集、程序的机器级表示	I/O指令	自陷指令
微结构	模拟器、单周期	总线、设备	异常机制





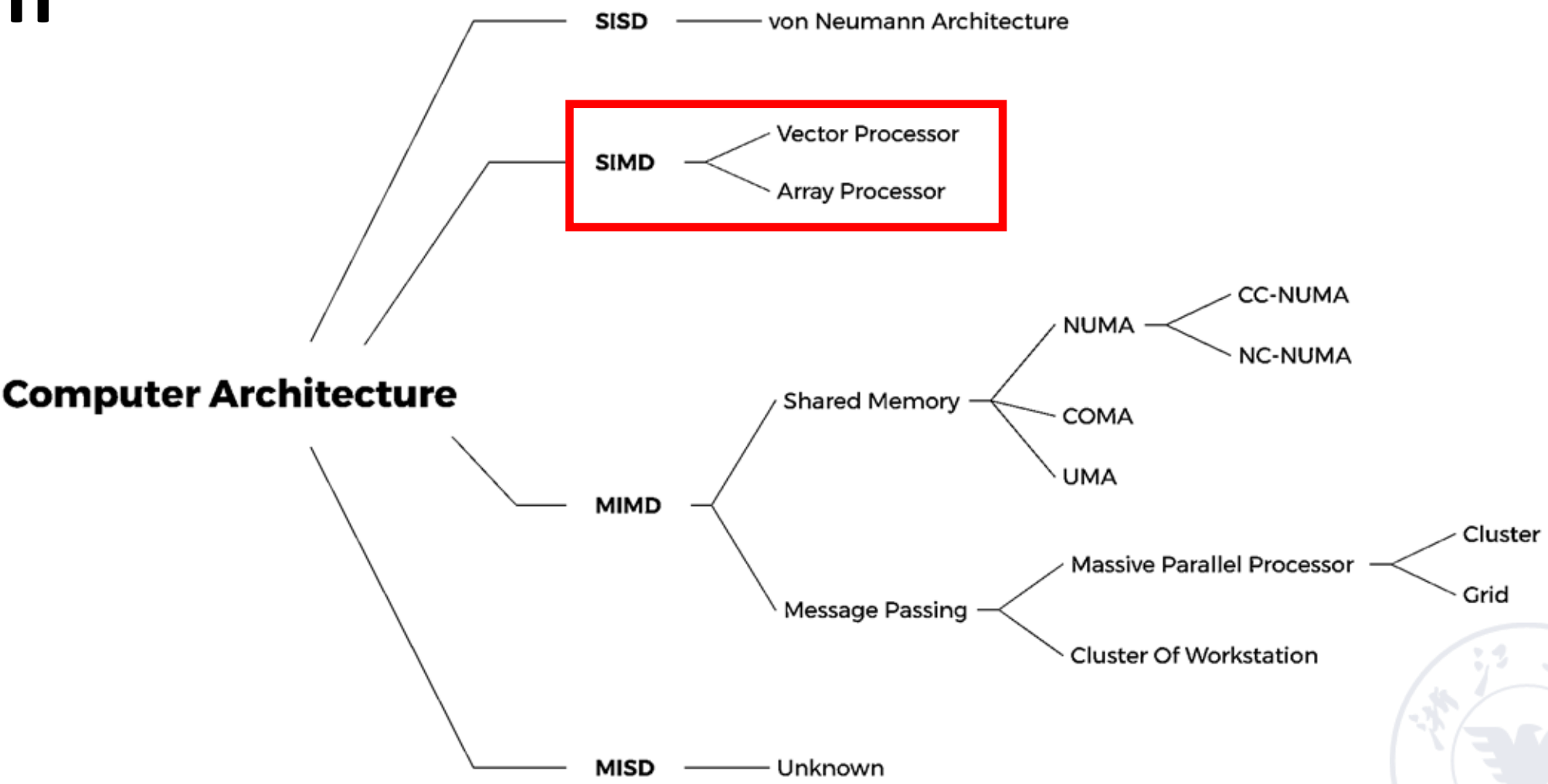




DLP and TLP



Flynn



Vector Processor & Scalar Processor

- A pipeline processor, in which the vector data representation and the corresponding vector instructions are set, is called the **vector processor**.
- A pipeline processor that does not have vector data representation and corresponding vector instructions is called a **scalar processor**.



Vector Processor & Scalar Processor

- The particularity of vector pipeline:
The elements in the vector are rarely correlated during operations.
- Improper vector handling can also cause related problem and frequent function switching.
- Problems to be solved by vector pipeline processing:
How to deal with vectors and arrays to maximize the effectiveness of the pipeline.



Vector Processor & Scalar Processor

- (1) Horizontal processing method:
 - Vector calculations are performed horizontally from left to right in a row.
- (2) Vertical processing method:
 - The vector calculation is performed vertically from top to bottom in a column manner.
- (3) Vertical and horizontal processing method (group processing method):
 - A combination of horizontal processing and vertical processing.



Vector Processor & Scalar Processor

Example: $D = A \times (B + C)$ A, B, C, D — vector of length N

1. Horizontal processing method

- Vector calculations are performed horizontally from left to right in a row-wise manner.
 - First calculate: $d_1 \leftarrow a_1 \times (b_1 + c_1)$
 - Recalculate: $d_2 \leftarrow a_2 \times (b_2 + c_2)$
 - ...
 - Final calculation: $d_N \leftarrow a_N \times (b_N + c_N)$
- Compose a loop program for processing.
 - $k_i \leftarrow b_i + c_i$
 - $d_i \leftarrow a_i \times k_i$
- Data related: N times Function switching: 2N times



Vector Processor & Scalar Processor

Example: $D = A \times (B + C)$ A, B, C, D — vector of length N

1. Horizontal processing method

Problems with horizontal processing:

- When calculating each component, **RAW** correlation occurs, and the **pipeline efficiency is low**.
- If a **static multi-functional pipeline** is used, the pipeline must be switched frequently; the **throughput** of the pipeline is lower than that of **sequential serial execution**.
- The horizontal processing method is not suitable for vector processors.



Vector Processor & Scalar Processor

Example: $D = A \times (B + C)$ A, B, C, D — vector of length N

2. Vertical processing method

Problems with Vertical processing:

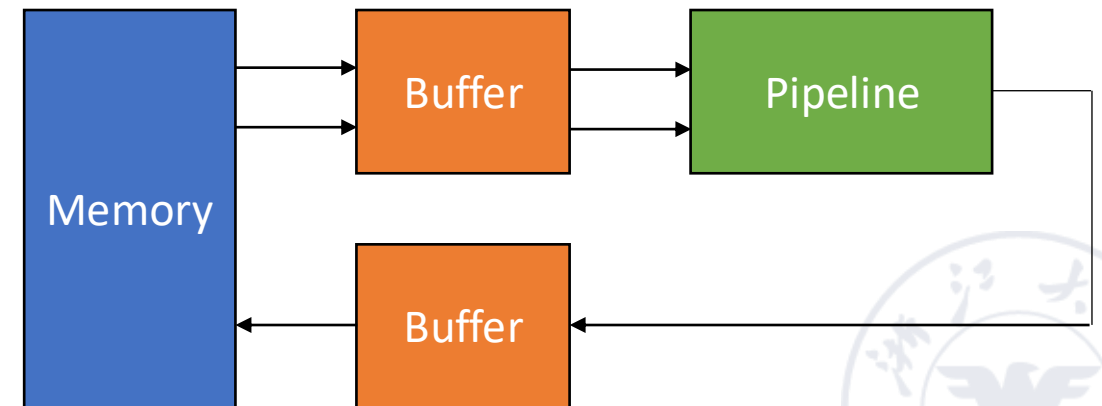
$$\begin{aligned} K &\leftarrow B + C \\ D &\leftarrow A \times K \end{aligned}$$

Data related: 1 times Function switching: 2 times



Vector Processor & Scalar Processor

- Requirements for processor structure: [memory-memory structure](#)
- The source vector and destination vector of the vector instruction are stored in the memory, and the intermediate result of the operation needs to be sent back to the memory.
- Memory-memory operation pipeline
 - For example: STAR-100, CYBER-205



Vector Processor & Scalar Processor

3. Vertical and horizontal (grouping) processing method

- Divide the vector into several groups, process them in a vertical manner, and process each group in turn.
- For the above example, let:
 - $N = S \times n + r$
 - Where N is the length of the vector, S is the number of groups, n is the length of each group, and r is the remainder.
 - If the remaining r numbers are also treated as a group, there are a total of $S + 1$ groups.



Vector Processor & Scalar Processor

3. Vertical and horizontal (grouping) processing method

- The calculation process is:
 - Calculate the first group: $d_{1\sim n} \leftarrow a_{1\sim n} \times (b_{1\sim n} + c_{1\sim n})$
 - Calculate the second group: $d_{(n+1)\sim 2n} \leftarrow a_{(n+1)\sim 2n} \times (b_{(n+1)\sim 2n} + c_{(n+1)\sim 2n})$
 - ...
 - Final calculation the last group: $d_{(s*n+1)\sim N} \leftarrow a_{(s*n+1)\sim N} \times (b_{(s*n+1)\sim N} + c_{(s*n+1)\sim N})$

Data related: $S+1$ times Function switching: $2(S+1)$ times



Vector Processor & Scalar Processor

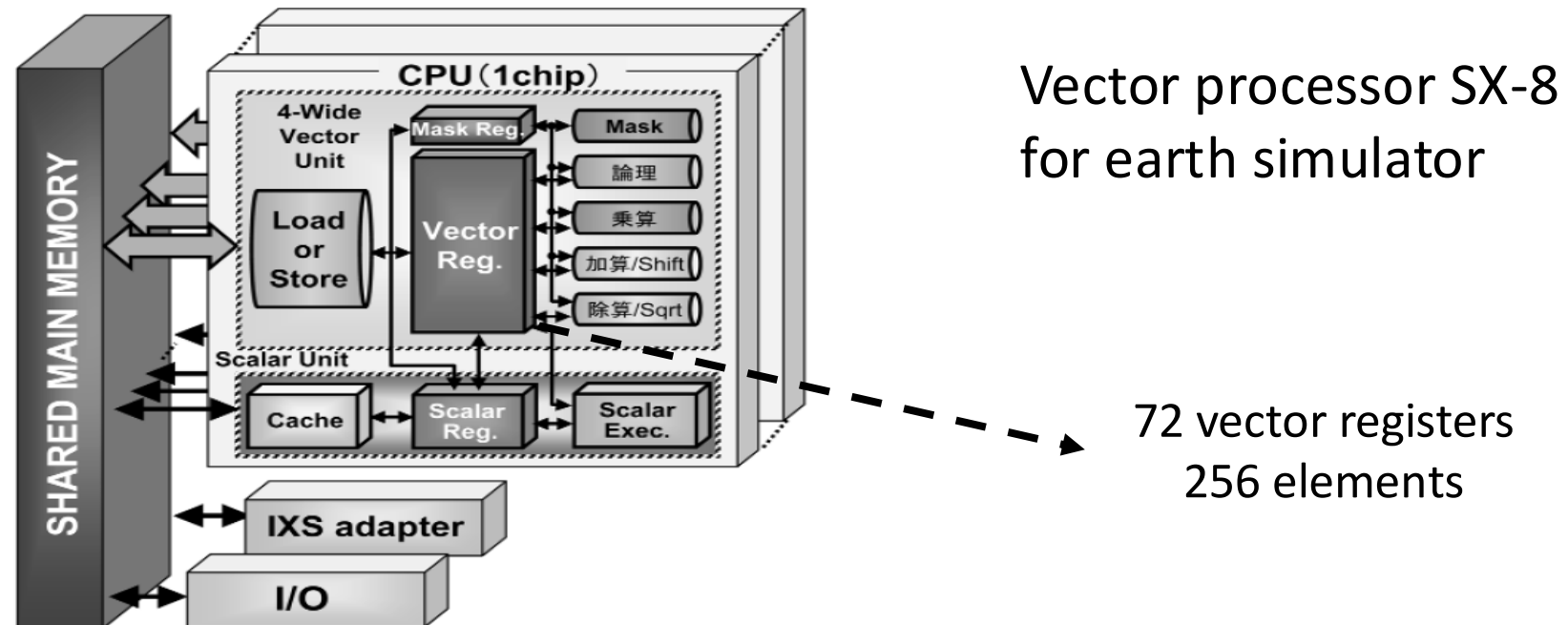
- Requirements for processor structure: [register-register structure](#)
- Set up vector registers that can be accessed quickly to store source vectors, destination vectors, and intermediate results, so that the input and output ends of the arithmetic components are connected with the vector registers to form a register-register type operation pipeline.



Vector Processor & Scalar Processor

- Typical register-vector processor with register structure

American CRAY-1, my country's YH-1 supercomputer, GRAP-3, earth simulator

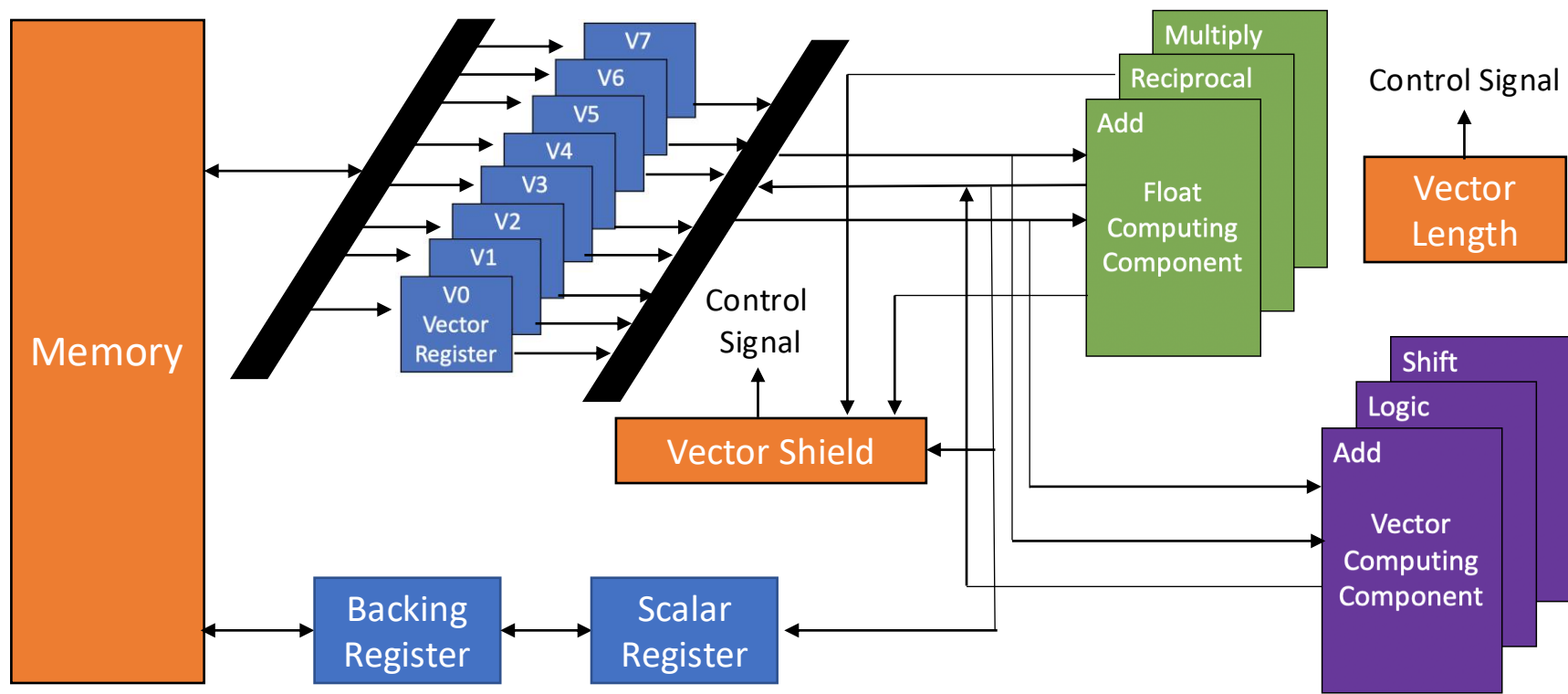


Vector Processor & Scalar Processor

- The structure of the vector pipeline processing machine varies from machine to machine.
- Take the vector pipeline processing part in CRAY-1 as an example to introduce some structural features of the register-register-oriented vector pipeline processing machine.



CRAY-1

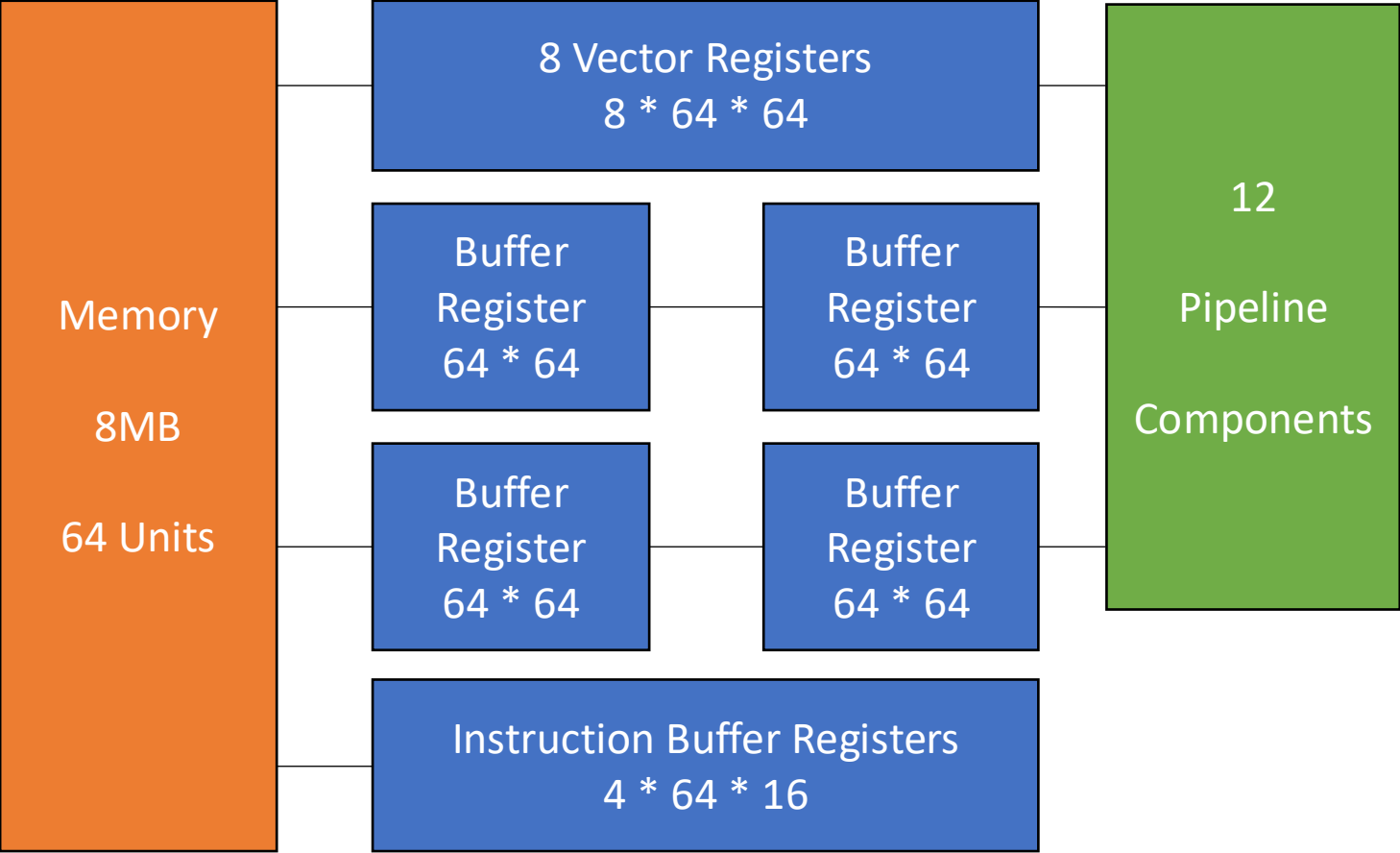


CRAY-1
American CRAY Company
100 million FLOPS
Clock Period: 12.5 ns

There are 12 single-function pipelines that can work in parallel, which can perform various operations of address, vector, and scalar in a pipeline.



CRAY-1



Features of CRAY-1 Vector Processor

- Each vector register V_i has a separate bus connected to 6 vector functional units.
- Each vector function unit also has a bus that returns the result of the operation to the vector register bus.
- As long as there is no V_i conflict and functional conflict, each V_i and each functional unit can work in parallel, which greatly speeds up the processing of vector instructions.



Features of CRAY-1 Vector Processor

- **Vi conflict**: The source vector or result vector of each vector instruction working in parallel uses the same V_i .
- Writing and reading data related:
 - $V0 \leftarrow V1 + V2$
 - $V3 \leftarrow V0 \times V4$
- Reading data related
 - $V0 \leftarrow V1 + V2$
 - $V3 \leftarrow V1 \times V4$

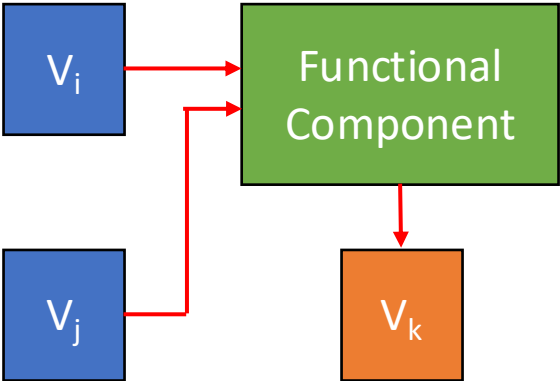


Features of CRAY-1 Vector Processor

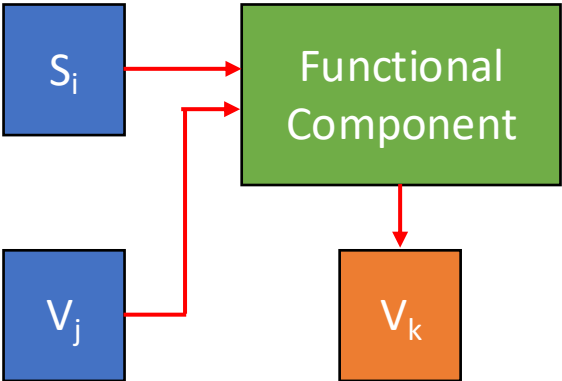
- **Functional conflict:** Each vector instruction working in parallel must use the same functional unit.
- For example: (all need to use multiplication feature)
 - $V3 \leftarrow V1 \times V2$
 - $V5 \leftarrow V4 \times V6$
- After the last component of the first vector instruction is executed and the float multiplication function is released, the second vector starts to execute.



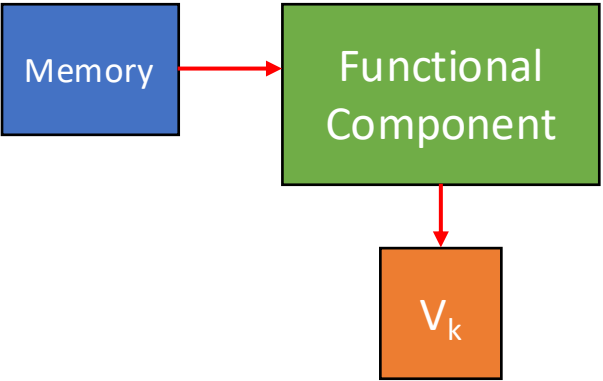
Instruction Types of CRAY-1



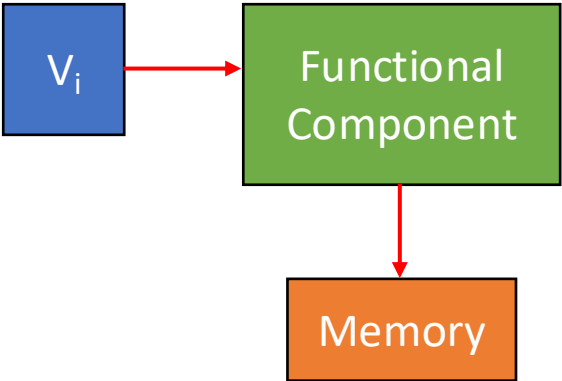
$$V_k \leftarrow V_i \text{ op } V_j$$



$$V_k \leftarrow S_i \text{ op } V_j$$



$$V_k \leftarrow \text{Memory}$$



$$\text{Memory} \leftarrow V_i$$



Improve the Performance of Vector Processor

- Set up multiple functional components and make them work in parallel.
- Use vector chaining technology to speed up the execution of a string of vector instructions.
- Adopt recycling mining technology to speed up recycling processing.
- Using a multi-processor system to further improve the performance.



Improve the Performance of Vector Processor

1. Set up multiple functional units and make them work in parallel.
 - These components can work in parallel, and each work in a pipeline manner, thus forming multiple parallel operation pipelines.

For example:

- **CRAY-1** vector processor has 4 groups of 12 single-function pipeline components:
 - Vector components: vector addition, shift, logic operation
 - Floating point components: float addition, multiplication, reciprocal calculation
 - Scalar components: scalar addition, shift, logic operation, number "1"/count
 - Address calculation components: integer addition, multiplication



Improve the Performance of Vector Processor

2. Use vector chaining technology to speed up the execution of a string of vector instructions.

- Vector chaining feature: It has two related instructions that are written first and then read. In the case of no conflicts between functional components and source vector conflicts, functional components can be chained for pipeline processing to achieve the purpose of speeding up execution.
- The essence of the chaining feature: the result of introducing the idea of pipeline orientation to the vector execution process.



Improve the Performance of Vector Processor

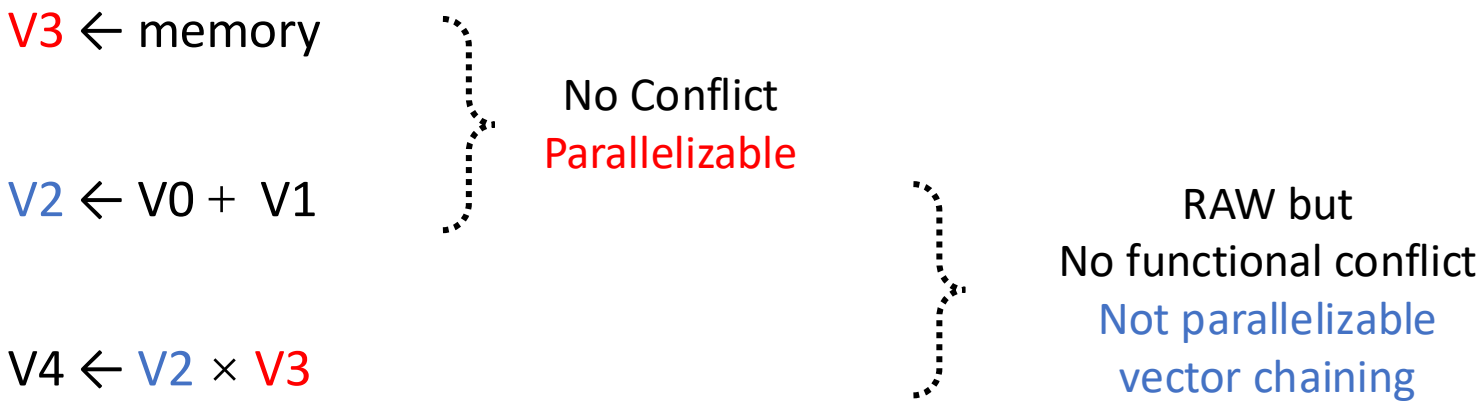
Example: Use vector chaining technology to perform vector operations on CRAY-1

- $D = A \times (B + C)$
- Assuming that the vector length is $N \leq 64$, the vector elements are floating numbers, and the vectors B and C have been stored in $V0$ and $V1$.
- Draw a vector chaining diagram and analyze the different execution times in the case of non-chaining execution and chaining execution.
- Solution: Use the following three vectors to complete the above operation:
 - $V3 \leftarrow \text{memory}$ // access vector A
 - $V2 \leftarrow V0 + V1$ // Vector B and Vector C perform floating point addition
 - $V4 \leftarrow V2 \times V3$ // Floating point multiplication, the result is stored in V4



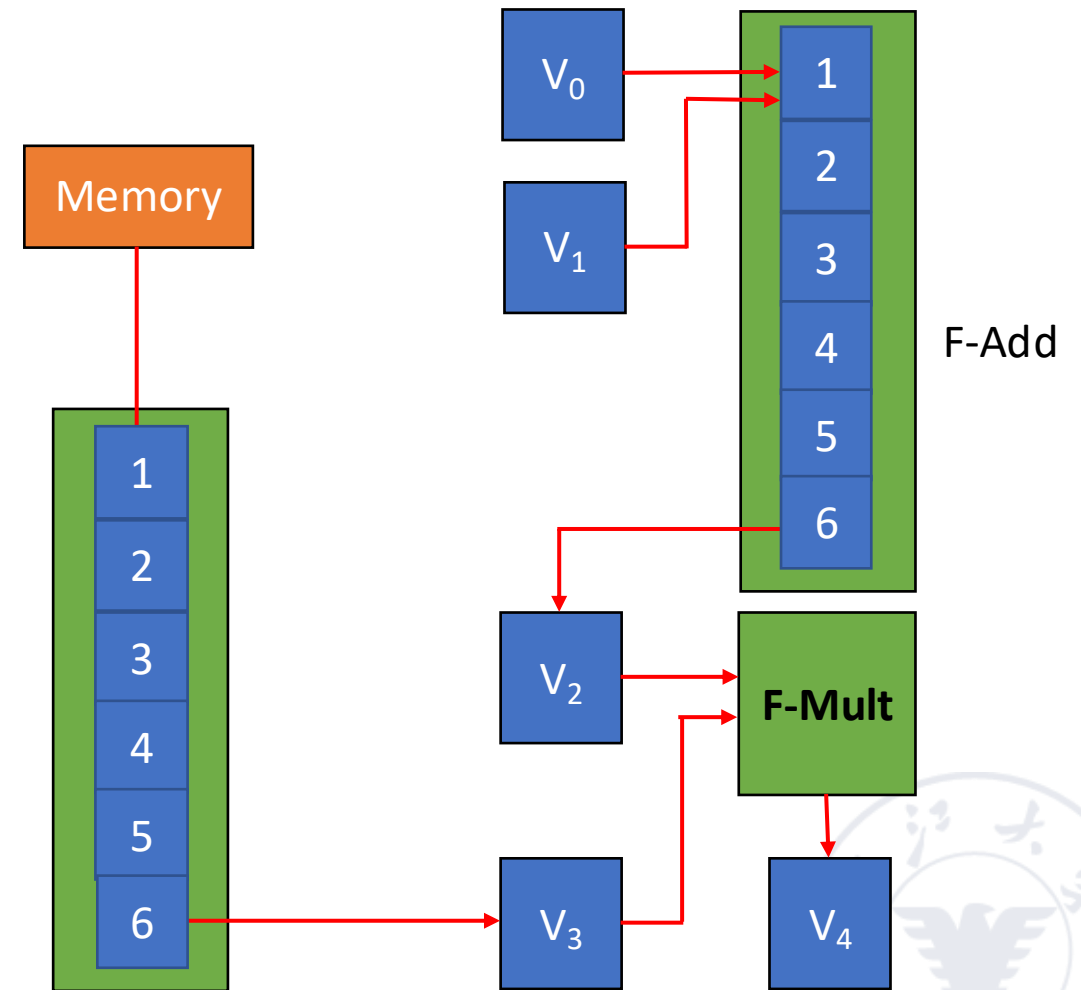
Improve the Performance of Vector Processor

Hypothesis: It takes **one beat** to send vector data elements to the vector function unit and to store the result in the vector register, and **one beat** time to send data from the memory to the fetch function unit.



Improve the Performance of Vector Processor

- Why 8 beats?
- It takes 1 beat of transmission delay to start the memory fetch, send the element to the functional part and send the calculation result to V_i .



Improve the Performance of Vector Processor

- Calculate $D = A \cdot (B + C)$, A, B, C, D are all N elements Vector, vector length $N \leq 64$, complete operation with 3 instructions,
 - (1). $V3 \leftarrow A$
 - (2). $V2 \leftarrow V0 + V1$
 - (3). $V4 \leftarrow V2 * V3$
- Calculate how many shots each of the following three methods will take to get all the results?
 - (1), (2), (3) Serial execution;
 - After (1) and (2) are executed in parallel, execute (3);
 - Use vector chaining technology.



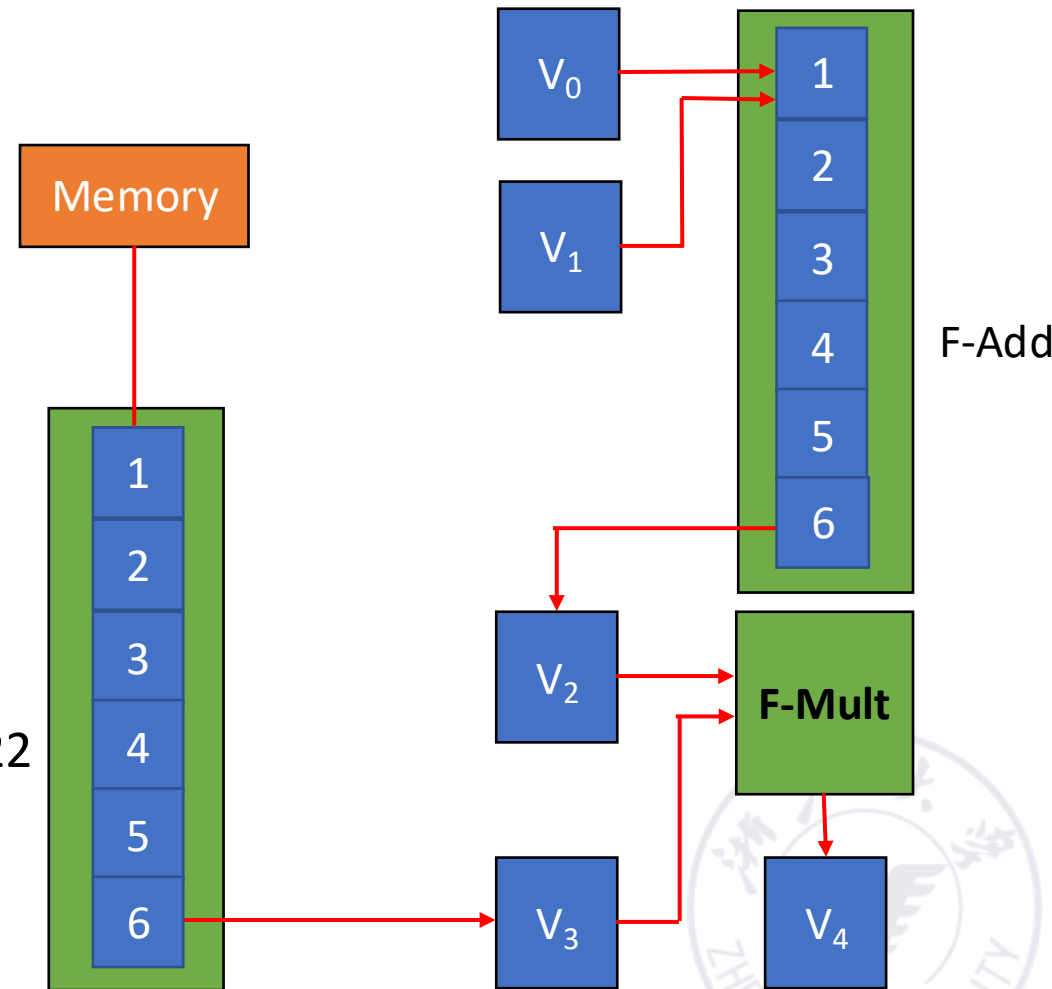
Improve the Performance of Vector Processor

- Let the vector length be N

- (1). $V_3 \leftarrow A$
- (2). $V_2 \leftarrow V_0 + V_1$
- (3). $V_4 \leftarrow V_2 * V_3$

- (1) The execution time using serial method is:

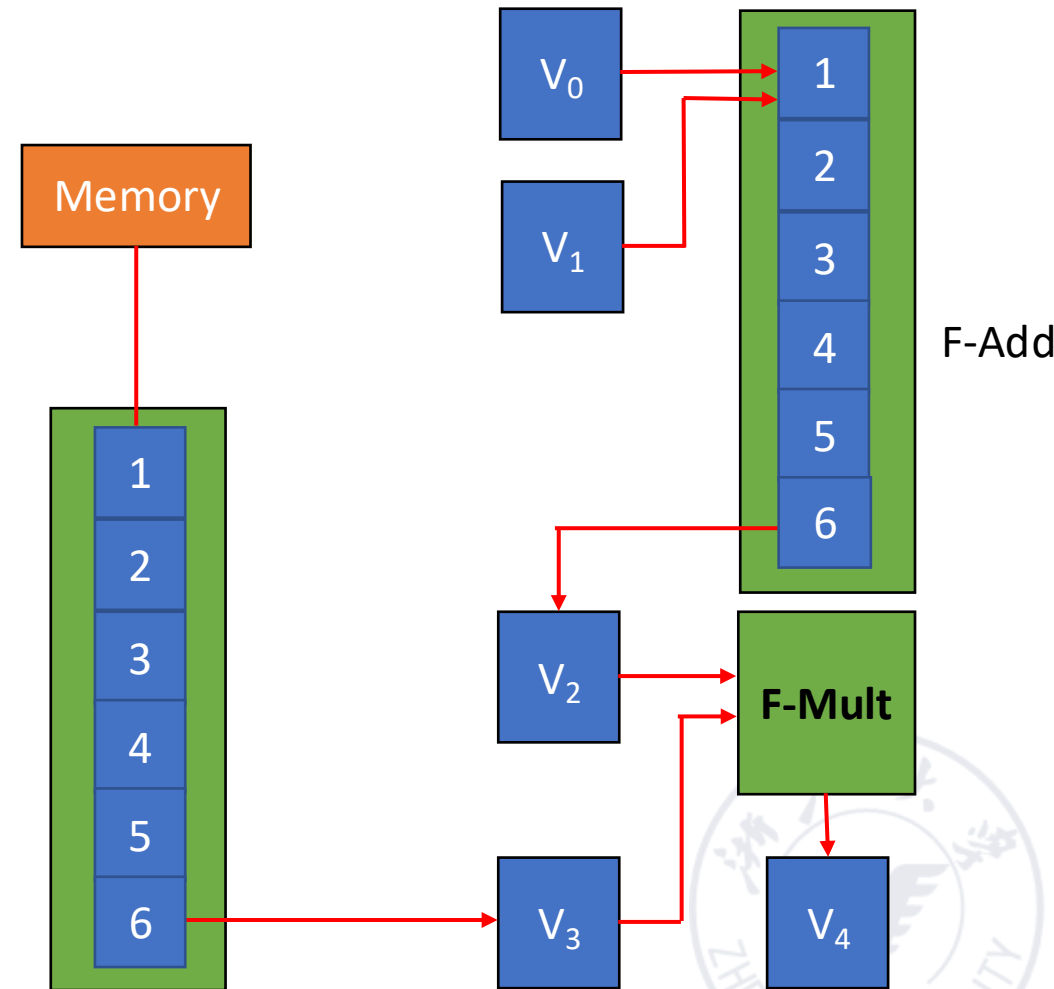
$$[(1+6+1)+N-1] + [(1+6+1)+N-1] + [(1+7+1)+N-1] = 3N + 22$$



Improve the Performance of Vector Processor

- Let the vector length be N
 - (1). $V_3 \leftarrow A$
 - (2). $V_2 \leftarrow V_0 + V_1$
 - (3). $V_4 \leftarrow V_2 * V_3$
- (2) The first two instructions are parallel, and the third is serial. The execution time is:

$$\max\{[(1+6+1)+N-1], [(1+6+1)+N-1]\} + [(1+7+1)+N-1] = 2N + 15$$

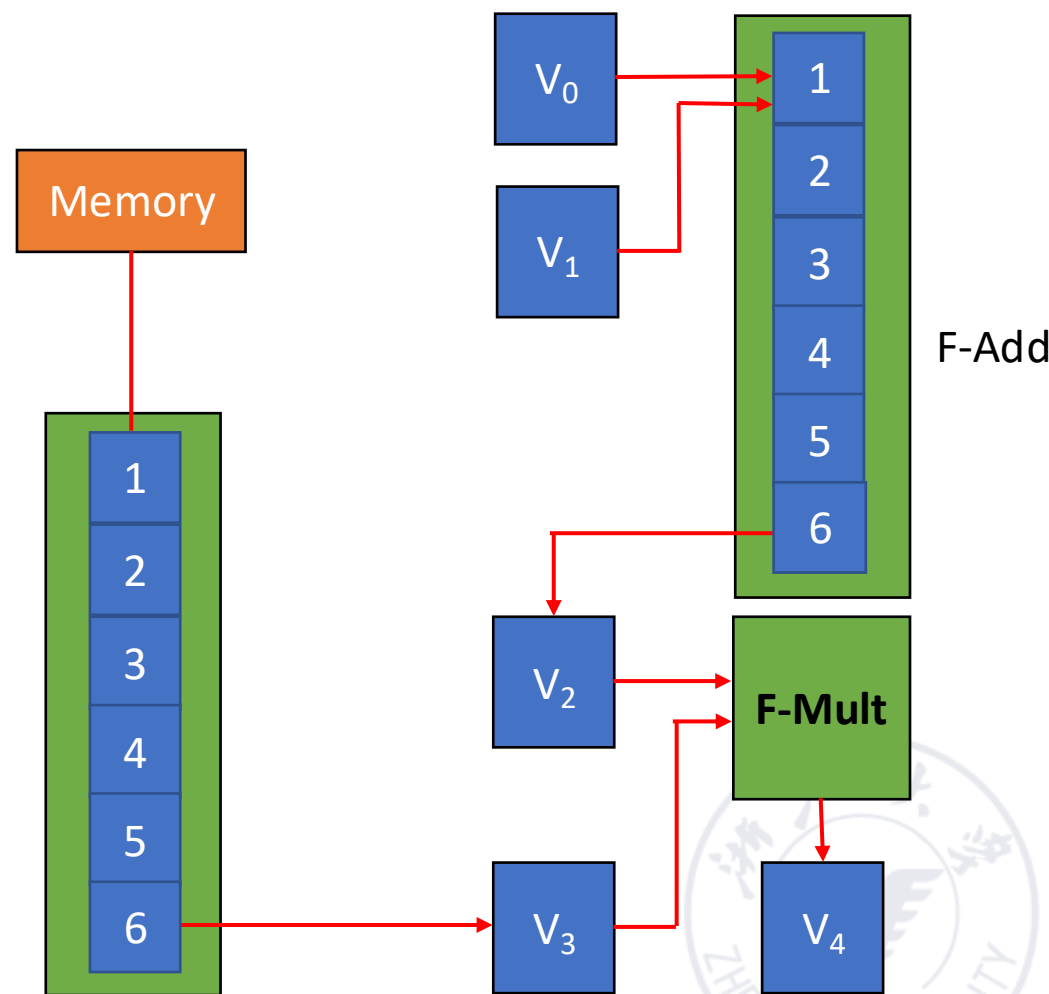


Improve the Performance of Vector Processor

- Let the vector length be N
 - (1). $V3 \leftarrow A$
 - (2). $V2 \leftarrow V0 + V1$
 - (3). $V4 \leftarrow V2 * V3$
- (3) Use vector chaining technology.

The execution time is:

$$\max\{(1+6+1), (1+6+1)\} + (1+7+1)+N-1 = N + 16$$



Improve the Performance of Vector Processor

3. Segmented Vector

What should I do if the length of the vector is greater than the length of the vector register?

- When the length of the vector is greater than the length of the vector register, the long vector must be divided into segments of fixed length, and then processed in loops, and only one vector segment is processed in each loop.
- The aboved is segmented vector technology.
 - It is controlled by system hardware and software and is transparent to programmers.



Improve the Performance of Vector Processor

4. Adopt multi-processor system

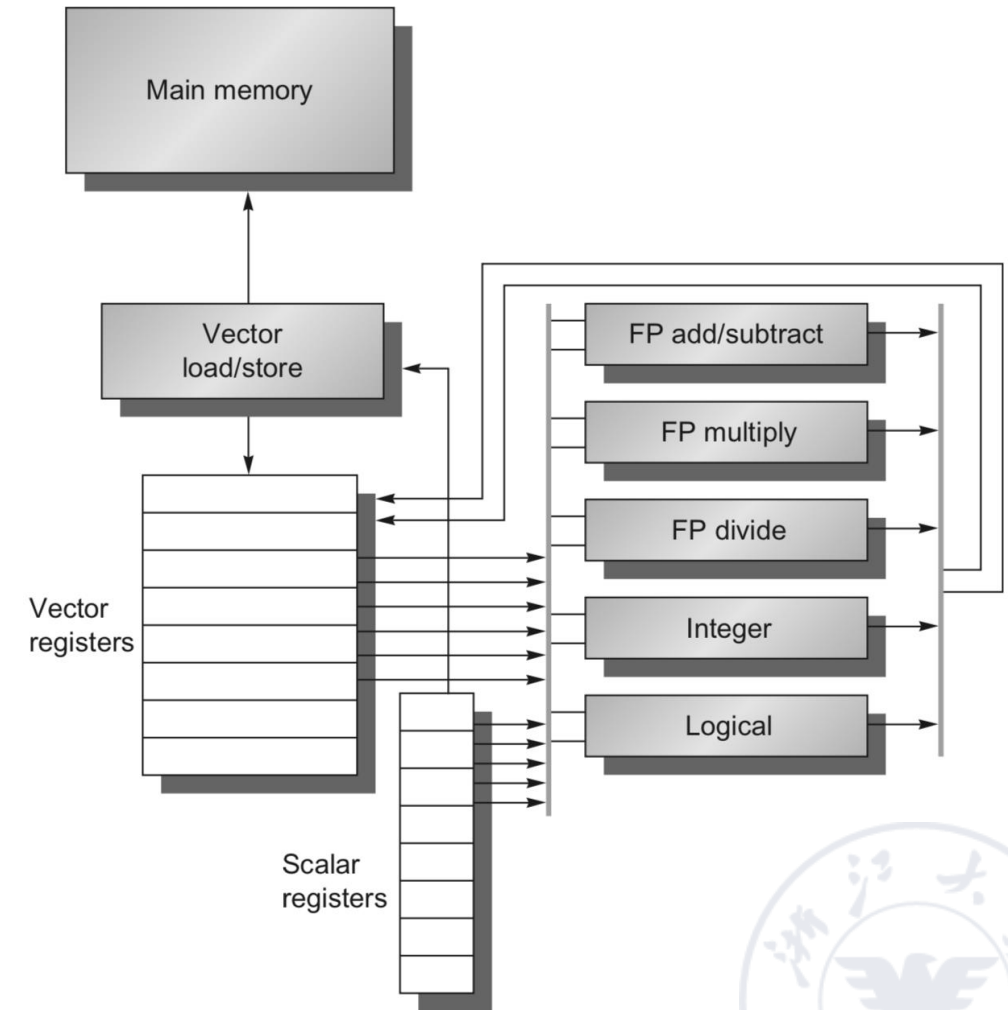
Many new vector processor systems have adopted a multi-processor system structure. E.g.

- CRAY-2
 - Contains 4 vector processors
 - Floating point calculation speed can reach up to 1800MFLOPS
- CRAY Y-MP, C90
 - Can contain up to 16 vector processors



RV64V

- Loosely based on Cray-1
- 32 62-bit vector registers
 - Register file has 16 read ports and 8 write ports
- Vector functional units
 - Fully pipelined
 - Data and control hazards are detected
- Vector load-store unit
 - Fully pipelined
 - One word per clock cycle after initial latency
- Scalar registers
 - 31 general-purpose registers
 - 32 floating-point registers



Example: DAXPY (Double Precision $a \times X$ plus Y)

RISC-V (scalar):

```

fld    f0,a           # Load scalar a
addi   x28,x5,#256     # Last address to load
Loop:  fld    f1,0(x5)  # Load X[i]
      fmul.d f1,f1,f0   # a × X[i]
      fld    f2,0(x6)  # Load Y[i]
      fadd.d f2,f2,f1   # a × X[i] + Y[i]
      fsd    f2,0(x6)  # Store into Y[i]
      addi   x5,x5,#8   # Increment index to X
      addi   x6,x6,#8   # Increment index to Y
      bne    x28,x5,Loop # Check if done

```

RV64V code:

```

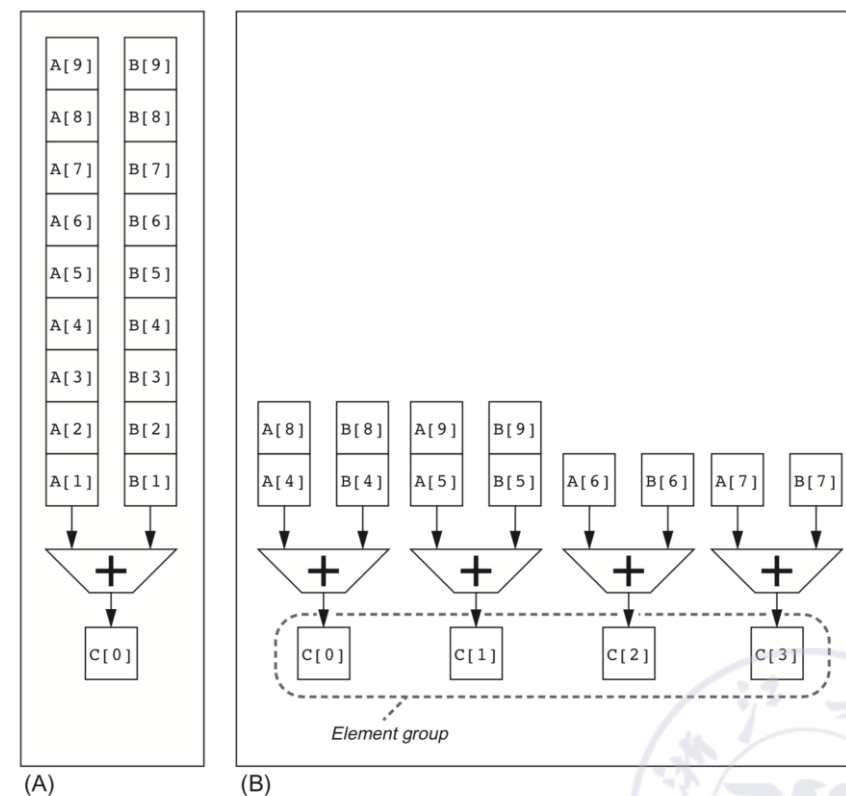
vsetdcfg  4*FP64      # Enable 4 DP FP vregs
fld       f0,a         # Load scalar a
vld       v0,x5        # Load vector X
vmul      v1,v0,f0      # Vector-scalar mult
vld       v2,x6        # Load vector Y
vadd      v3,v1,v2      # Vector-vector add
vst       v3,x6        # Store the sum
vdisable  # Disable vector regs

```

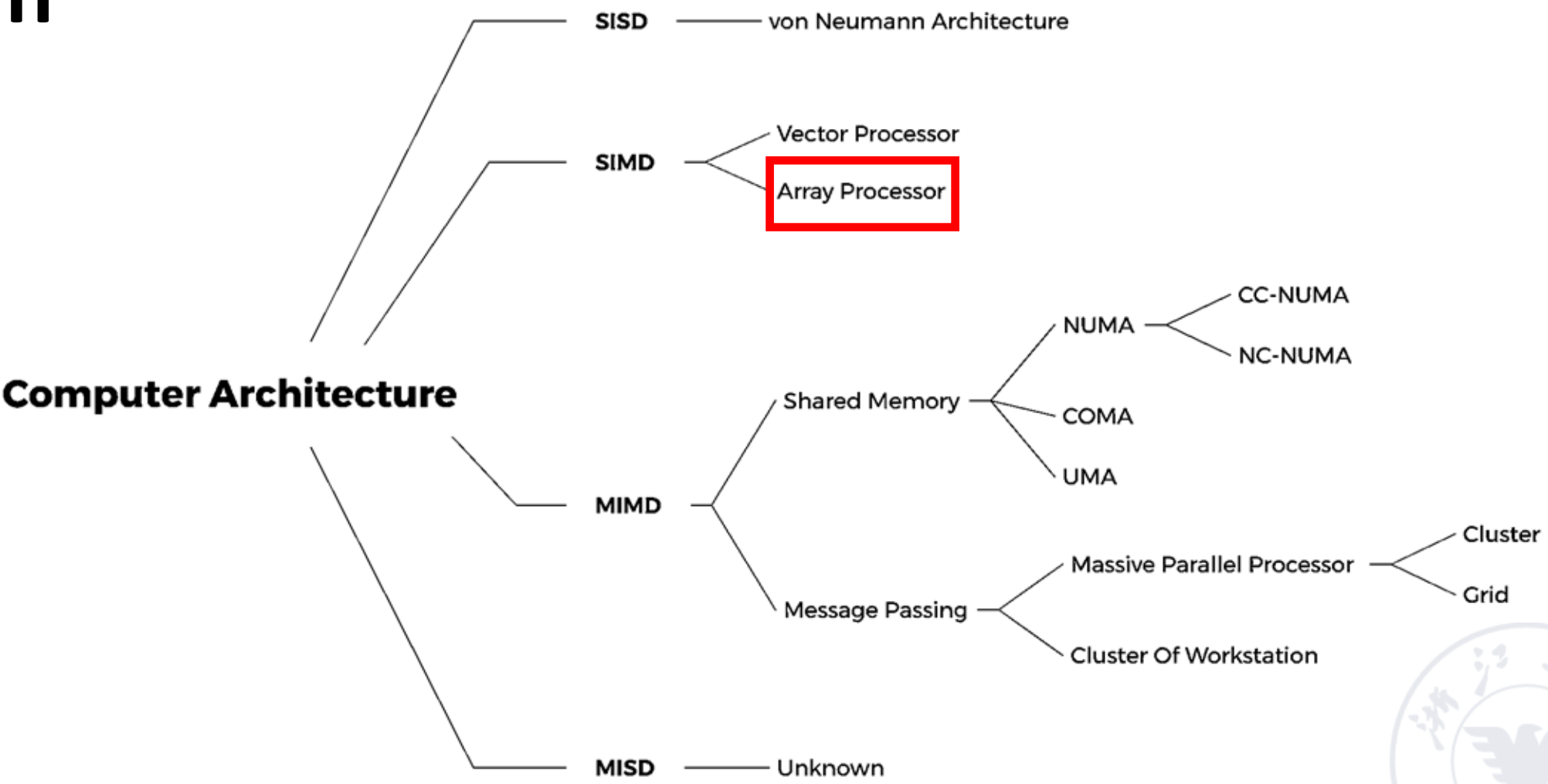


Multiple Lanes: Beyond One Element per Clock Cycle

- Element n of vector register A is “hardwired” to element n of vector register B
 - Allows for multiple hardware lanes
- The RV64V instruction set has the property that all vector arithmetic instructions only allow element N of one vector register to take part in operations with element N from other vector registers.
 - This dramatically simplifies the design of a highly parallel vector unit, which can be structured as multiple parallel *lanes*.



Flynn

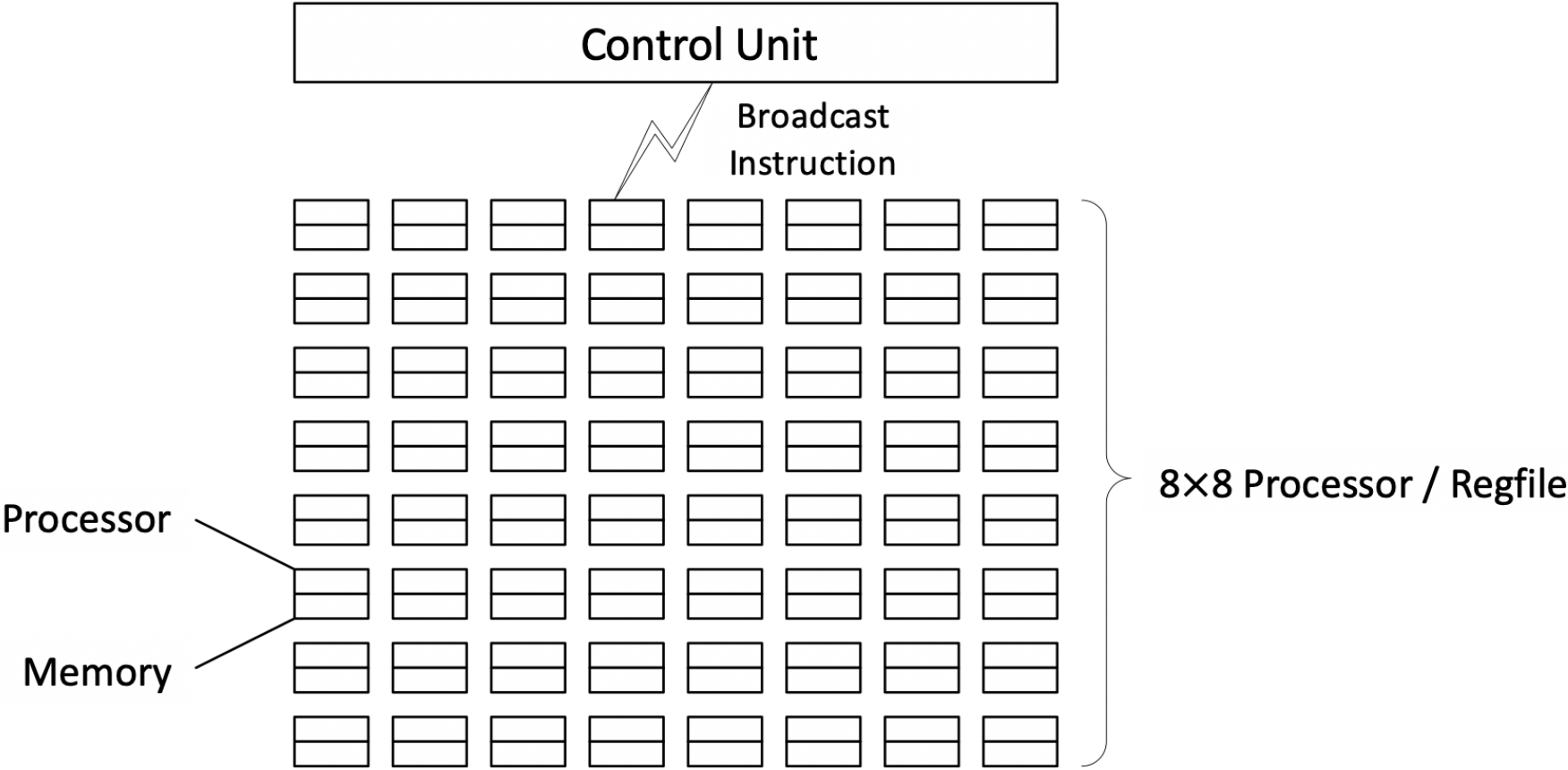


Array Processor

- N processing elements PE_0 to PE_{N-1} are repeatedly set.
- Interconnect in a certain way to form an array.
- Under the control of a single control unit, the operations specified by the same instruction are completed in parallel for the data allocated by the processing units.
- Array processors are sometimes called parallel processors.



ILLIAC IV



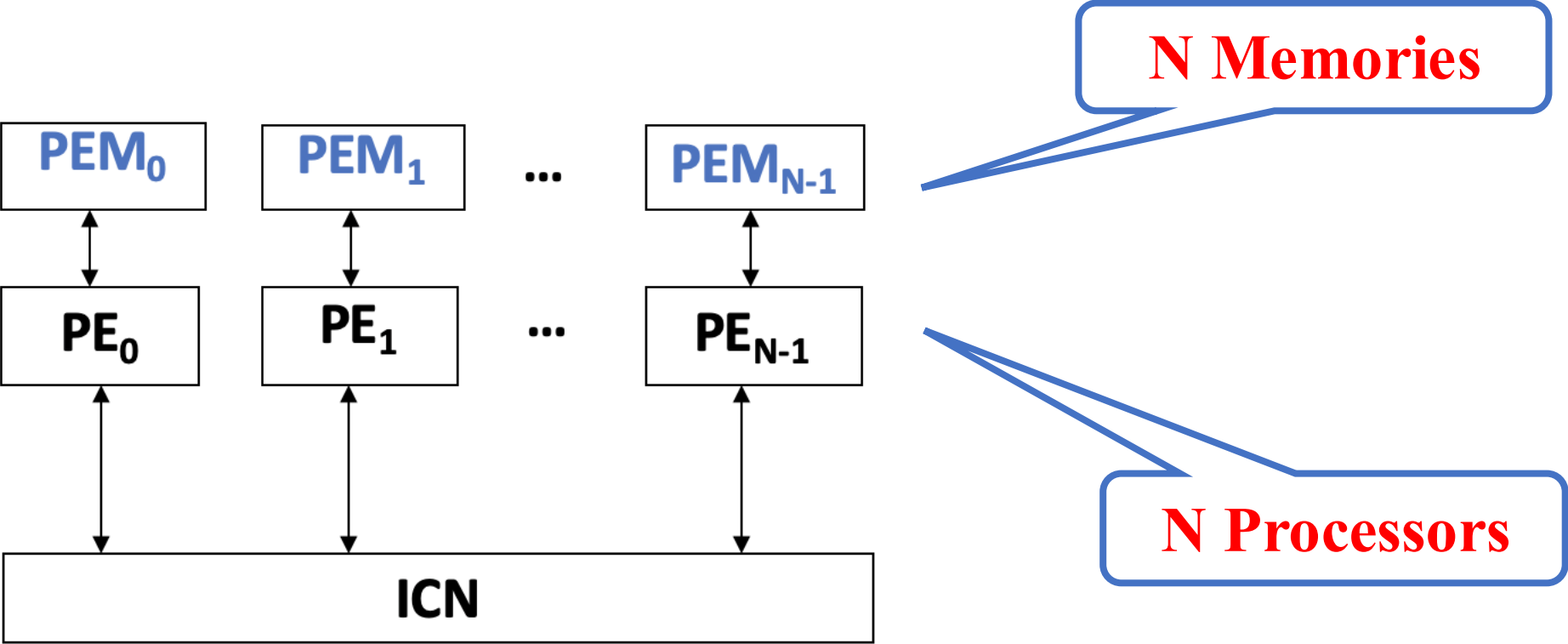
Basic Structure

According to the composition of the memory in the system, the array processor can be divided into two basic structures:

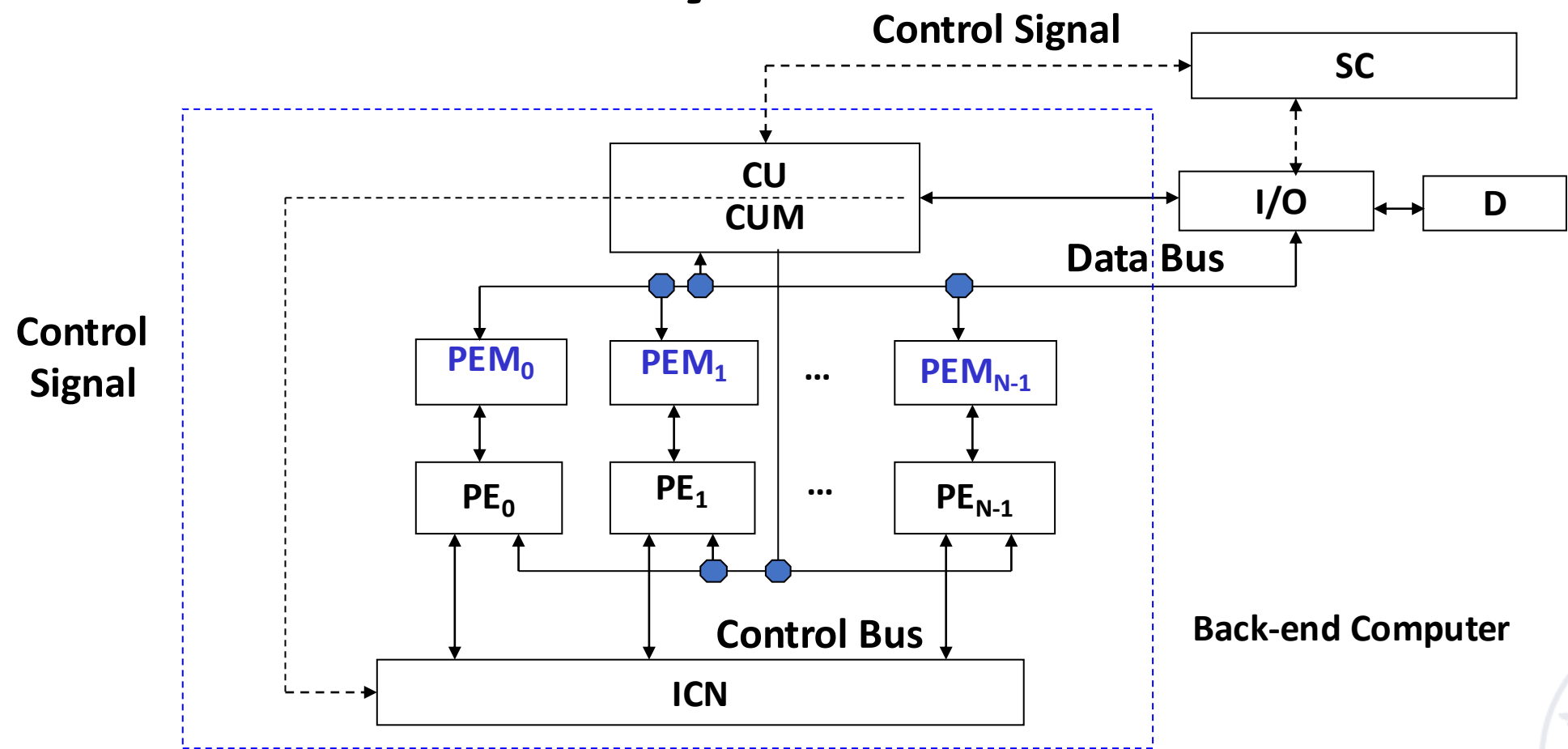
- Distributed memory
- Centralized shared memory



Distributed memory



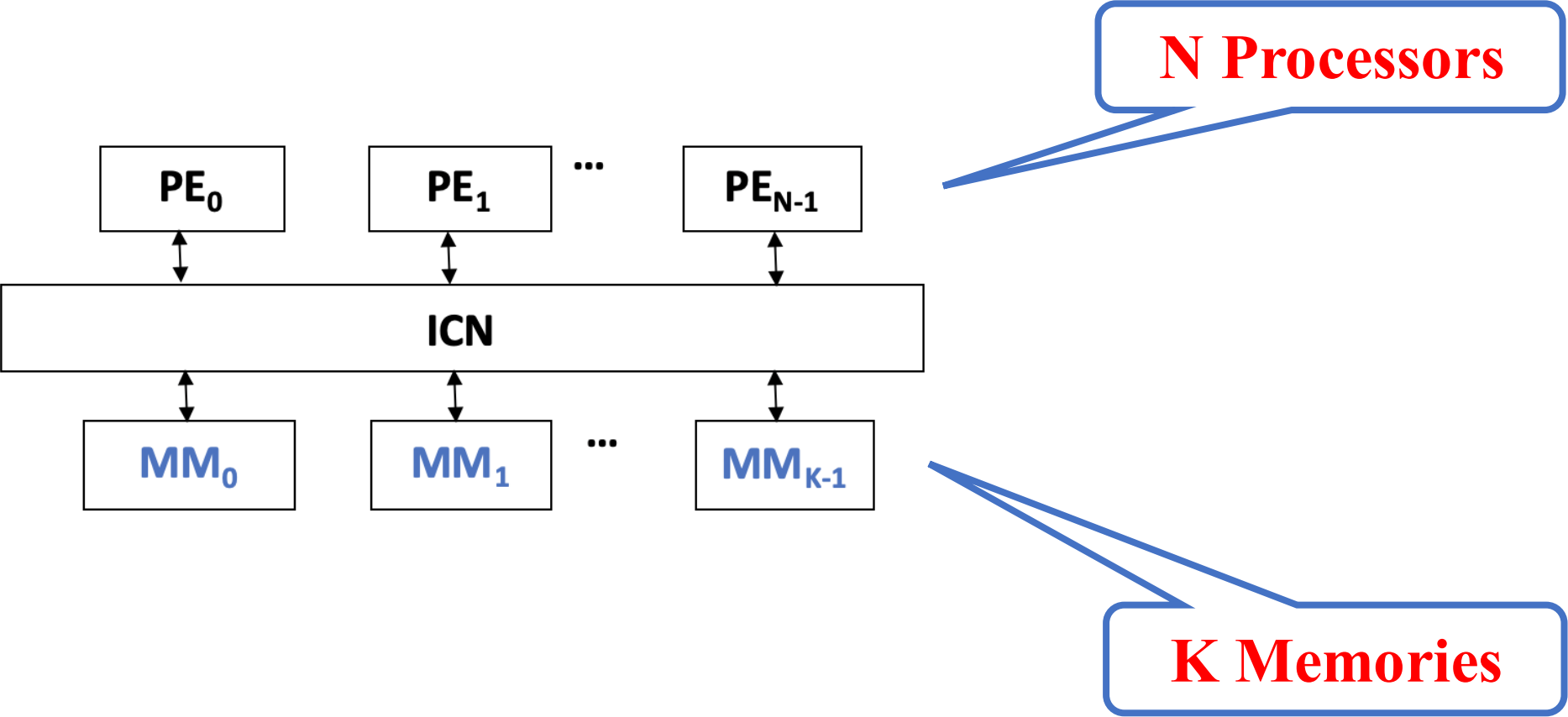
Distributed memory



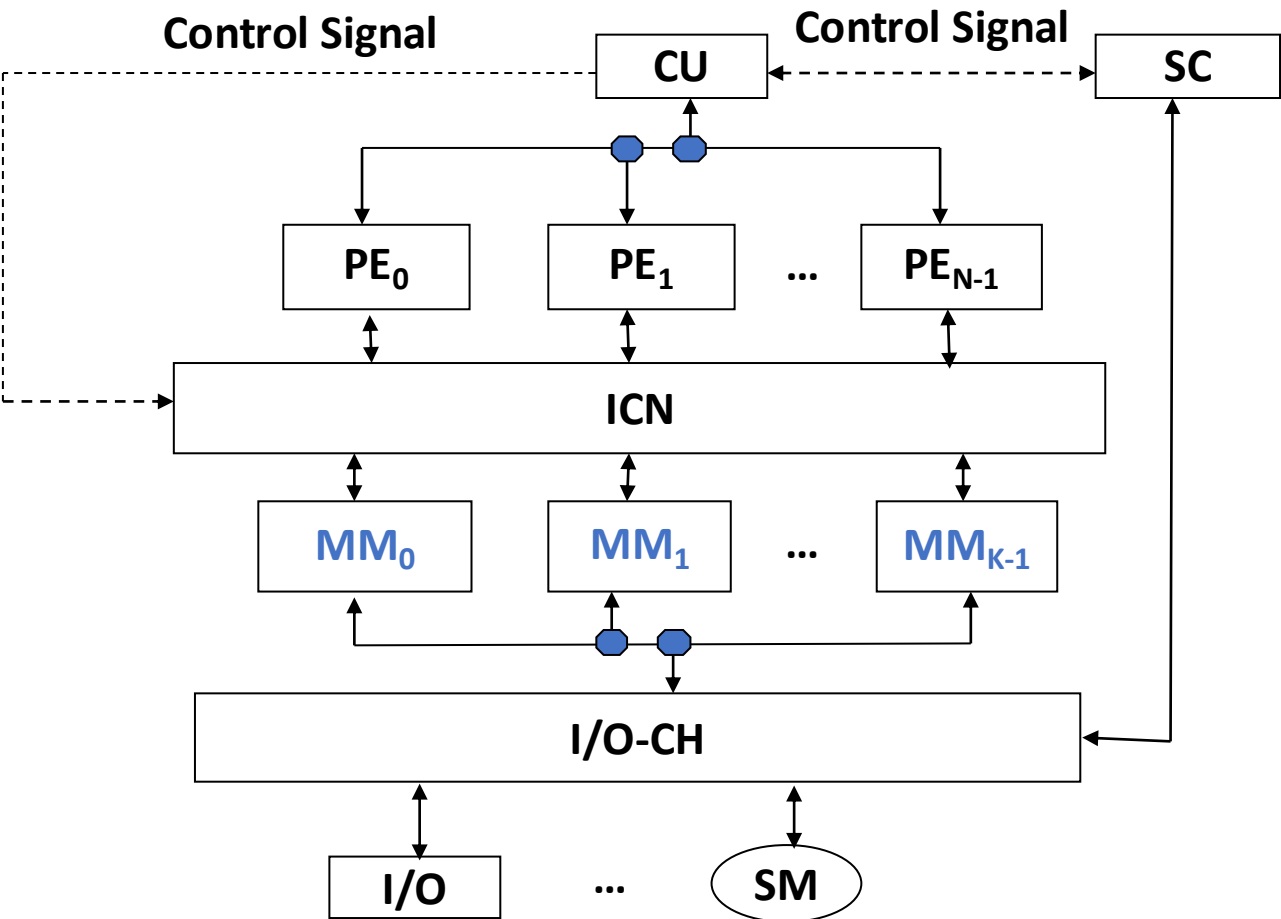
Distributed memory configuration is the mainstream of SIMD array processor.



Centralized shared memory



Centralized shared memory



The difference with distributed memory type:

- 1. the distribution of memory is different
- 2. the role of interconnection network is different



Question

- If a direct connection path is required between n processing units, how many pairs of connections are required?

$$P = C_n^2 = \frac{n(n - 1)}{2}$$

- Direct path → difficult to achieve
- Indirect path → try to achieve



Parallel computer design

- The **communication architecture** of the parallel computer is the core of the system
 - The **underlying** interconnection network;
 - Communication support provided by **upper-level** languages, software toolkits, compilers, and operating systems.
- The design problem of parallel computer system
 - Interconnection network of parallel computer systems
 - Performance problems of parallel computer systems
 - The software problems of parallel computer systems



Parallel computer design

- The interconnection network is inside the parallel computer system
- Definition: A network composed of switching units according to a certain topology and control mode to realize the interconnection between multiple processors or multiple functional components within a computer system.
- There are many similarities between computer networks in terms of working principles, concepts and terminology. The interconnection networks in some parallel computer systems are just high-speed Ethernet and ATM networks.



Interconnection network

- The interconnection network generally consists of the following five parts:

CPU, memory, interface, link and switch node

- Interface: It is a device that obtains information from CPU and memory and sends information to another CPU and memory. Typical devices are network interface cards.
- Link: A physical channel to transmit data bits. The link can be a cable, twisted pair or optical fiber, it can be serial or parallel, and each link has its maximum bandwidth. The link can be simplex half-duplex and full-duplex, the clock mechanism used by the link can be synchronous or asynchronous.
- Switch node: It is the information exchange and control station of the interconnected network. It is a device with multiple input ports and multiple output ports which is able to perform data buffer storage and path selection.



Some key points

- Topology of interconnection network
 - Static topology
 - Dynamic topology
- Timing mode of interconnection network
 - Synchronization system: Use a unified clock. Such as SIMD array processor
 - Asynchronous system: No uniform clock. Each processor in the system works independently
- Exchange method of interconnection network
 - Circuit switching
 - Packet switching
- Control Strategy of interconnection network
 - Centralized control mode: have a global controller
 - Distributed control mode: no global controller



Classification of interconnection network

- Static network
 - Static Networks refer to networks with fixed connection paths between nodes and this connection remains unchanged during program execution.
- Dynamic network
 - Dynamic Networks are composed of switches, which can dynamically change the connection status according to the requirements of the application. Such as bus, crossbar switch, multi-level switching network, etc.



Goal of interconnection network

- Through a limited number of connection methods, any two PEs can achieve information transmission in one step or a few steps to complete a certain problem-solving algorithm.



Goal of interconnection network

- **Single-stage interconnection network**: There are only a limited number of connections at the only level to realize information transmission between any two processing units.
- **Multi-stage interconnection network**: It is composed of multiple single-level networks in series to realize the connection between any two processing units.

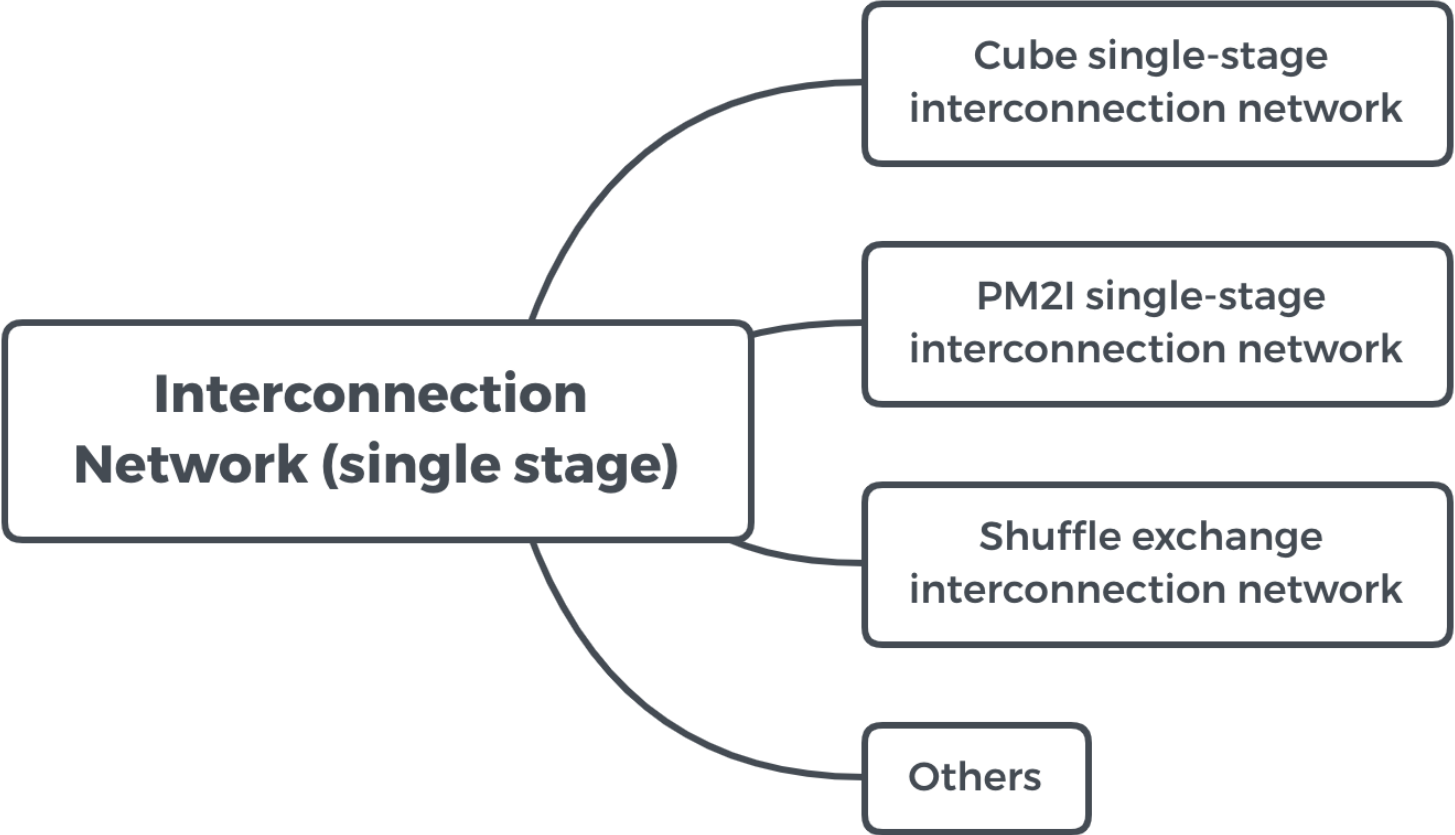


Goal of interconnection network

- For all N input terminals $(0, 1, \dots, j, \dots, N-1)$ of the interconnection network, there is a functional correspondence between the input terminal j and the output terminal $f(j)$.
- The input j and the output $f(j)$ generally use binary coding, and the corresponding function law can be found from the binary coding of the two. This corresponding law is the interconnection function.



Single-stage interconnection network



Cube single-stage interconnection network

- The N inputs and outputs are encoded with n -bit ($n = \log_2 N$) binary code $P_{n-1} \dots P_i \dots P_1 P_0$.
- There are n different interconnection functions:

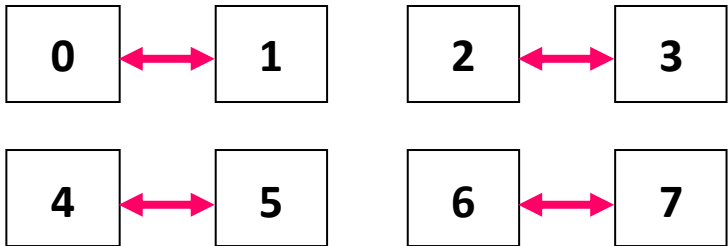
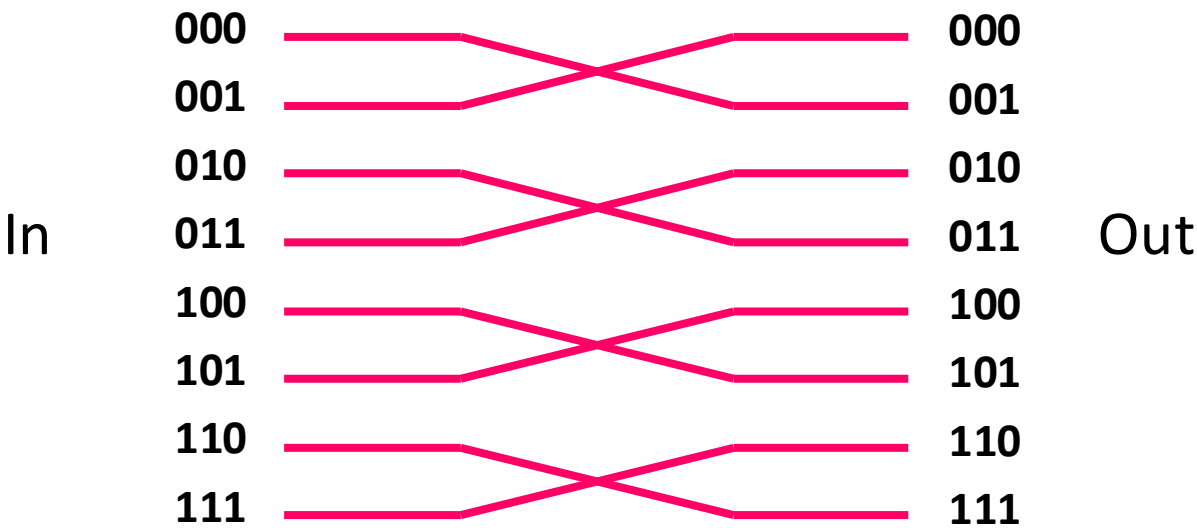
$$Cube_i(P_{n-1} \dots P_i \dots P_1 P_0) = P_{n-1} \dots \overline{P_i} \dots P_1 P_0$$



Cube single-stage interconnection network

Example:

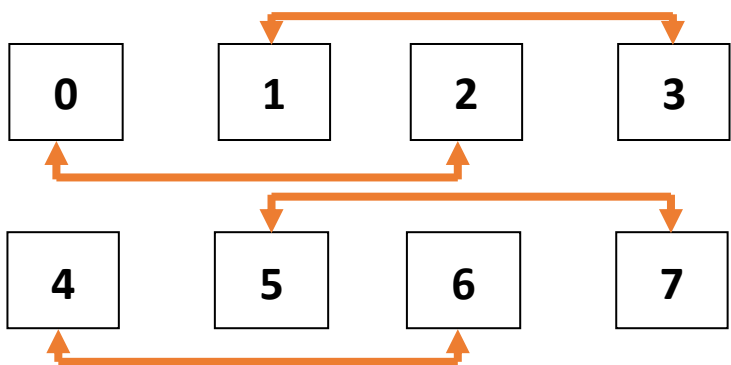
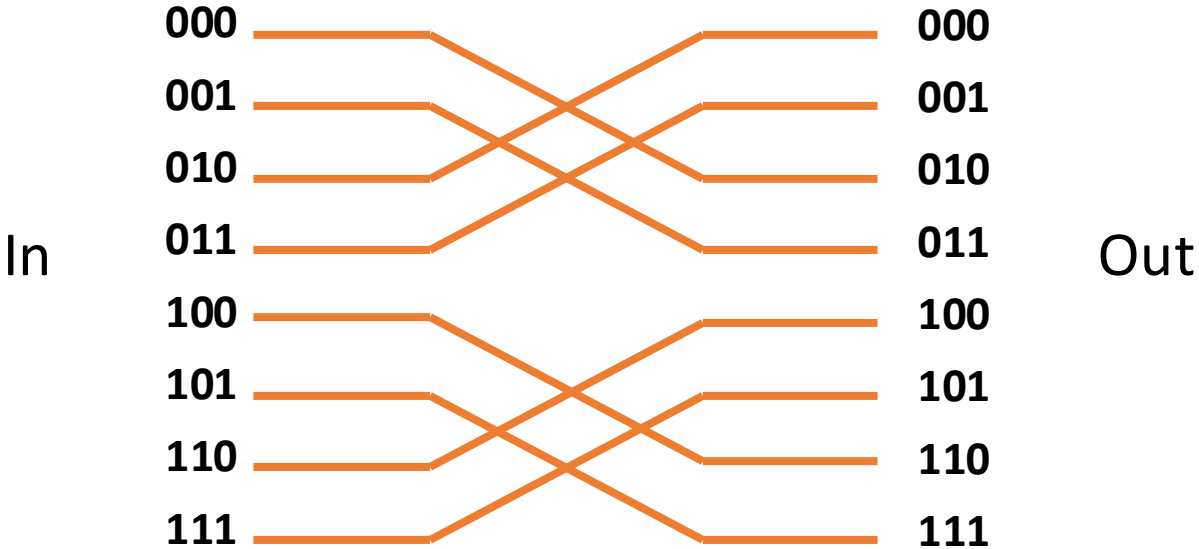
- In a cube single-stage interconnection network with 8 PEs, try to find out its interconnection functions.
- $cube_0(X_2X_1X_0) = (X_2X_1\overline{X_0})$



Cube single-stage interconnection network

Example:

- In a cube single-stage interconnection network with 8 PEs, try to find out its interconnection functions.
- $cube_1(X_2X_1X_0) = (X_2\overline{X_1}X_0)$

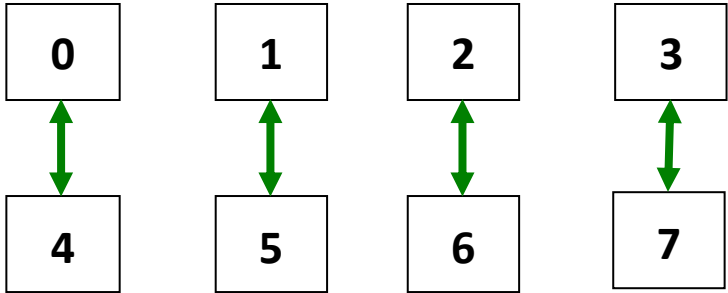
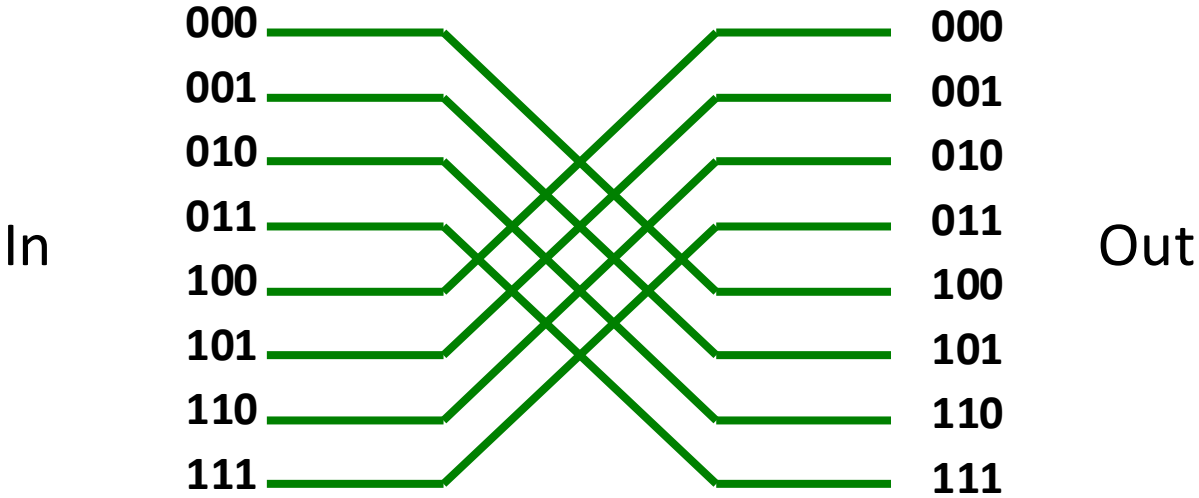


Cube single-stage interconnection network

Example:

- In a cube single-stage interconnection network with 8 PEs, try to find out its interconnection functions.

- $cube_2(X_2X_1X_0) = (\overline{X_2}X_1X_0)$



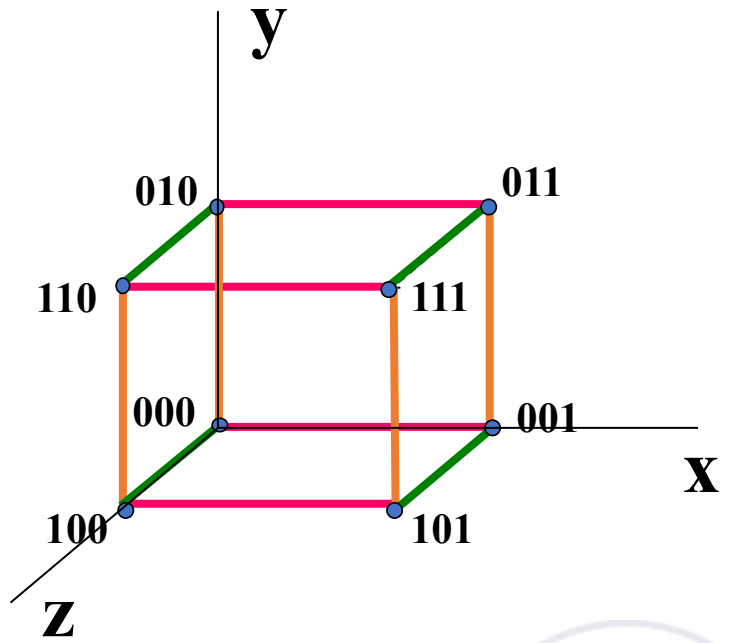
Cube single-stage interconnection network

- The 3D cube can transfer data between any two processing elements at most 3 times.

Cube₀

Cube₁

Cube₂



Hyper cube interconnection network

- When the dimension $n > 3$, it is called a hyper cube network.

$$Cube_i(P_{n-1} \cdots P_i \cdots P_1 P_0) = P_{n-1} \cdots \overline{P_i} \cdots P_1 P_0$$

- $0 \leq i \leq n-1$, P_i is the i -th bit of the binary code of the input port number.

The maximum distance of a single-stage n -dimensional cube network is n .

➔ After n transfers at most, data transfer between any two PEs can be realized.



PM2I single-stage interconnection network

- PM2I (**P**lus **M**inus 2^i) single-stage network
- Interconnection Function(2n):

$$PM2_{+i}(j) = (j + 2^i) \bmod N \text{ and } PM2_{-i}(j) = (j - 2^i) \bmod N$$

- N is the number of interconnection network nodes

$$0 \leq j \leq N - 1, 0 \leq i \leq \log_2 N - 1$$



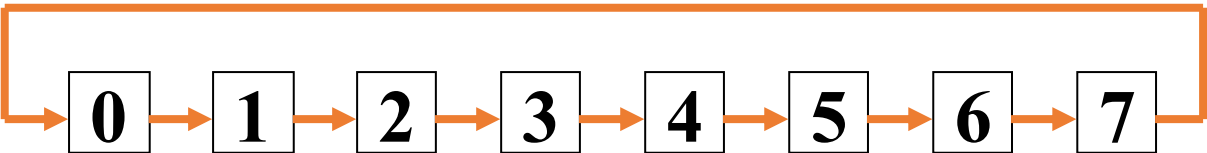
PM2I single-stage interconnection network

Example: $N = 8$

- Interconnection Function($i = 0$):

$$PM2_{+0}(j) = (j + 2^0) \bmod 8$$

- The topology of the interconnection network



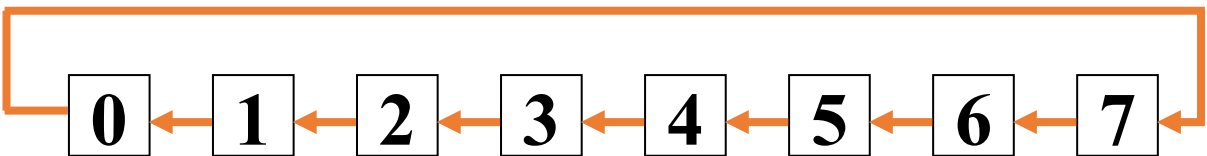
PM2I single-stage interconnection network

Example: $N = 8$

- Interconnection Function($i = 0$):

$$PM2_{-0}(j) = (j - 2^0) \bmod 8$$

- The topology of the interconnection network



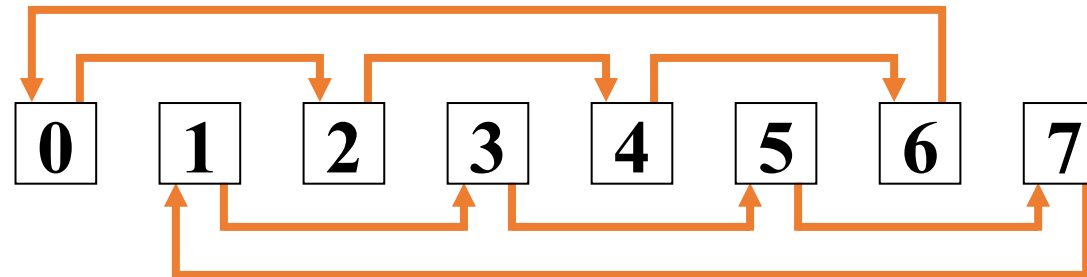
PM2I single-stage interconnection network

Example: $N = 8$

- Interconnection Function($i = 1$):

$$PM2_{+1}(j) = (j + 2^1) \bmod N$$

- The topology of the interconnection network



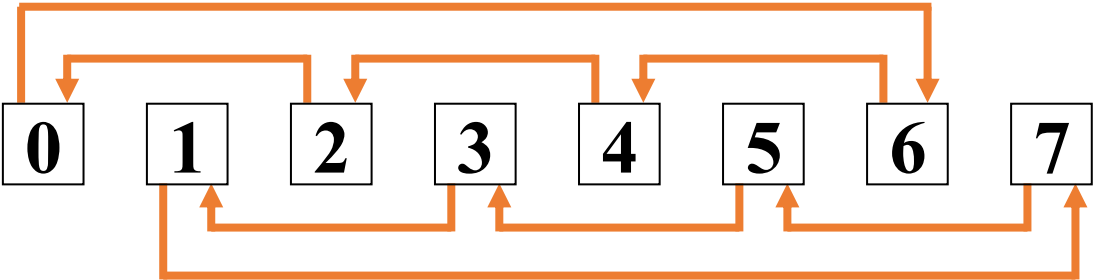
PM2I single-stage interconnection network

Example: $N = 8$

- Interconnection Function($i = 1$):

$$PM2_{-1}(j) = (j - 2^1) \bmod N$$

- The topology of the interconnection network



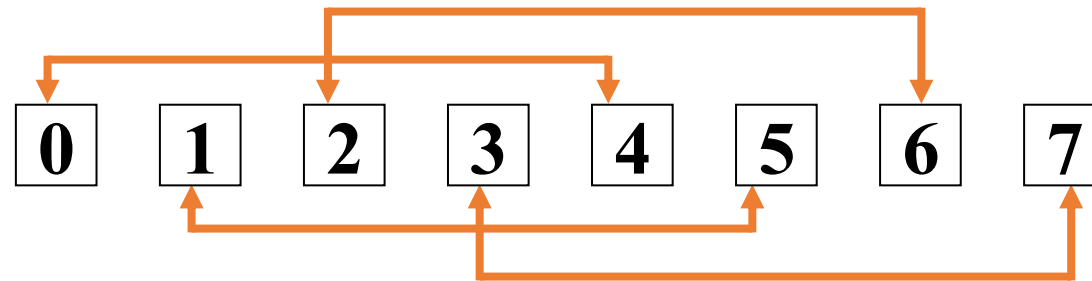
PM2I single-stage interconnection network

Example: $N = 8$

- Interconnection Function($i = 2$):

$$PM2_{\pm 2}(j) = (j \pm 2^2) \bmod N$$

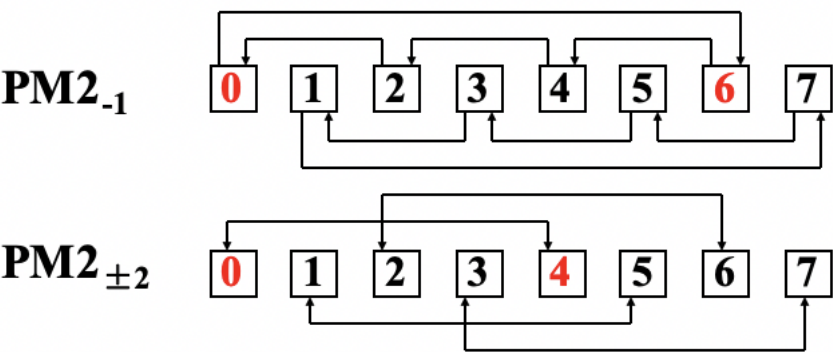
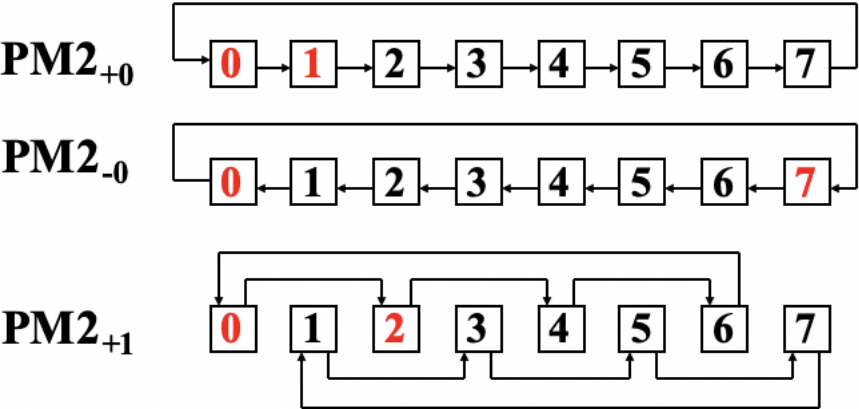
- The topology of the interconnection network



PM2I single-stage interconnection network

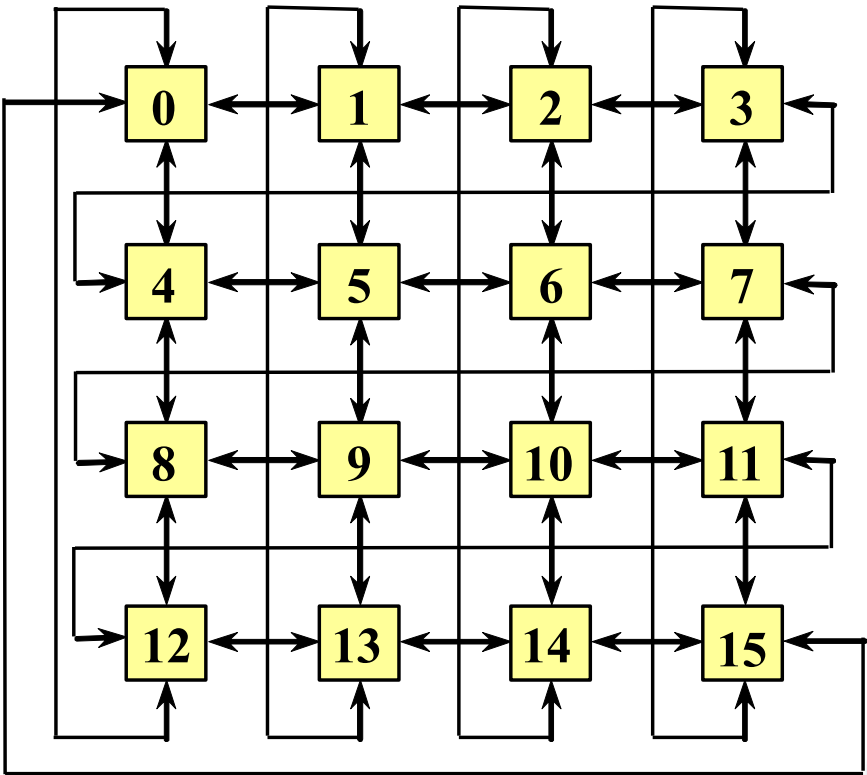
Example: $N = 8$

- Node 0 can reach nodes 1, 2, 4, 6, 7 in 1 step
- Node 0 can reach nodes 3, 5 in 2 steps



ILLIAC IV array processor

- It uses $PM2_{\pm 0}$ and $PM2_{\pm n/2}$ to form its interconnection network to realize the up, down, left and right interconnection between the PEs.



Shuffle exchange network

- Composed of two parts: **Shuffle** + **Exchange**
- N-dimension shuffle function:

$$\text{shuffle}(P_{n-1}P_{n-2} \dots P_1P_0) = P_{n-2} \dots P_1P_0P_{n-1}$$

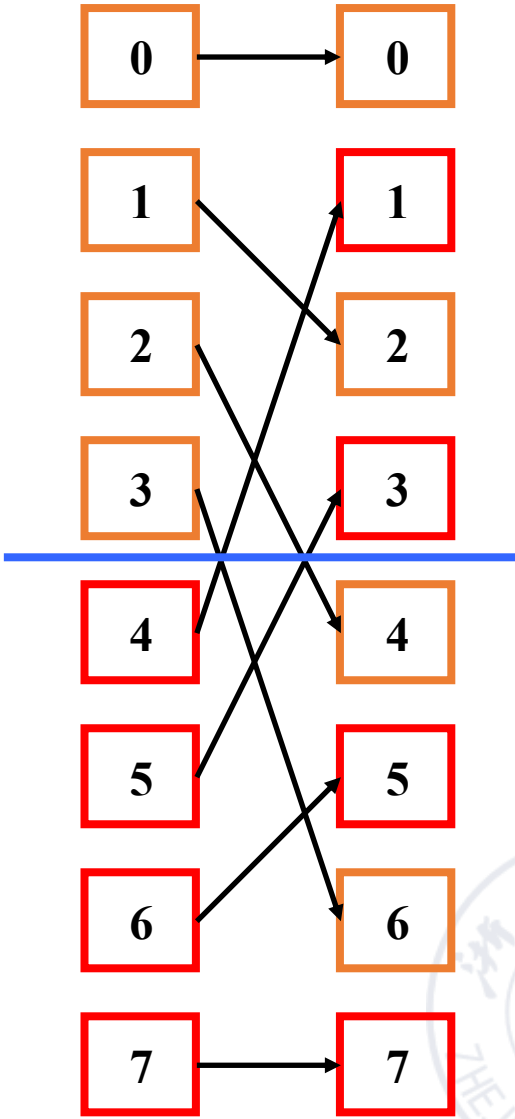
$n = \log_2 N$, $P_{n-1}P_{n-2} \dots P_1P_0$ are binary codes for the input number



Example: 8 PEs

Shuffle function: $shuffle(P_2P_1P_0) = P_1P_0P_2$

Initial Code	Before Shuffle	After 1 st Shuffle	Connected PE
0	000	000	0
1	001	010	2
2	010	100	4
3	011	110	6
4	100	001	1
5	101	011	3
6	110	101	5
7	111	111	7



Example: 8 PEs

Shuffle function: $shuffle(P_2P_1P_0) = P_1P_0P_2$

Initial Code	Before Shuffle	After 1 st Shuffle	Connected PE	After 2 nd Shuffle	Connected PE
0	000	000	0	000	0
1	001	010	2	100	4
2	010	100	4	001	1
3	011	110	6	101	5
4	100	001	1	010	2
5	101	011	3	110	6
6	110	101	5	011	3
7	111	111	7	111	7



Example: 8 PEs

Shuffle function: $shuffle(P_2P_1P_0) = P_1P_0P_2$

Initial Code	Before Shuffle	After 2 nd Shuffle	Connected PE	After 3 rd Shuffle	Connected PE
0	000	000	0	000	0
1	001	100	4	001	1
2	010	001	1	010	2
3	011	101	5	011	3
4	100	010	2	100	4
5	101	110	6	101	5
6	110	011	3	110	6
7	111	111	7	111	7

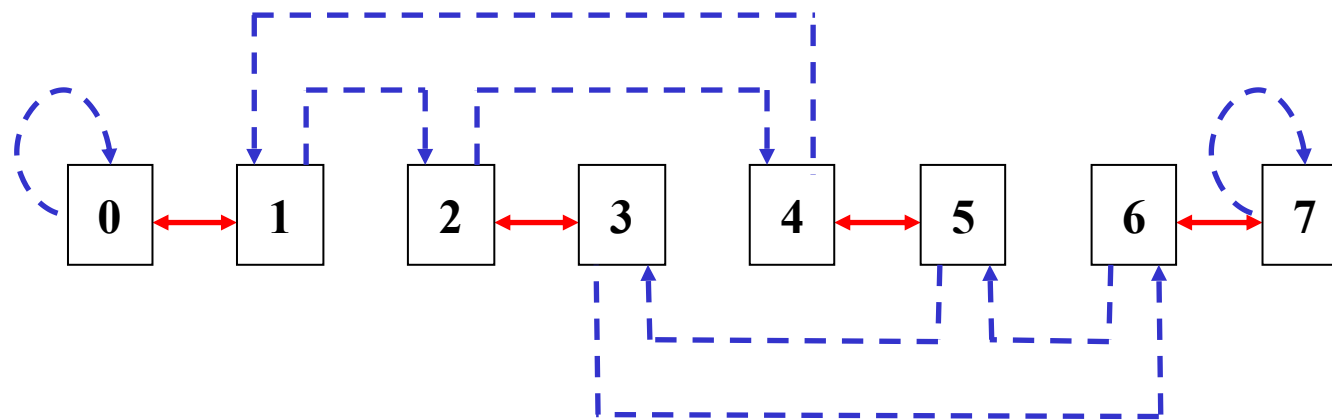


Shuffle exchange network

An important feature of the Shuffle function:

- After N times of shuffling, all N PEs are restored to the initial arrangement order.

$$\text{shuffle}(P_2 P_1 P_0) = P_1 P_0 P_2$$



3 exchanges and 2 shuffles



Shuffle exchange network

The maximum distance of shuffle exchange network

- From the nodes numbered all “0” to all “1”, the data transmission needs to go through at most:

n exchanges and $n-1$ shuffles

- Maximum distance: $2n-1$

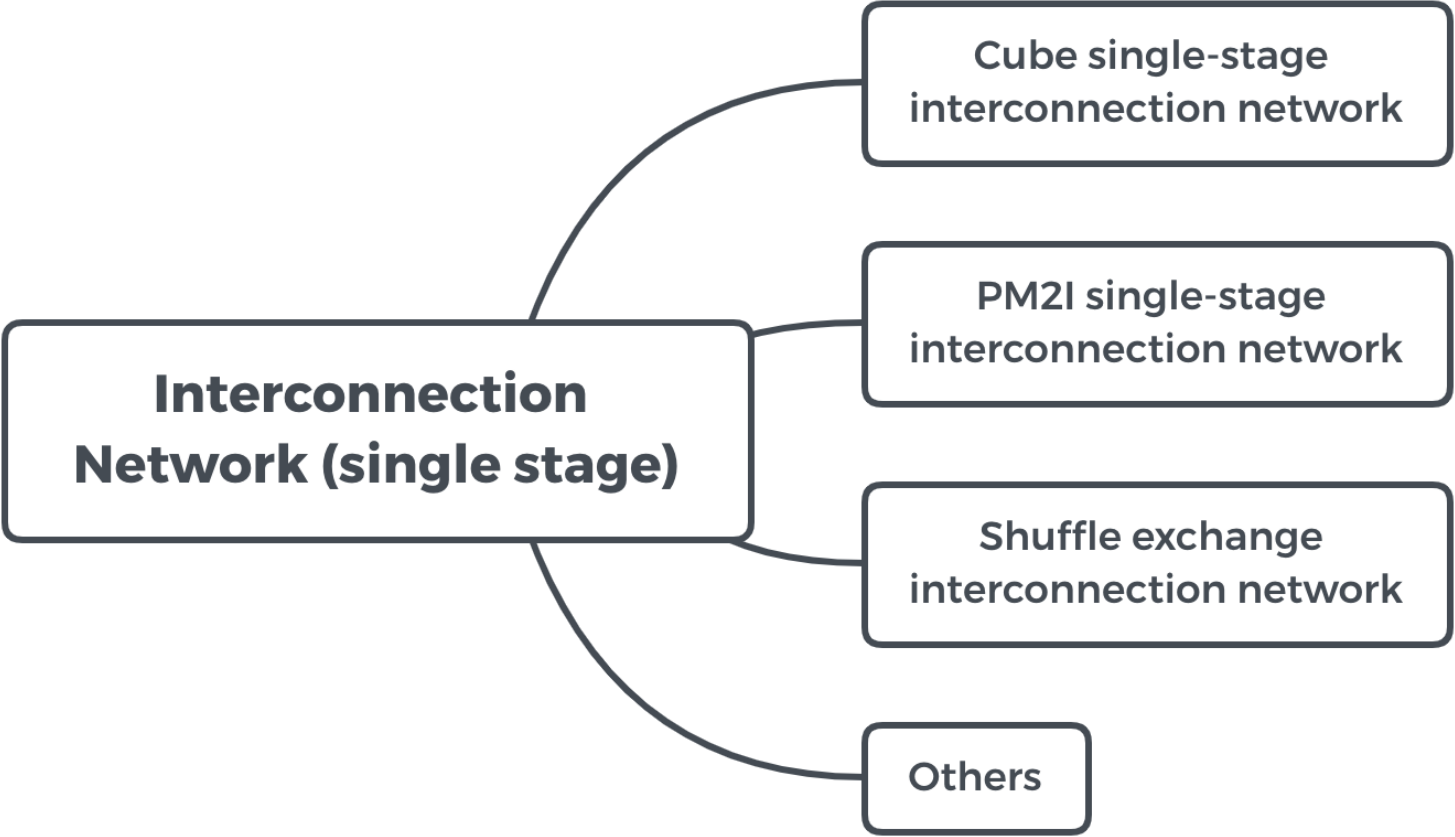


Features of single-stage interconnection networks

- Simple structure and low cost
- Flexible connection to meet the needs of algorithms and applications
- The number of transfer steps is small, and the array operation speed is improved
- Regularity and modularity are better to enhance the scalability of the system
- Facilitate large-scale integration



Single-stage interconnection network



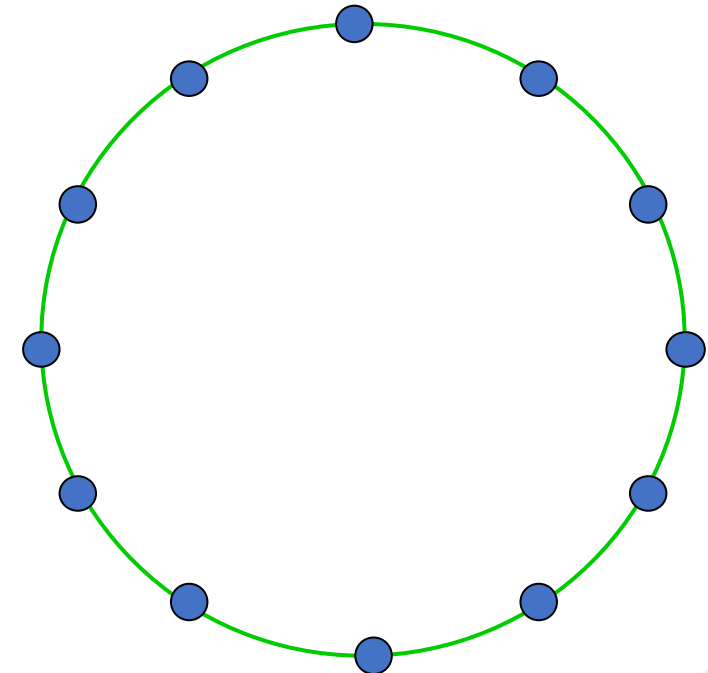
Linear array

- For a linear array of N nodes, there are $N-1$ links, the diameter is $N-1$ (the maximum distance between any two points), the degree is 2 , asymmetry, and the equal division width is 1 .
- When N is large, the communication efficiency is very low.



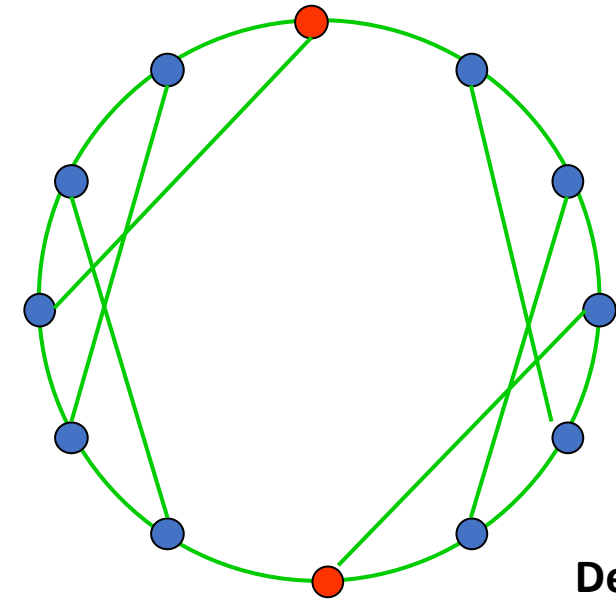
Circular array

- For a ring of N nodes, consider the data transfer direction of adjacent nodes:
- Two-way ring: The number of links is N , the diameter is $\lfloor N/2 \rfloor$, the degree is 2 , symmetrical, and the equal width is 2 .
- Unidirectional ring: The number of links is N , the diameter is $N-1$, the degree is 2 , symmetrical, and the equal width is 2 .

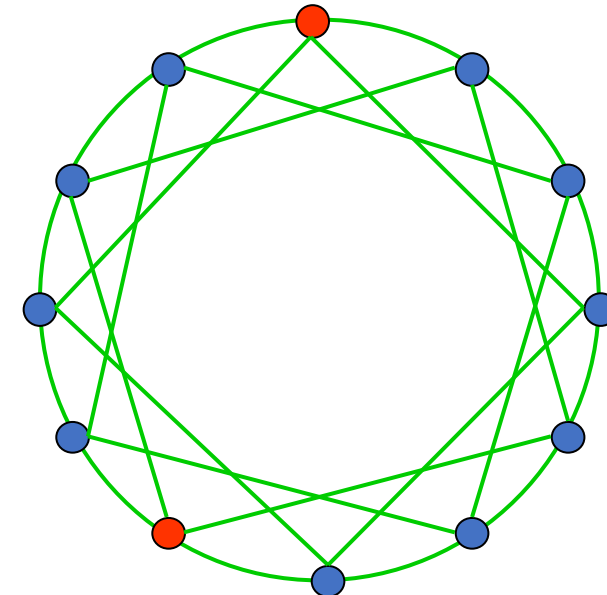


Loop with chord array

- For the two-way loop with chord with 12 nodes in the figure
- **Node degree is 3**: The number of links is 18, the diameter is **4** (for example, the red node), the degree is **3**, asymmetric, and the equal width is **2**.
- **Node degree is 4**: The number of links is 24, the diameter is **3** (for example, the red node), the degree is **4**, symmetrical, and the equal width is **8**.



Degree = 3

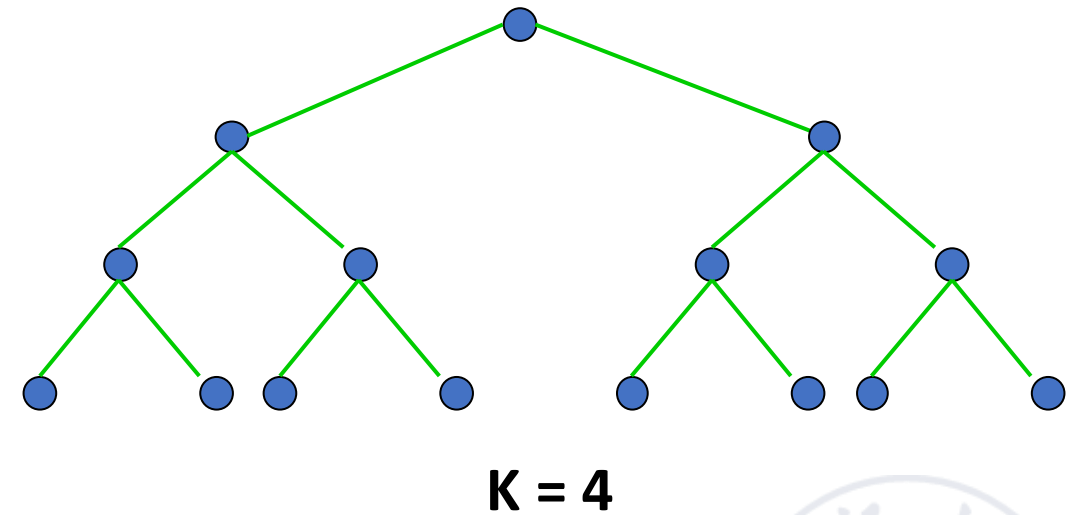


Degree = 4

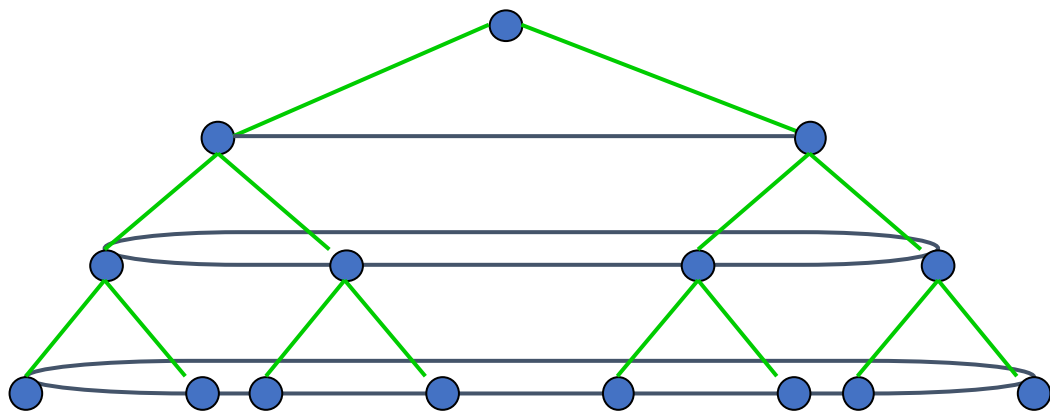


Tree array

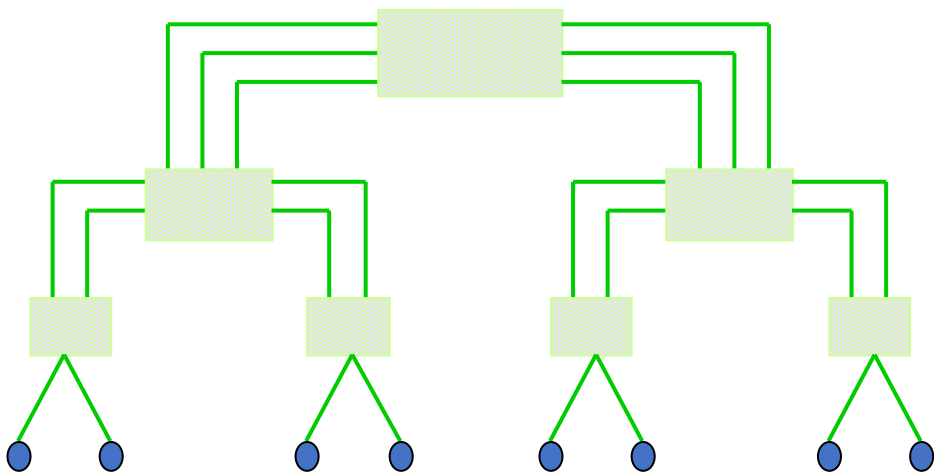
- A K -level complete binary tree should have $N = 2^K - 1$ nodes, the maximum node degree is 3 , and the diameter is $2(K-1)$ (that is, any leaf node on the right to any leaf node on the left), asymmetrical, equal width is 1 .



Extensions of tree array



Tree with loop

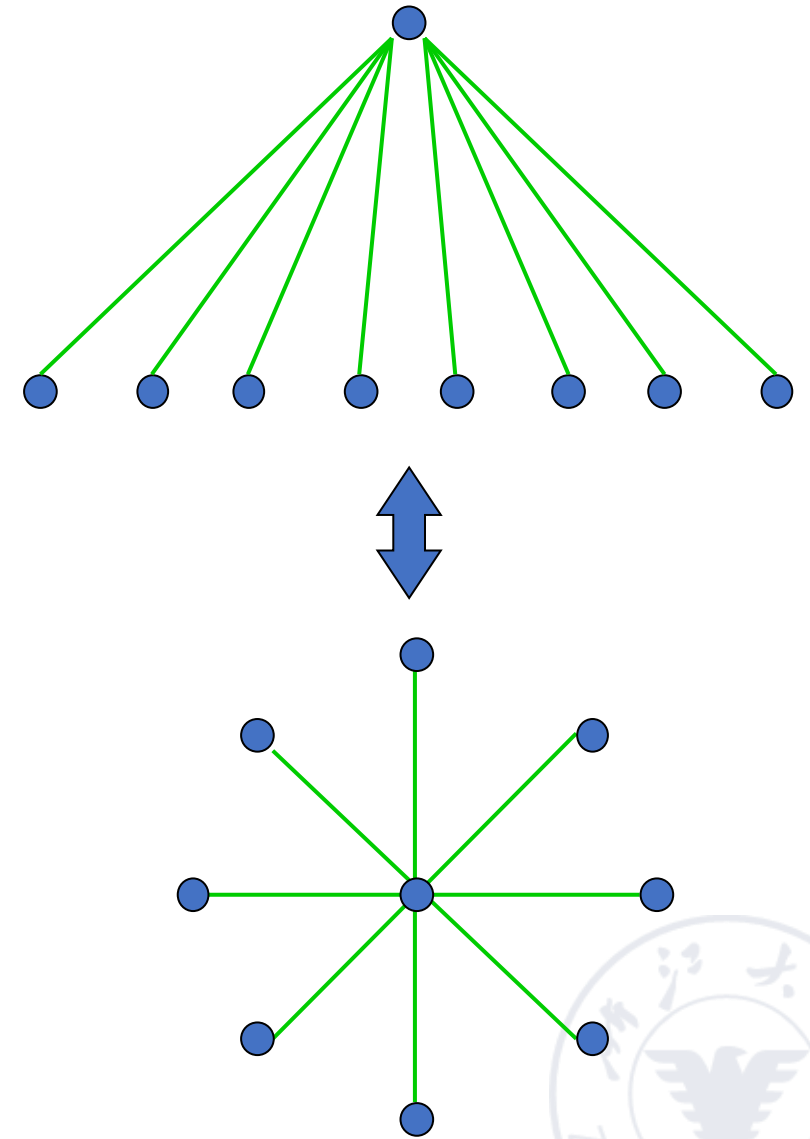


Binary fat tree



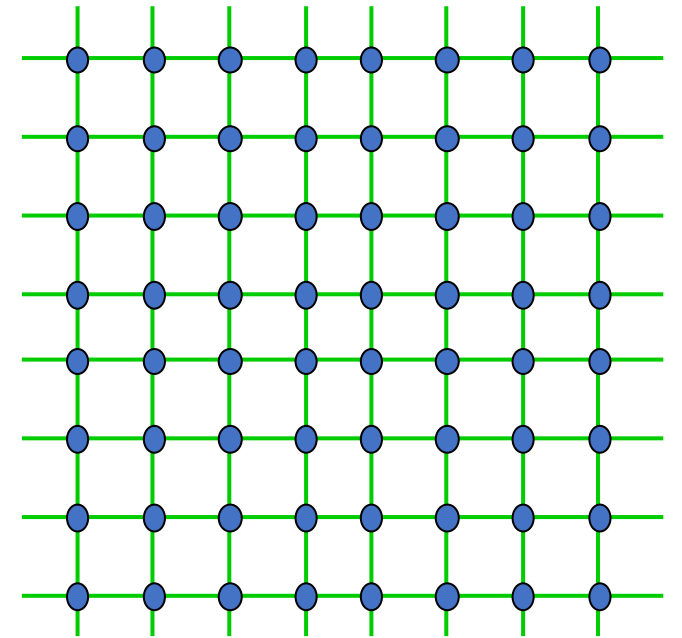
Star array

- The star array is actually a two-level tree (as shown on the right).
- A star network with N nodes has $N-1$ links, a diameter of 2 , a maximum node degree of $N-1$, asymmetrical, and an equal division width of 1 .



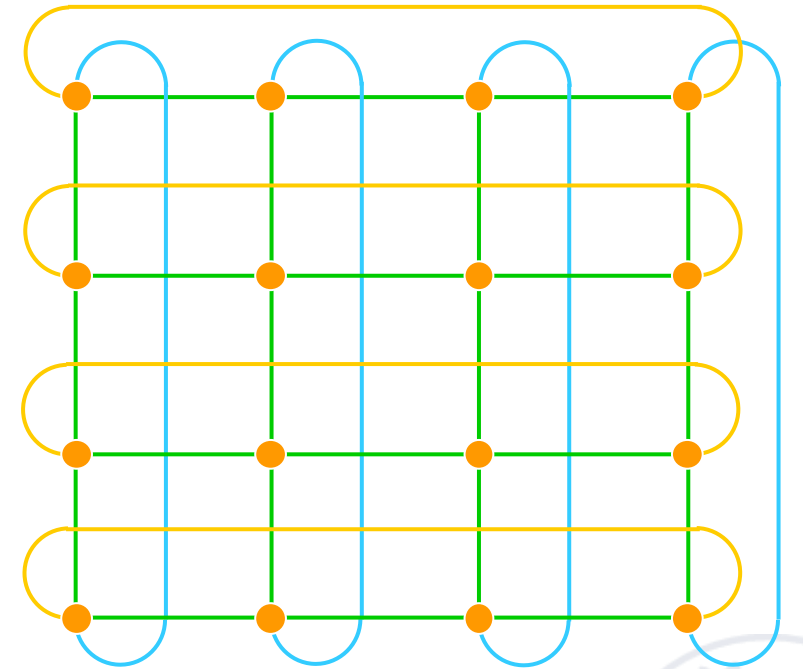
Grid

- An $r \times r$ network with N nodes has $2N-2r$ links, a diameter of $2(r-1)$, a node degree of 4, asymmetrical, and an equal division width of r .
- $r = \sqrt{N}$



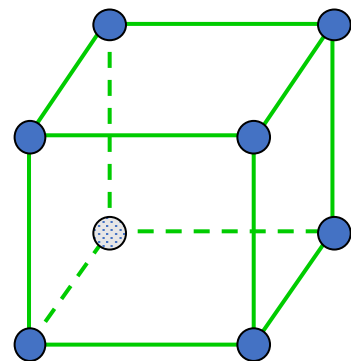
2D torus

- An $r \times r$ network with N nodes has $2N$ links, a diameter of $2\lfloor r/2 \rfloor$, a node degree of 4, asymmetrical.
- $r = \sqrt{N}$

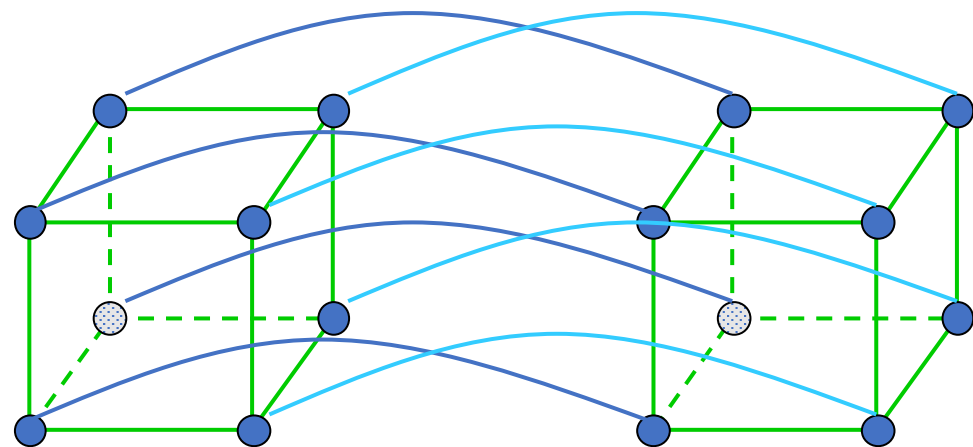


Hypercube

- An n-cube consists of $N = 2^n$ nodes, which are distributed in n dimensions, with two nodes in each dimension. The diameter is n , the degree is n , symmetrical.



3-cube

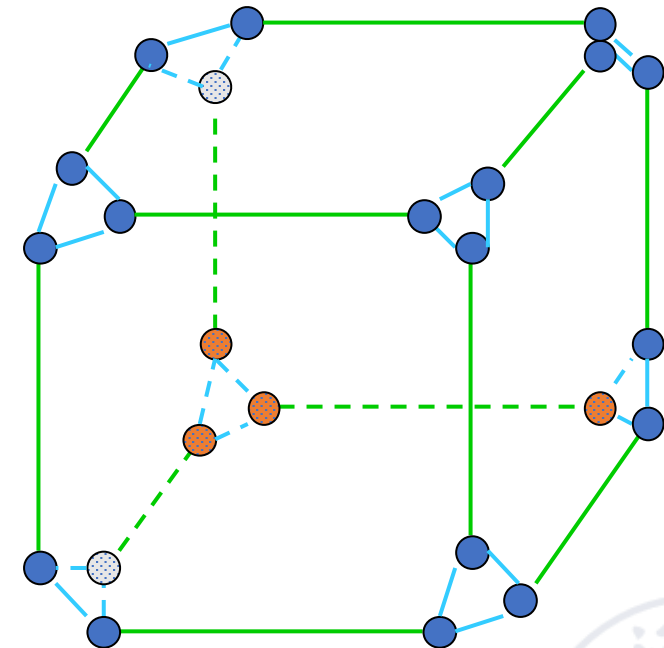


4-cube



Cube with loop

- An n -cube with loop is composed of $N = 2^n$ node rings, each node ring is a ring with n nodes, so the total number of nodes is $n2^n$, the node degree is 3 , and it is symmetric.



Summary

	Scale	Degree	Diameter	Width	Symmetry	Link
Linear	N	2	$N-1$	1	No	$N-1$
Circular	N	2	$\lfloor N/2 \rfloor$	1	Yes	N
Binary tree	N	3	$2(\lceil \log N \rceil - 1)$	1	No	$N-1$
Star	N	$N-1$	2	$N/2$	No	$N-1$
Grid	N	4	$2(\sqrt{N} - 1)$	$\sqrt{N}$	No	$2(N-\sqrt{N})$
2D torus	N	4	$2\lceil \sqrt{N}/2 \rceil$	$2\sqrt{N}$	Yes	$2N$
Hypercube	$N = 2^n$	N	n	$N/2$	Yes	$nN/2$
Cube with loop	$N = k2^k$	3	$2k-1+\lceil k/2 \rceil$	$N/2^k$	Yes	$3N/2$



Network characteristics

- The connection in the dynamic network is not fixed and can be changed as needed during the execution of the program.
- The switching elements of the network are active, and the link can be reconstructed by setting the state of these switches.
- Only the switching elements on the network boundary can be connected to the processor.
- Dynamic networks mainly include [buses](#), [crossbar switches](#), and [multi-stage interconnection networks](#).



- Bus

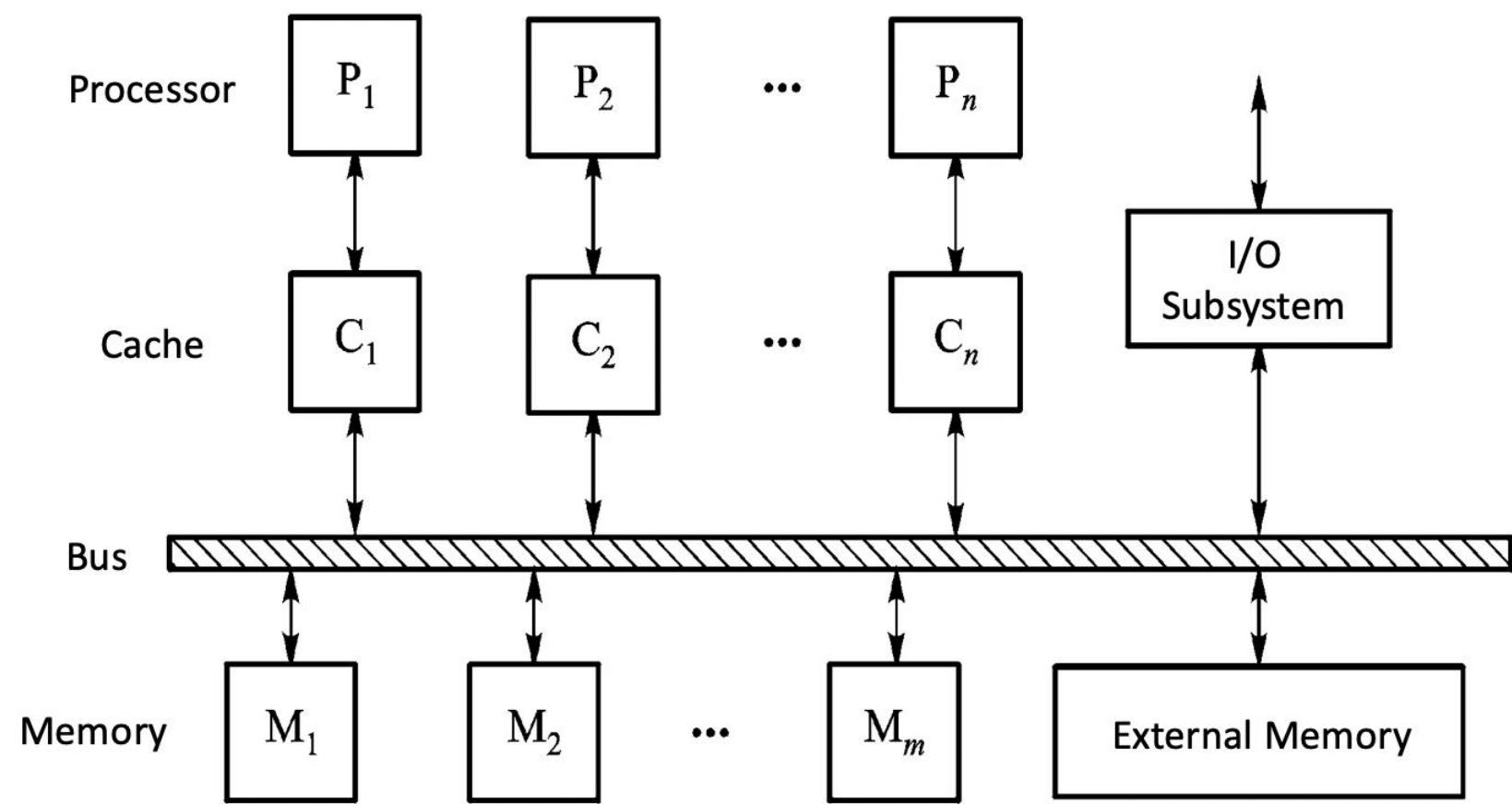
- The bus is actually a set of wires and sockets that connect peripheral devices such as processors, memories, and I/O.
- It can only be used to transfer data between a pair of source and destination at a certain time.
- When there are multiple pairs of source and destination requests to use the bus, bus arbitration is required. When the number of CPUs is large, the bus contention is serious (≤ 32).

- The difference between linear array and bus

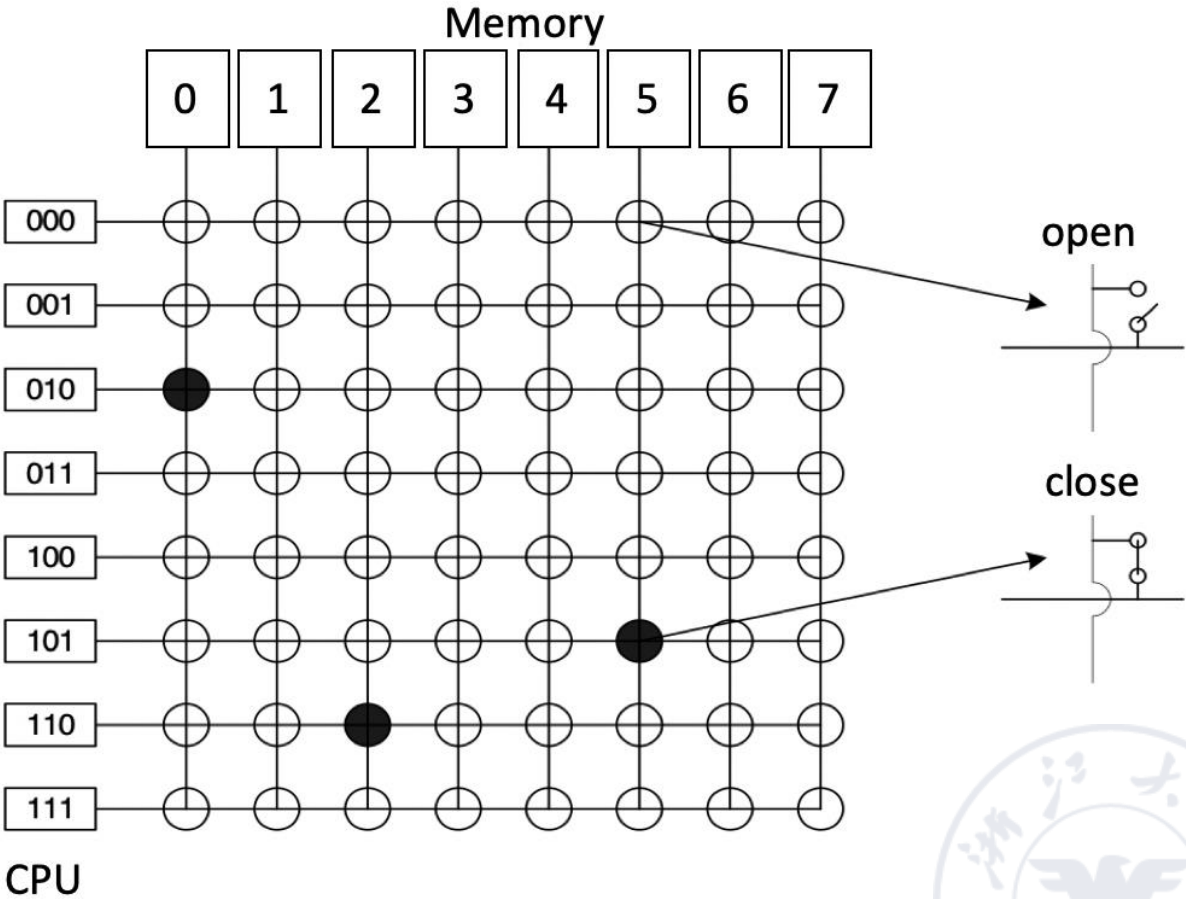
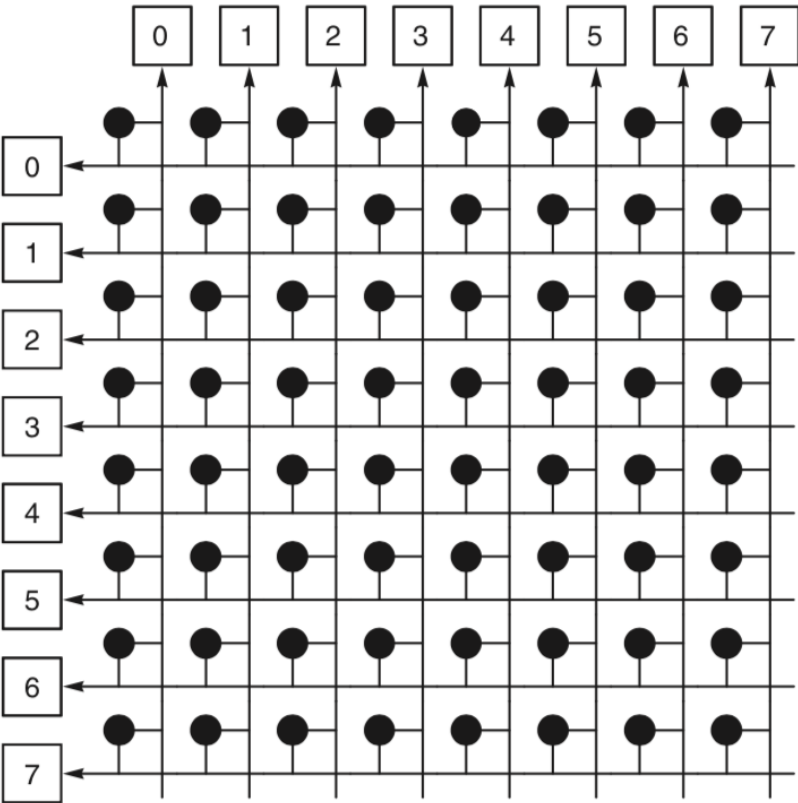
- Linear array: Allows different source and destination nodes to concurrently use different parts of the system.
- Bus: Realize time-division characteristics by switching many nodes connected to it. Only one pair of nodes are transmitting data at the same time.



A multiprocessor system connected by bus

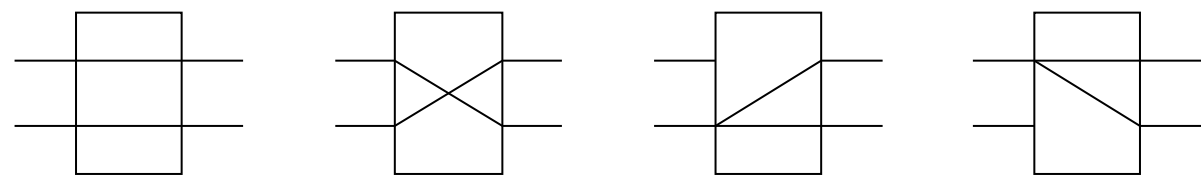


Crosspoint switches



Multi-stage interconnection network

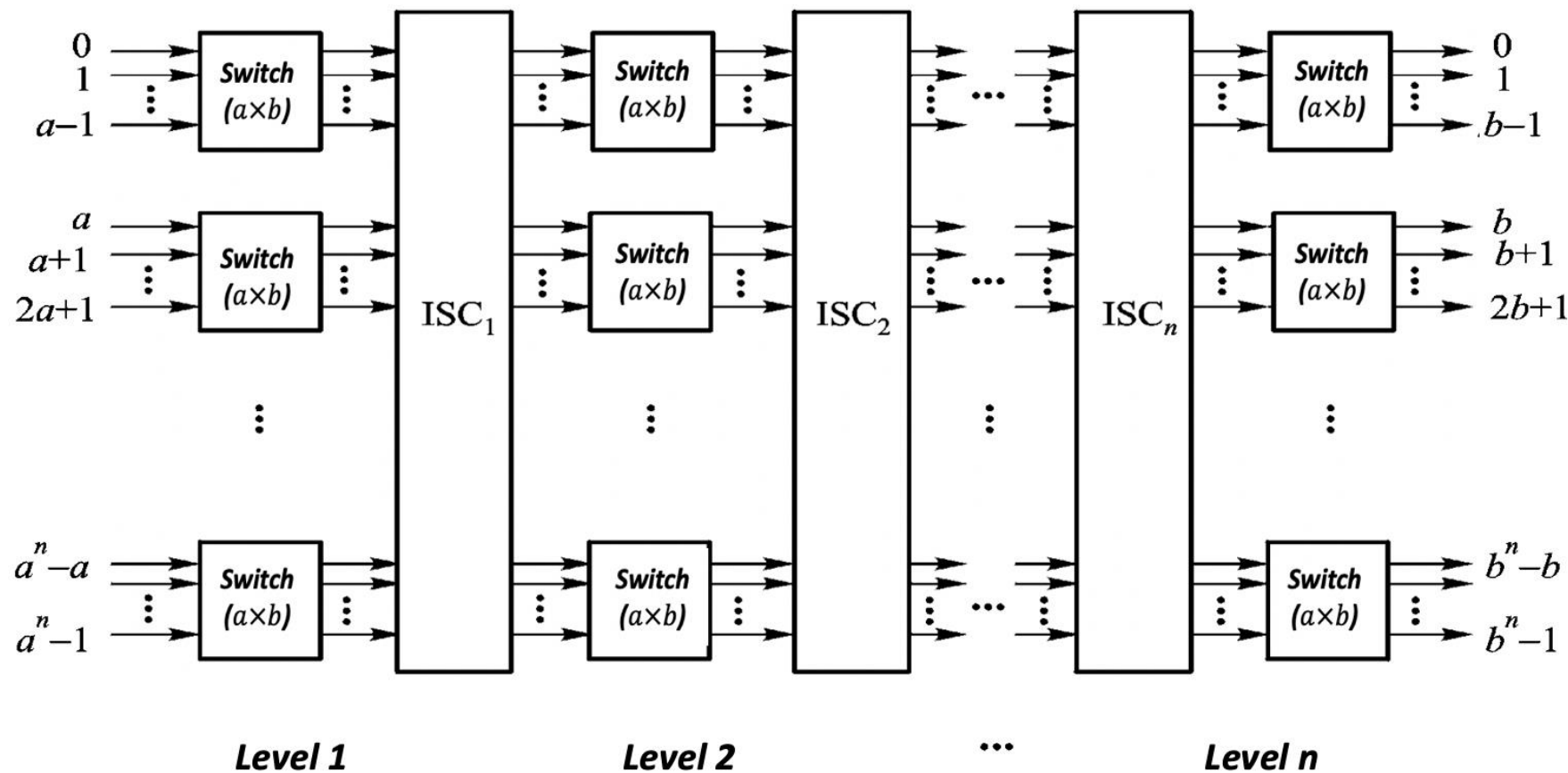
- Switch unit: a switch unit with m inputs and m outputs is recorded as a switch unit of $m \times m$, $m = 2^k$. Common ones are 2×2 , 4×4 , 8×8 and so on.
- According to the function of the switch unit, 2×2 can be divided into two-function and four-function switches.



Module Size	Legal Status	Exchange Connection
2×2	2	2
4×4	256	24
8×8	16,777,216	40,320
$N \times N$	N^N	$N!$



Multi-stage interconnection network



Multi-stage interconnection network

In order to realize the connection between any PEs, you can use:

- Circular interconnection network:
 - A single-stage network can be used to cycle through multiple times
- Multi-level interconnection network:
 - Connect multiple single-stage interconnection networks in series
- Multi-level circular interconnection network:
 - Based on multi-level interconnection network, recycle through multiple times



Multi-stage interconnection network

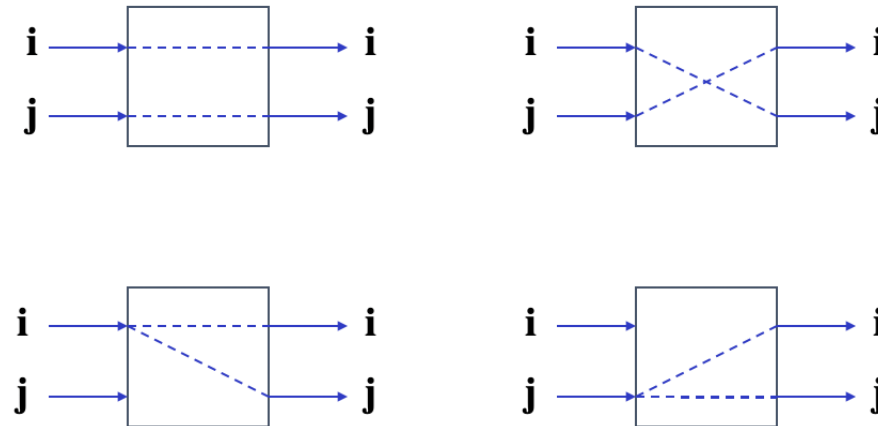
The differences between different multi-level networks are:

- Switch function
- Switch control method
- Topology



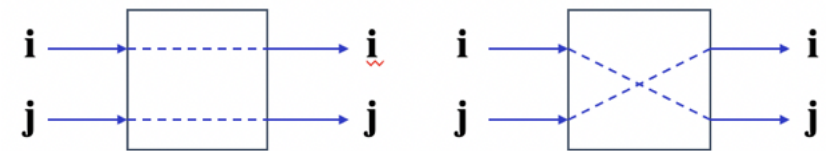
Switching unit

- A switching unit with two inputs and two outputs is the basic component of various multi-level interconnection networks.
- Regardless of input and output, the one located above are represented by i , and the one located below are represented by j .
- The statuses of switching unit:
 - Straight
 - Exchange
 - Upper broadcast
 - Lower broadcast

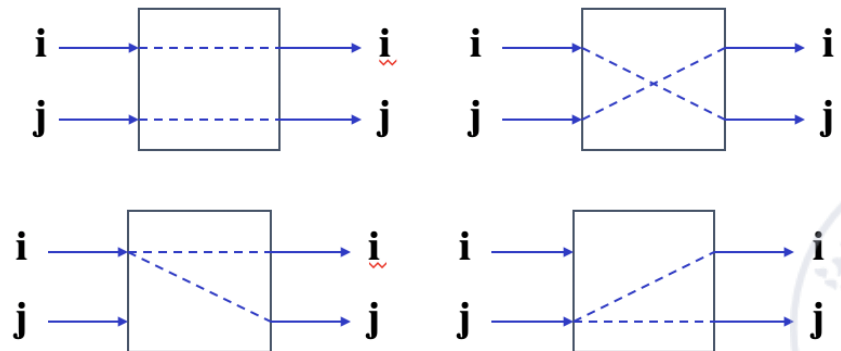


Multi-stage interconnection network

- Two-function switching unit
 - Straight and exchange unit only

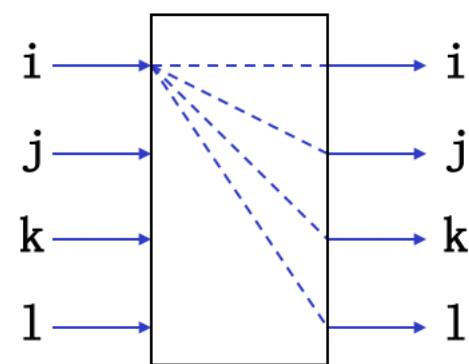


- Four-function switching unit
 - Include four basic functions

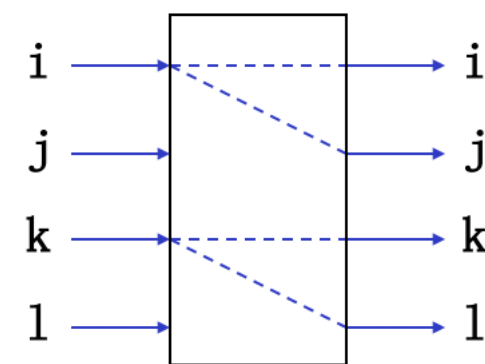


Multi-stage interconnection network

- Multi-end switching unit
 - Add broadcast and multicast module



Broadcast



Multicast



Multi-stage interconnection network

- Topology: The mode in which the input/output ends of the switches at all levels are connected to each other.
 - Multi-stage cube
 - Multi-stage shuffle exchange
 - Multi-stage PM2I
 - The combination of the above three networks



Characteristics of multi-stage cube interconnection network

- Switch unit: two-function switch unit
- Control mode: stage, part stage and unit control
- Topology: cube structure

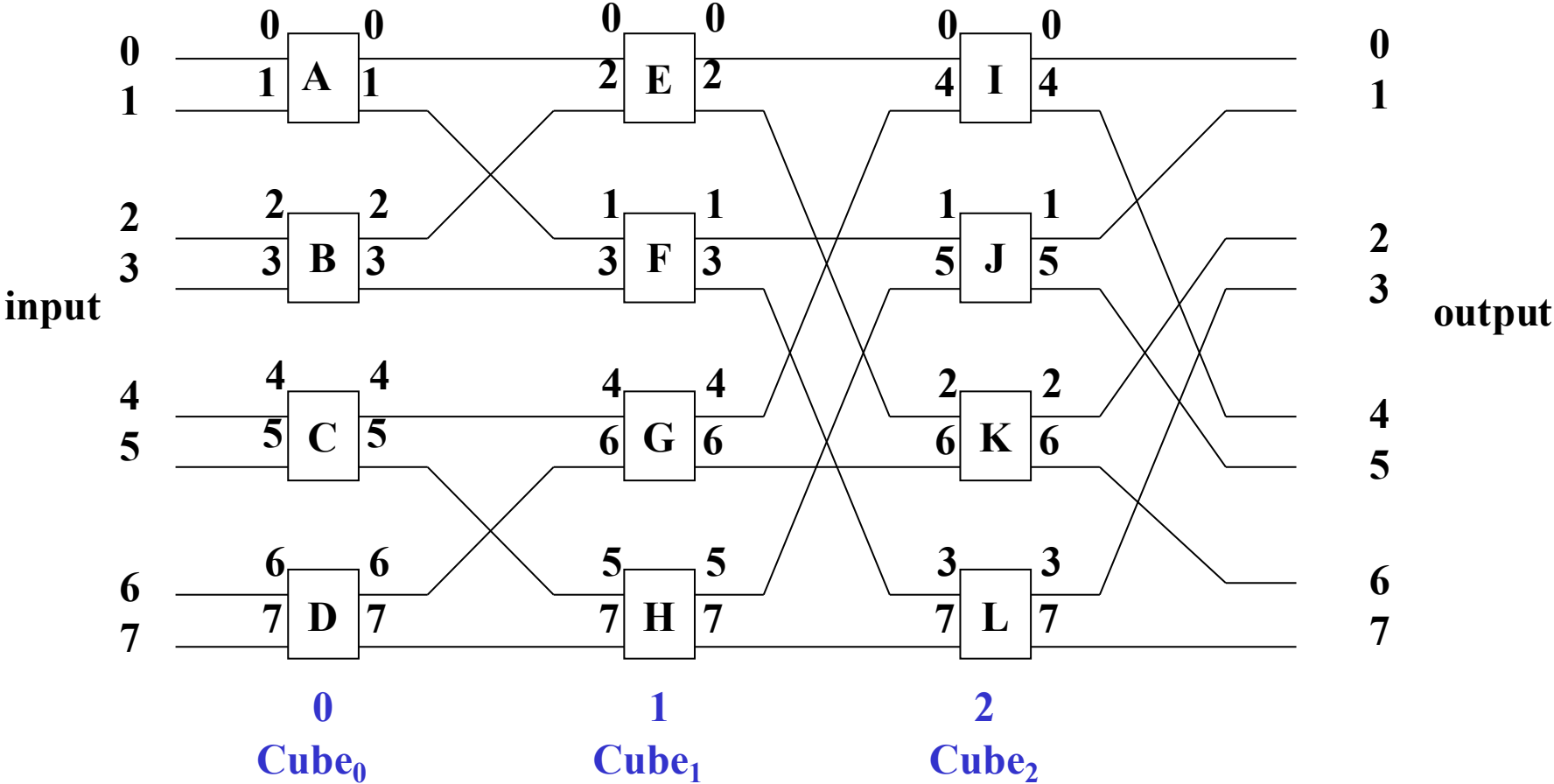


Multi-stage cube interconnection network

- N-unit multi-stage cube network topology diagram
 - From $n = \log_2 N$, find the number n of the multi-stage cube network
 - The stage number is set from input to output as 0, 1, ..., $n-1$
 - Each stage draws $N/2$ two-function switch units
 - Let the two input/output terminals of all the i^{th} stage switch units be numbered according to the Cube_i relationship
 - Connect the ends of the switches with the same number on each stage



Three-stage cube interconnection network



Multi-stage cube interconnection network

The multi-level cube network can be divided into:

- Switched network
- Mobile number network
- Indirect binary n-cube network

STARAN Network



Multi-stage cube interconnection network

Flip Network

- The multi-stage cube network that adopts the stage control mode is called the switching network, which can only realize the function of the switching function

Exchange function

- Exchange a group of elements symmetrically. If a group of elements contains 2^k elements, that is, all the k^{th} elements in the group are exchanged with the $[2^l - (k+1)]^{th}$ elements.



N = 8		Stage control signal ($K_2K_1K_0$), K_i : i^{th} stage (0=connect, 1=exchange)							
		000	001	010	011	100	101	110	111
No.	0	0	1	2	3	4	5	6	7
	1	1	0	3	2	5	4	7	6
	2	2	3	0	1	6	7	4	5
	3	3	2	1	0	7	6	5	4
	4	4	5	6	7	0	1	2	3
	5	5	4	7	6	1	0	3	2
	6	6	7	4	5	2	3	0	1
	7	7	6	5	4	3	2	1	0
Function		8 × 1	4×2	4×2 + 2×4	2×4	2×4 + 1×8	4×2 + 2×4 + 1×8	4×2 + 1×8	1×8
		i	Cube ₀	Cube ₁	Cube ₀ +Cube ₁ 1	Cube ₂	Cube ₀ + Cube ₂	Cube ₁ +Cube ₂	Cube ₀ + Cube ₁ + Cube ₂



Multi-stage cube interconnection network

Flip Network

- The multi-stage cube network that adopts the stage control mode is called the switching network, which can only realize the function of the switching function

Exchange function

- Exchange a group of elements symmetrically. If a group of elements contains 2^k elements, that is, all the k^{th} elements in the group are exchanged with the $[2^l-(k+1)]^{th}$ elements.



Multi-stage cube interconnection network

The parallel processor has 16 processors. To achieve the function equivalent of 4 groups of 4-element exchanges, 2 groups of 8-element exchanges, and a set of 16-element exchanges, the requirements are:

- Write the **general formula of the interconnection function** implemented between the processors
- Draw the corresponding multi-stage network topology diagram
- Mark the status of each stage of switch unit



Multi-stage cube interconnection network

- Input number sequence:

0 1 2 3 4 5 6 7 8 9 A B C D E F

Exchange ↓ 4 group * 4 elements

3 2 1 0 7 6 5 4 B A 9 8 F E D C

Exchange ↓ 2 group * 8 elements

4 5 6 7 0 1 2 3 C D E F 8 9 A B

Exchange ↓ 1 group * 16 elements

B A 9 8 F E D C 3 2 1 0 7 6 5 4



Multi-stage cube interconnection network

- Interconnection function:

(0 B) (1 A) (2 9) (3 8)

(4 F) (5 E) (6 D) (7 C)

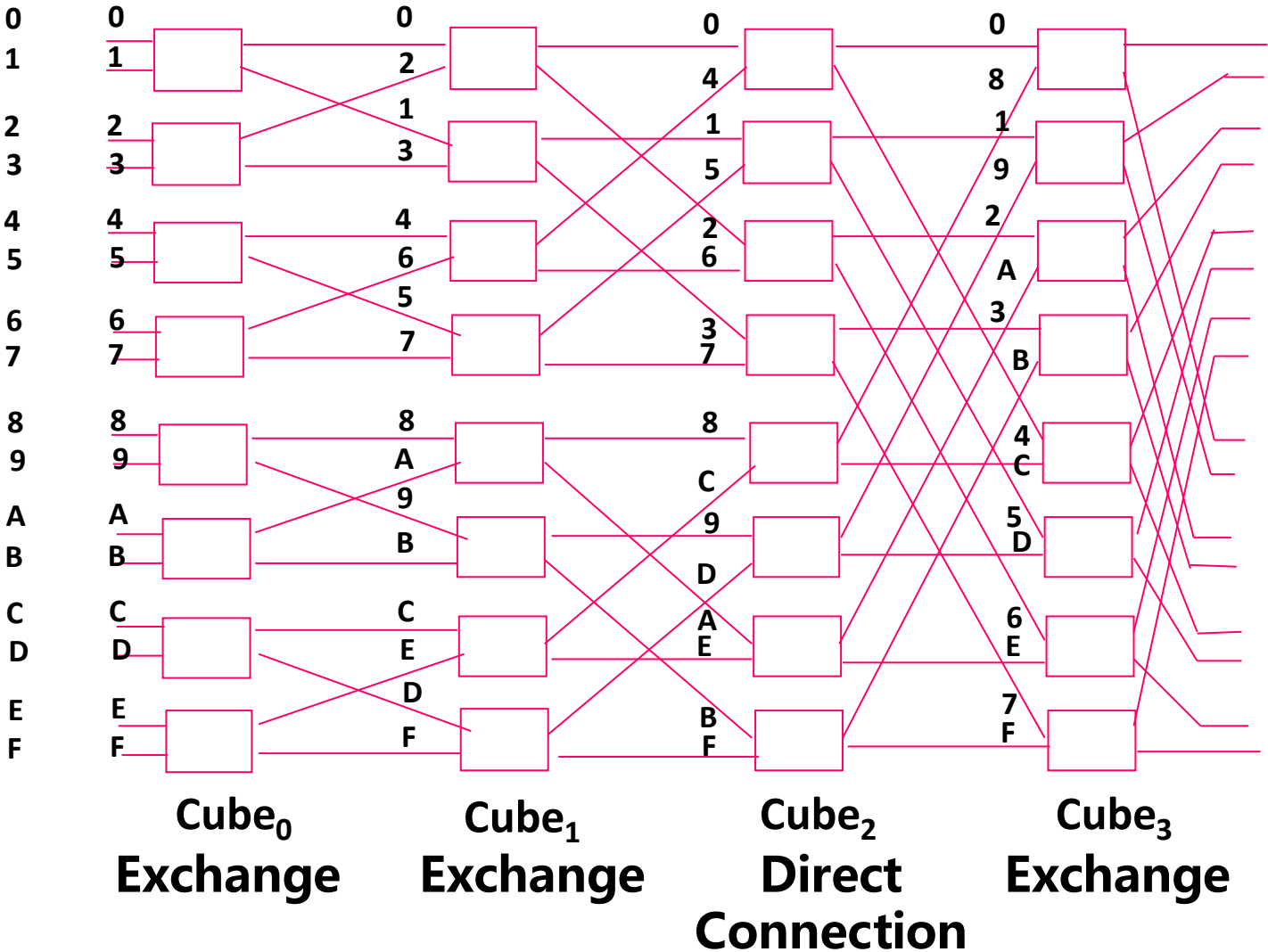
- General formula of the interconnection function :

$$f(P_3P_2P_1P_0) = \overline{P_3}P_2\overline{P_1}\overline{P_0}$$



$$f(P_3P_2P_1P_0) = \overline{P_3}P_2\overline{P_1}\overline{P_0}$$

Input



Output



Multi-stage shuffle exchange network

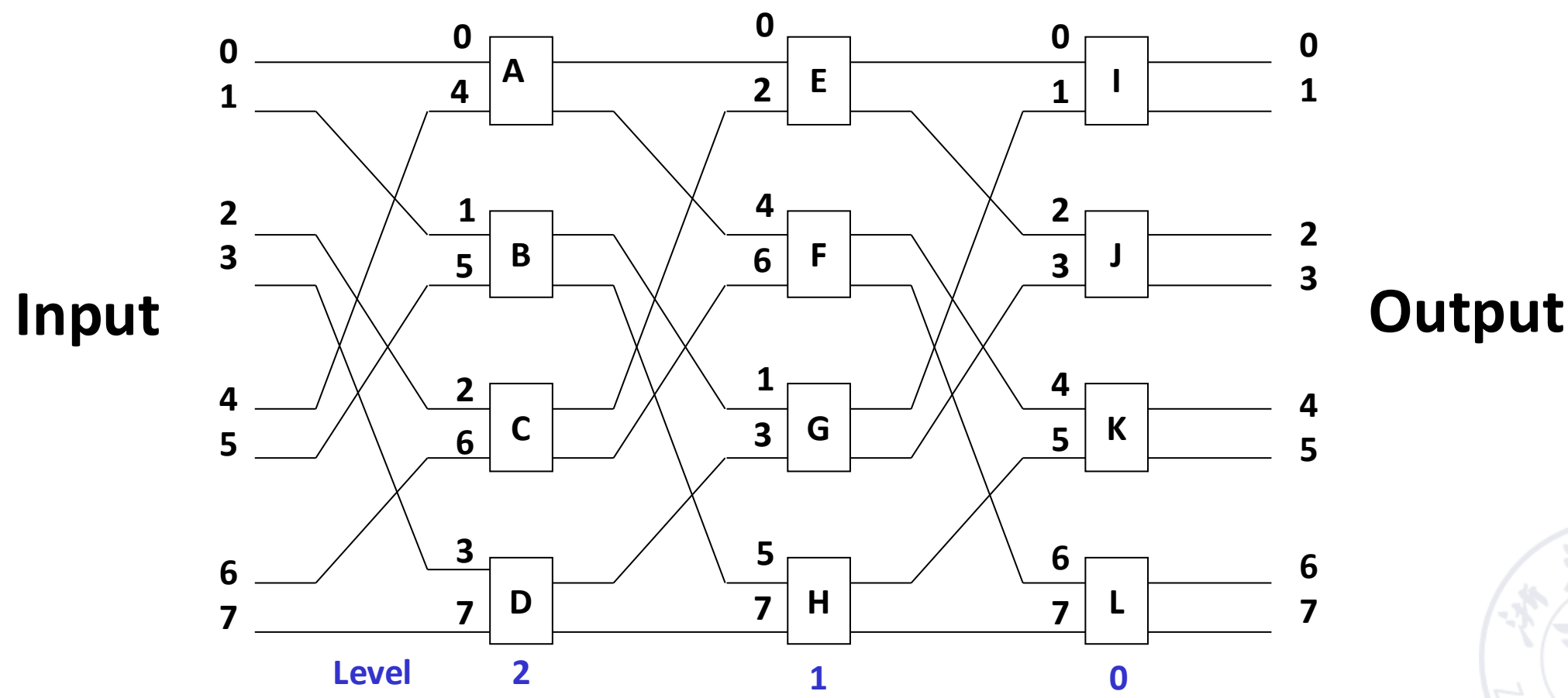
- Multi-level shuffle exchange network is also called **Omega network**.

Features:

- The switch function has four functions
- The topological structure is shuffled topology followed by a four-function switch
- Control mode is unit control

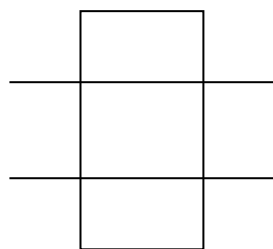


N = 8 Omega Network

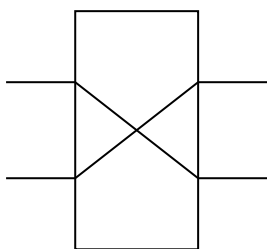


Multi-stage shuffle exchange network

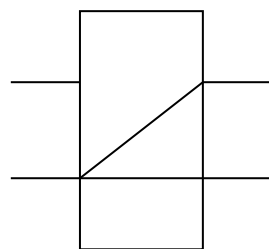
- 1. Number of stages $n = \log_2 N$
- 2. The stage number from input to output is $n-1, \dots, 1, 0$
- 3. Number of units $N/2$
- 4. Shuffle topology + four-function switch



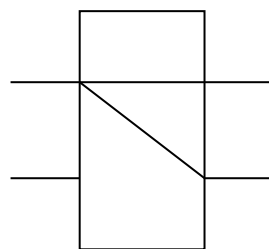
Straight



Exchange



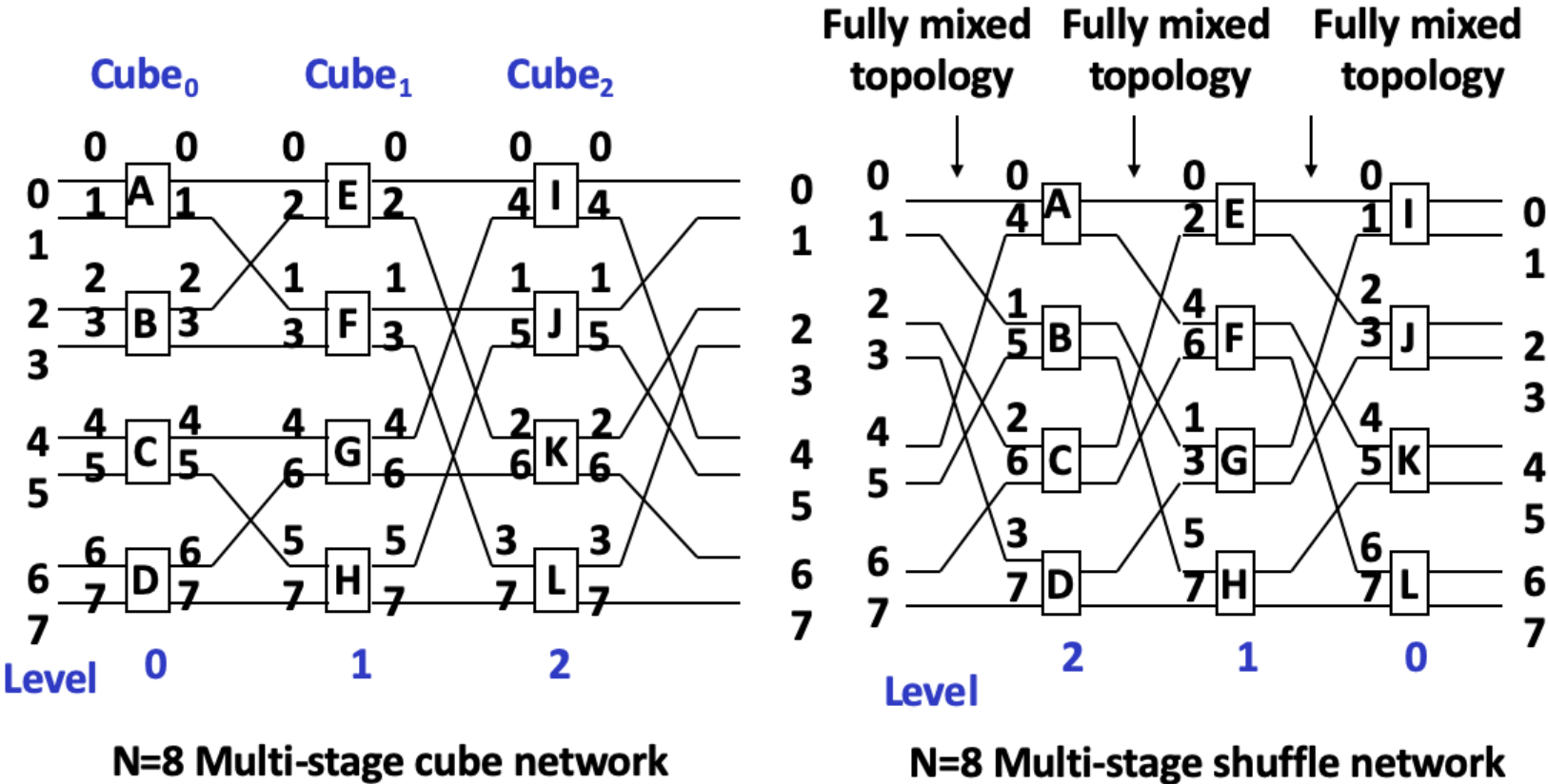
Upper
broadcast



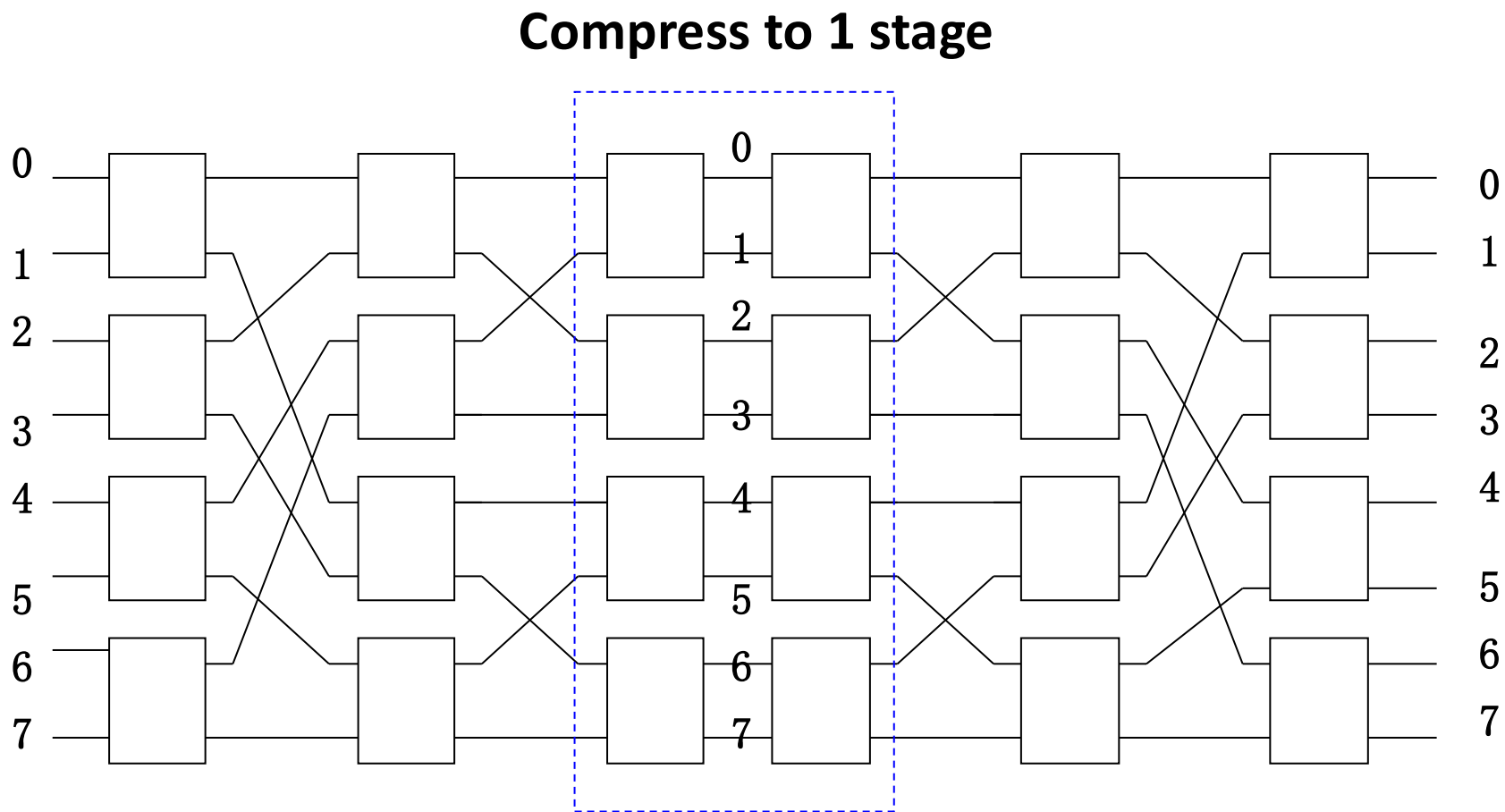
Lower
broadcast



If the switching unit of the Omega network is restricted to only use the "direct connect" and "exchange" functions, it becomes the inverse network of the n-cube network.



Example of multi-stage full array network (Benes network)



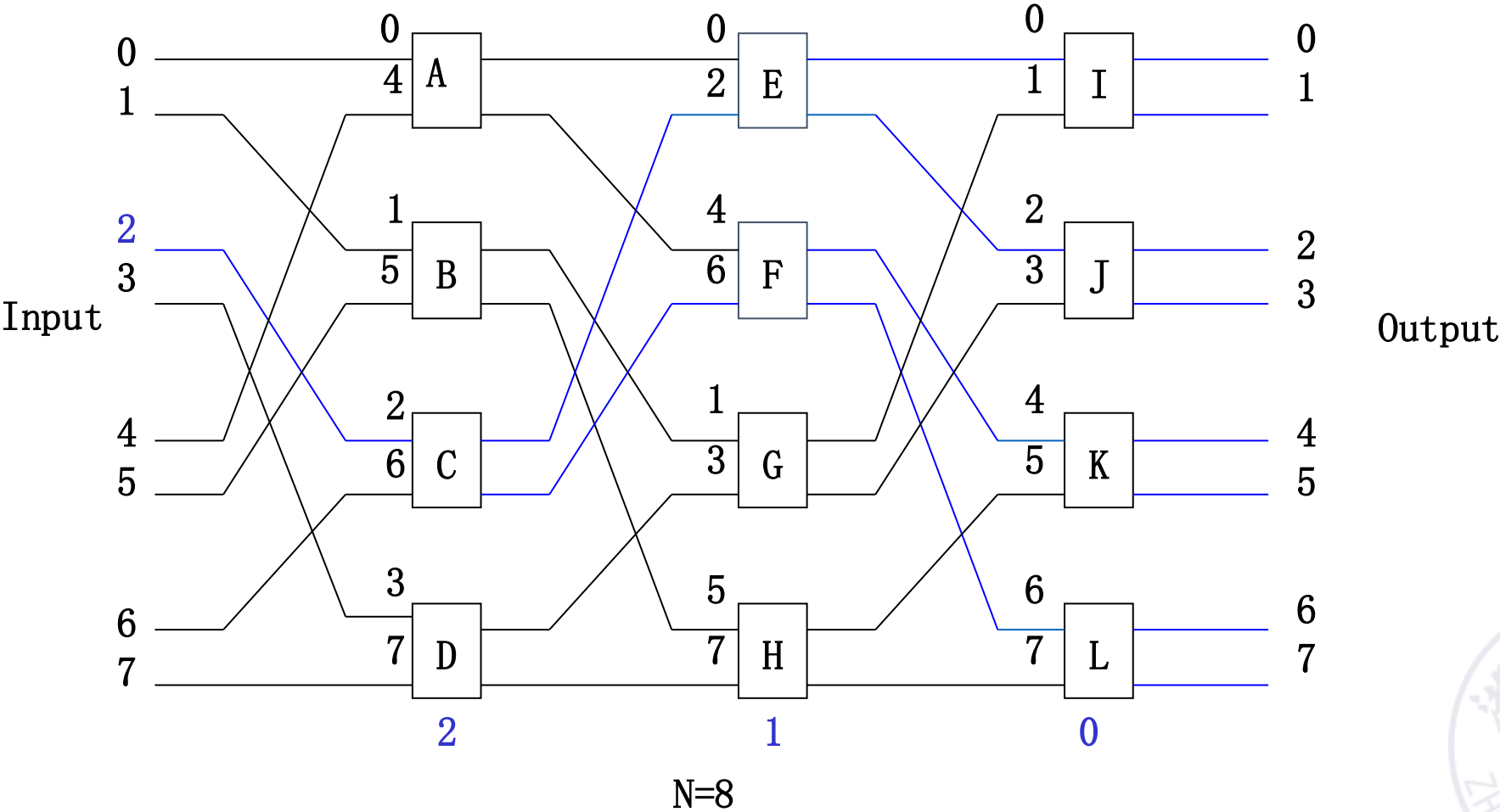
Omega network and n-cube network

The difference between Omega network and n-cube network:

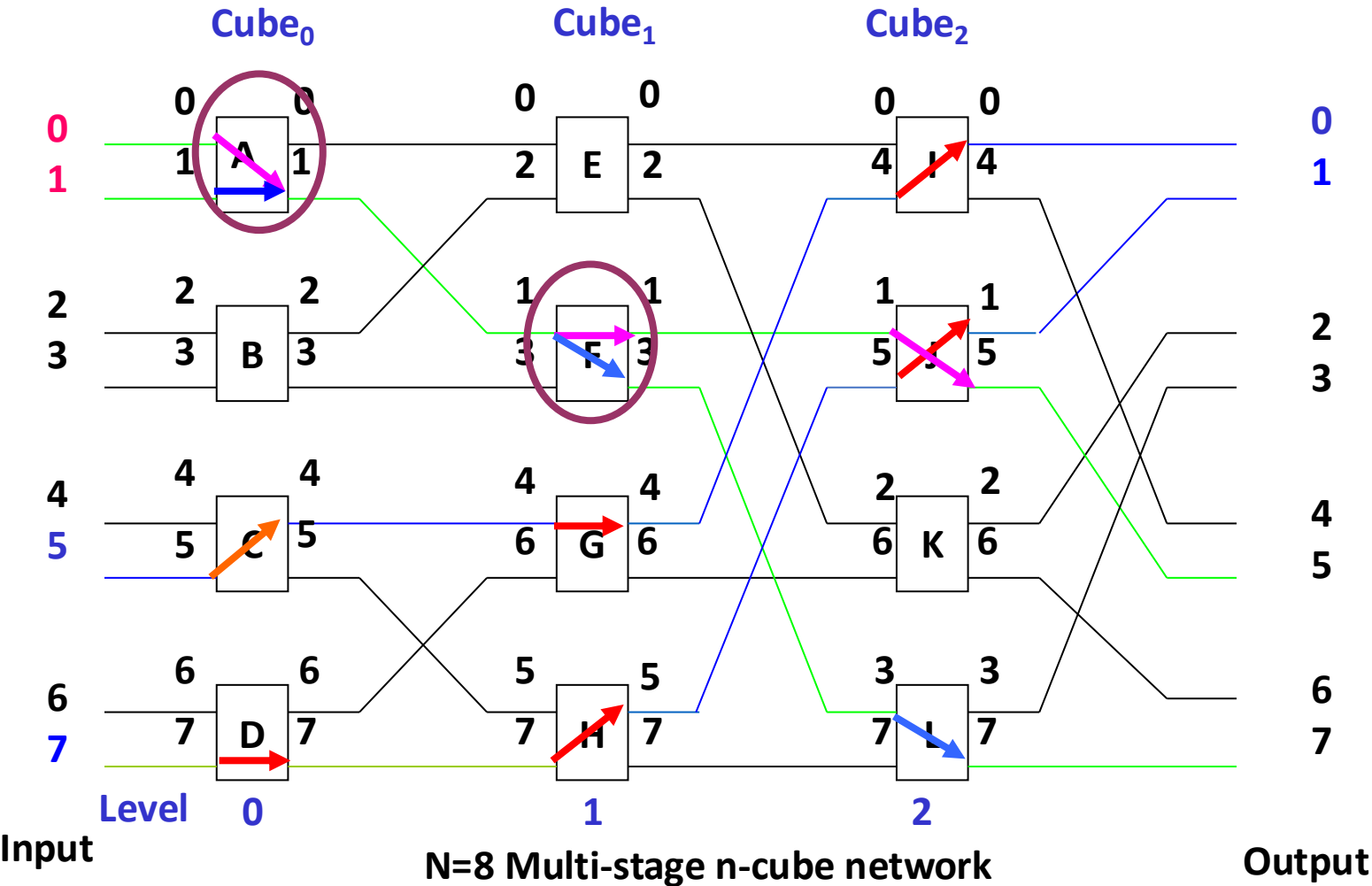
- 1. The level of Omega network data flow: $n-1, n-2, \dots, 1, 0$.
The level of n-cube network data flow: $0, 1, \dots, n-1$.
- 2. The Omega network uses a four-function exchange unit.
The n-cube network uses a two-function exchange unit.
- 3. Omega network can realize one-to-many broadcasting function.
N-cube network cannot achieve.



Omega network



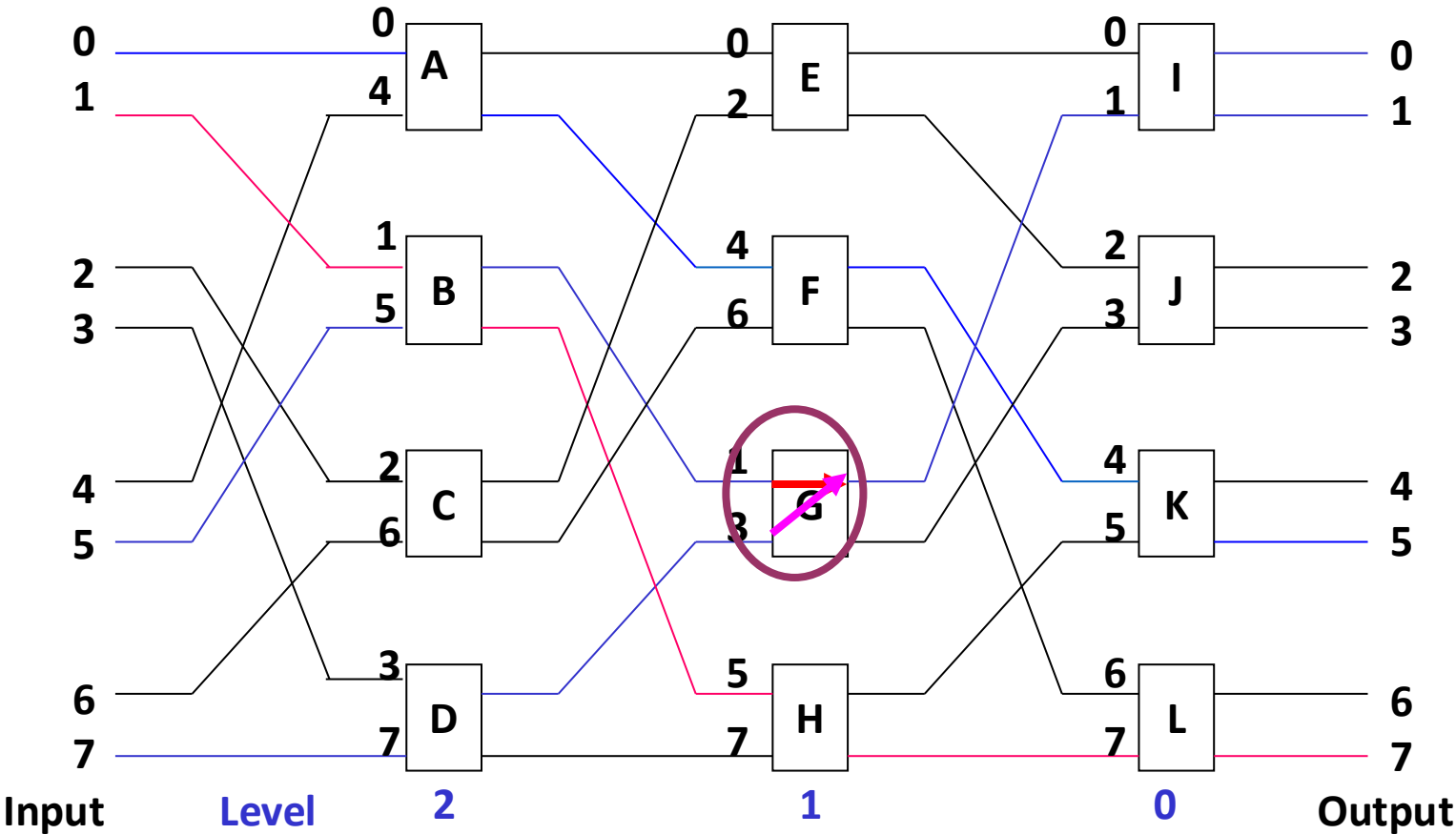
n-cube network



If the connection of **5 to 0** and **7 to 1** can be realized at the same time, can the connection of **0 to 5** and **1 to 7** be realized at the same time?



Omega network and n-cube network



N=8 Omega Network

The connection of **5 to 0** and **7 to 1** can be realized at the same time, but the connection of **0 to 5** and **1 to 7** cannot be realized at the same time!



Comparison of dynamic interconnection networks

	Bus System	Multi-stage Network	Crosspoint Switches
Bandwidth	$O(w/n)$ to $O(w)$	$O(w)$ to $O(nw)$	$O(w)$ to $O(nw)$
Complexity of Link	$O(w)$	$O(nw \log_k n)$	$O(n^2 w)$
Complexity of Switch	$O(n)$	$O(n \log_k n)$	$O(n^2)$
Connection and Path Finding	One to one once	Broadcast and exchange to some extents	Fully exchange, on
Explanation	n processors on bus, the bus bandwidth is w bits	$n * n$ MIN with $k * k$ switches, the bandwidth is w bits	$n * n$ crosspoint switches with w -bit bandwidth



Advantages of SIMD

- SIMD architectures can exploit significant data-level parallelism for:
 - Matrix-oriented scientific computing
 - Media-oriented image and sound processors
- SIMD is more energy efficient than MIMD
 - Only needs to fetch one instruction per data operation
 - Makes SIMD attractive for personal mobile devices
- SIMD allows programmer to continue to think sequentially

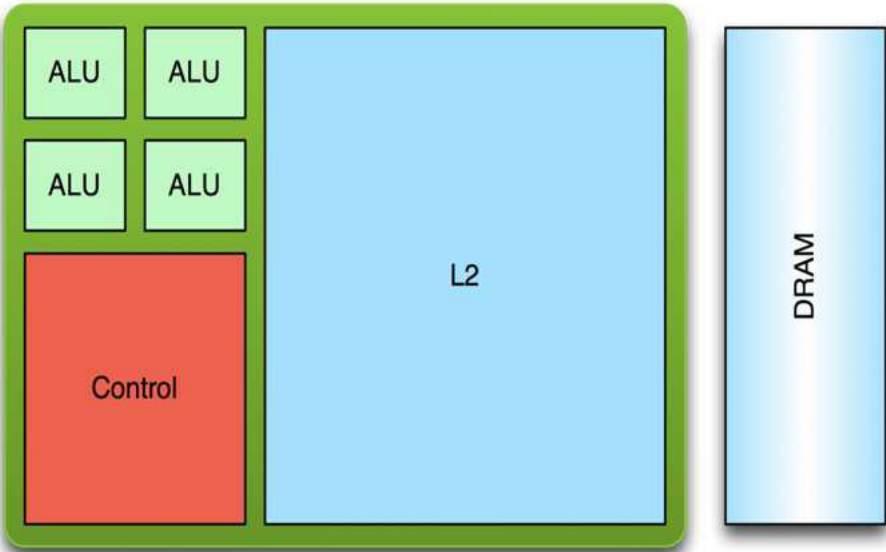


Graphical Processing Units

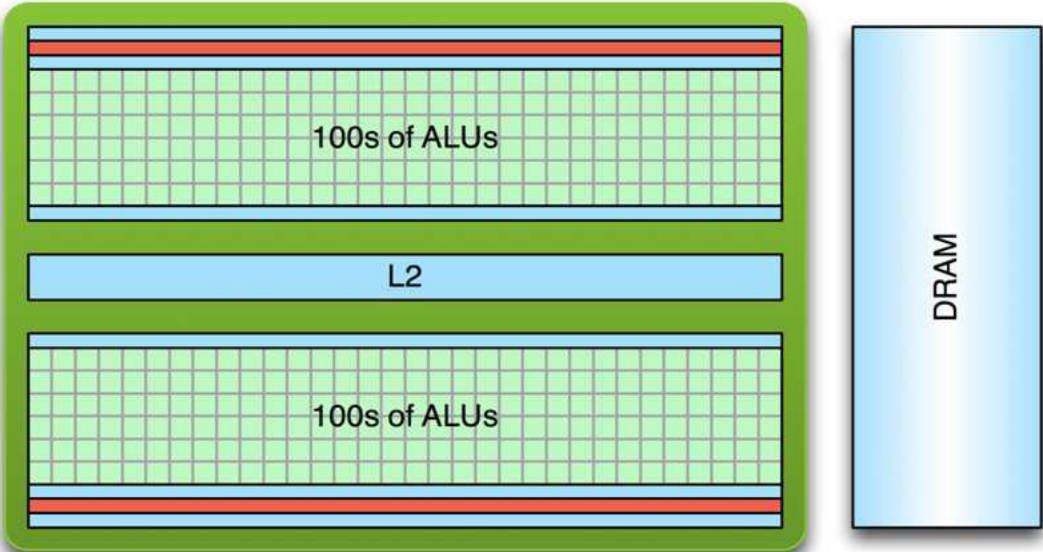
- Basic idea:
 - *Heterogeneous* execution model
 - CPU is the *host*, GPU is the *device*
 - Develop a C-like programming language for GPU
 - Unify all forms of GPU parallelism as *CUDA thread*
 - Programming model is “*Single Instruction Multiple Thread*”



Graphical Processing Units



CPU



GPU

- *Many but small cores*
- *Suitable for parallelism*
- *Application: Graphics and Deep Learning*



Programming the GPU: CUDA

- CUDA: Compute Unified Device Architecture
- NVIDIA decided that the unifying theme of all these forms of parallelism is the *CUDA Thread*. We call the hardware that executes a whole block of threads a *multithreaded SIMD Processor*.
- Actually, **GPUs are really just multithreaded SIMD Processors.**



Example: DAXPY

- C code:

```
// Invoke DAXPY
daxpy(n, 2.0, x, y);
// DAXPY in C
void daxpy(int n, double a, double *x, double *y)
{
    for (int i = 0; i < n; ++i)
        y[i] = a*x[i] + y[i];
}
```

- CUDA code:

```
// Invoke DAXPY with 256 threads per Thread Block
__host__
int nblocks = (n+ 255) / 256;
daxpy<<<nblocks, 256>>>(n, 2.0, x, y);
// DAXPY in CUDA
__global__
void daxpy(int n, double a, double *x, double *y)
{
    int i = blockIdx.x*blockDim.x + threadIdx.x;
    if (i < n) y[i] = a*x[i] + y[i];
}
```

CPU (Host)

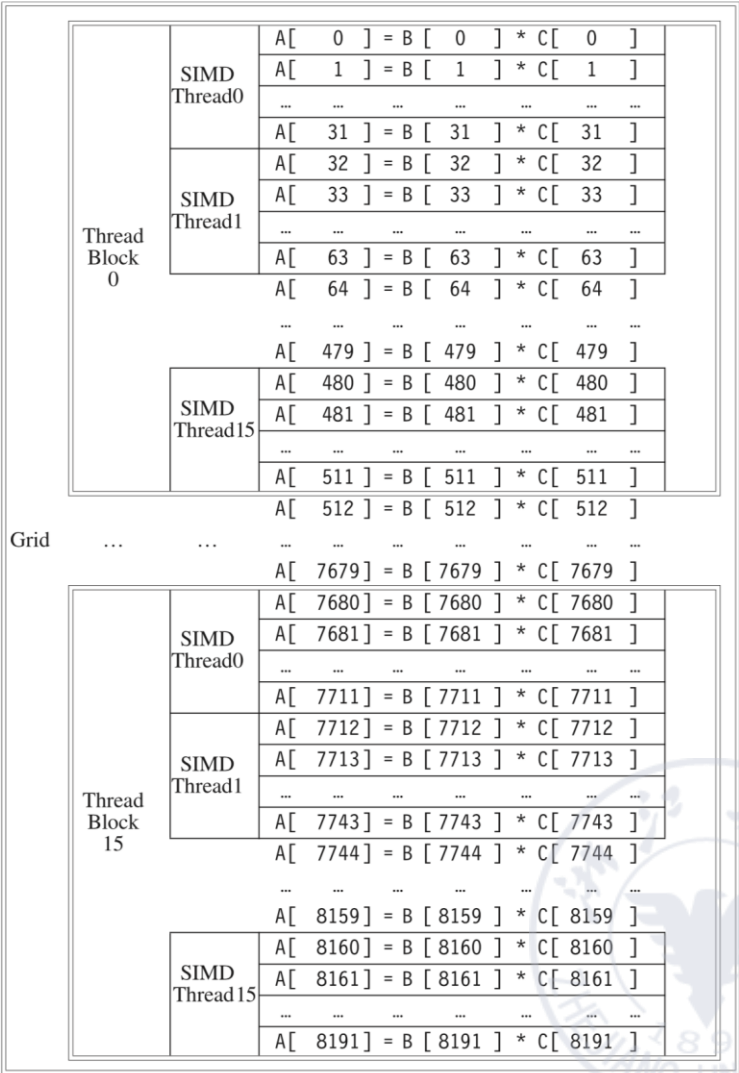
GPU (Device)



Grid, Thread Blocks and Threads

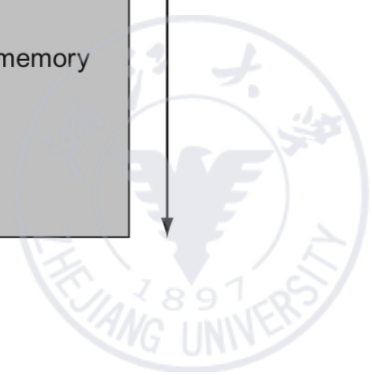
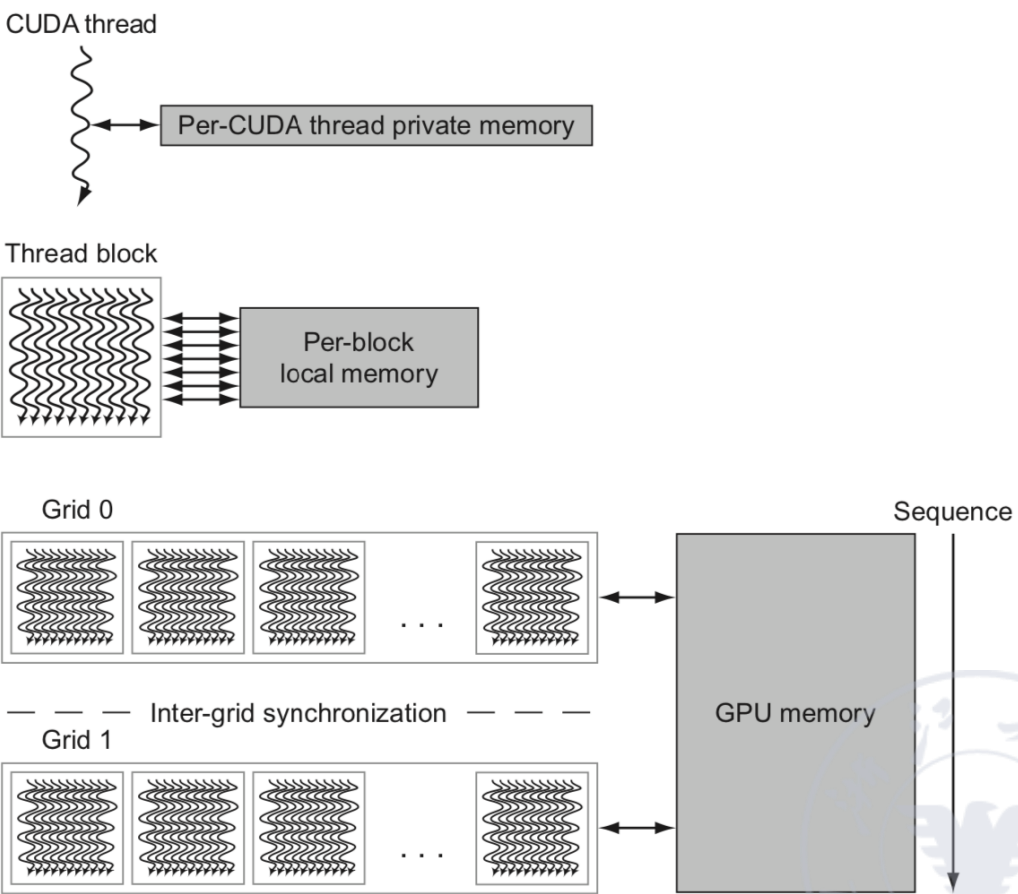
- A *thread* is associated with each data element
- Threads are organized into *blocks*
- Blocks are organized into a *grid*

- GPU hardware handles thread management, not applications or OS

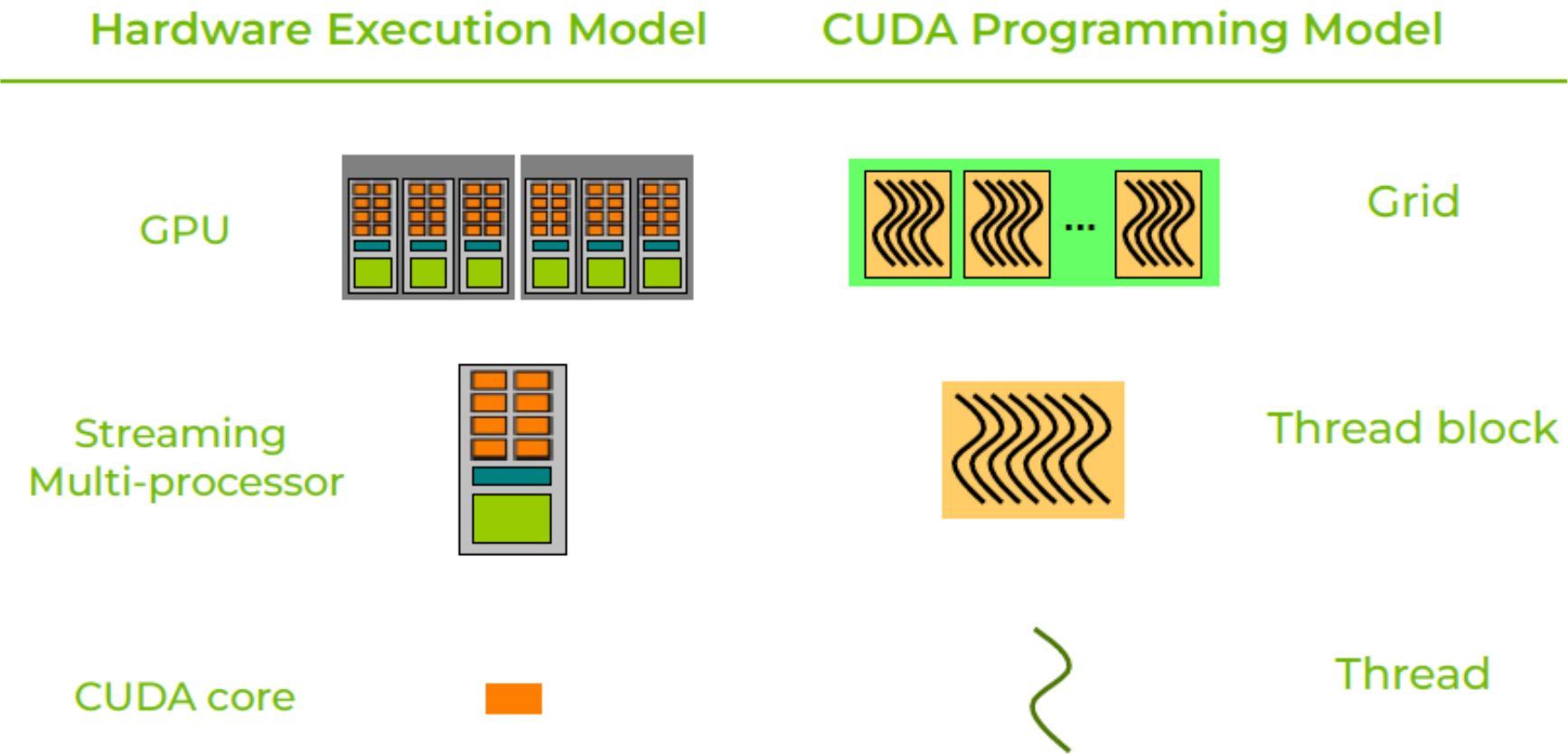


GPU memory structures

- **GPU memory** is shared by all *Grids* (vectorized loops).
- **Local memory** is shared by all threads of SIMD instructions within *a Thread Block* (body of a vectorized loop).
- **Private memory** is private to a single *CUDA Thread*.



GPU memory structures

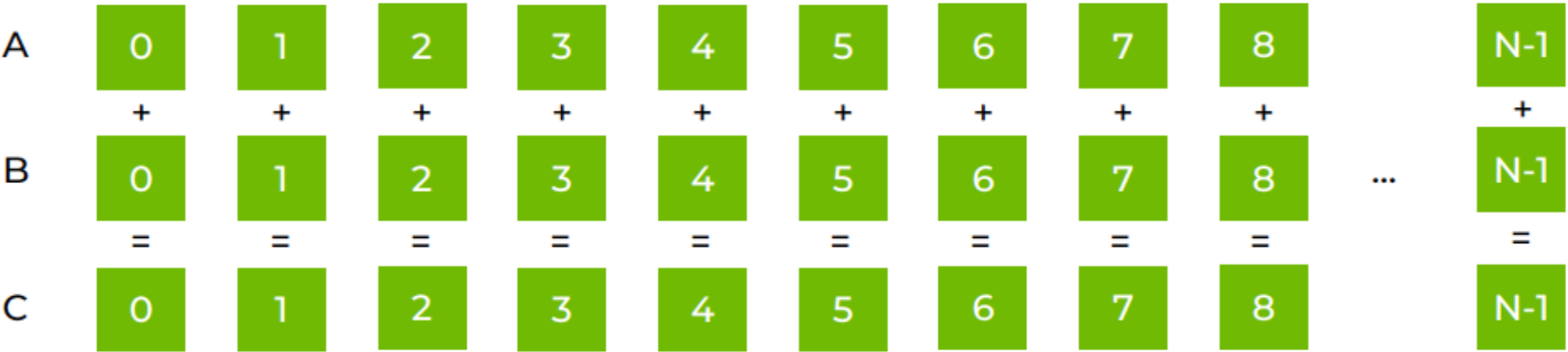


GPU memory structures

Example: Vector Addition

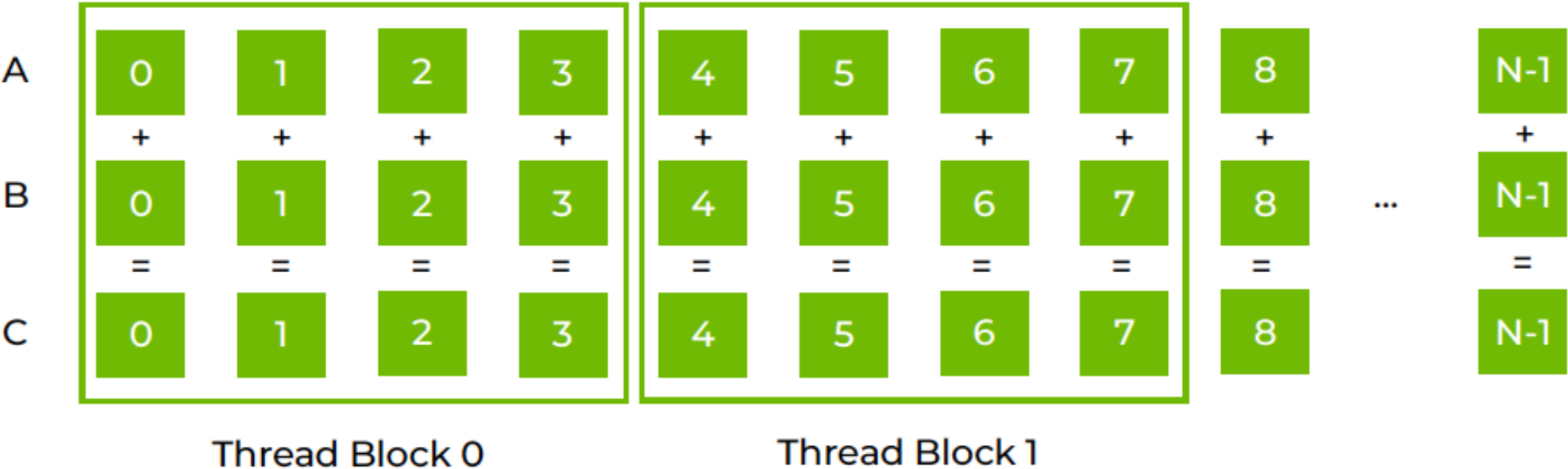
```
__global__ void vecadd_kernel(float* A, float* B, float* C, int N) {
    int i = blockDim.x * blockIdx.x + threadIdx.x;
    C[i] = A[i] + B[i];
}

vecadd_kernel<<<numBlocks, numThreadsPerBlock>>>(A_d, B_d, C_d, N);
```



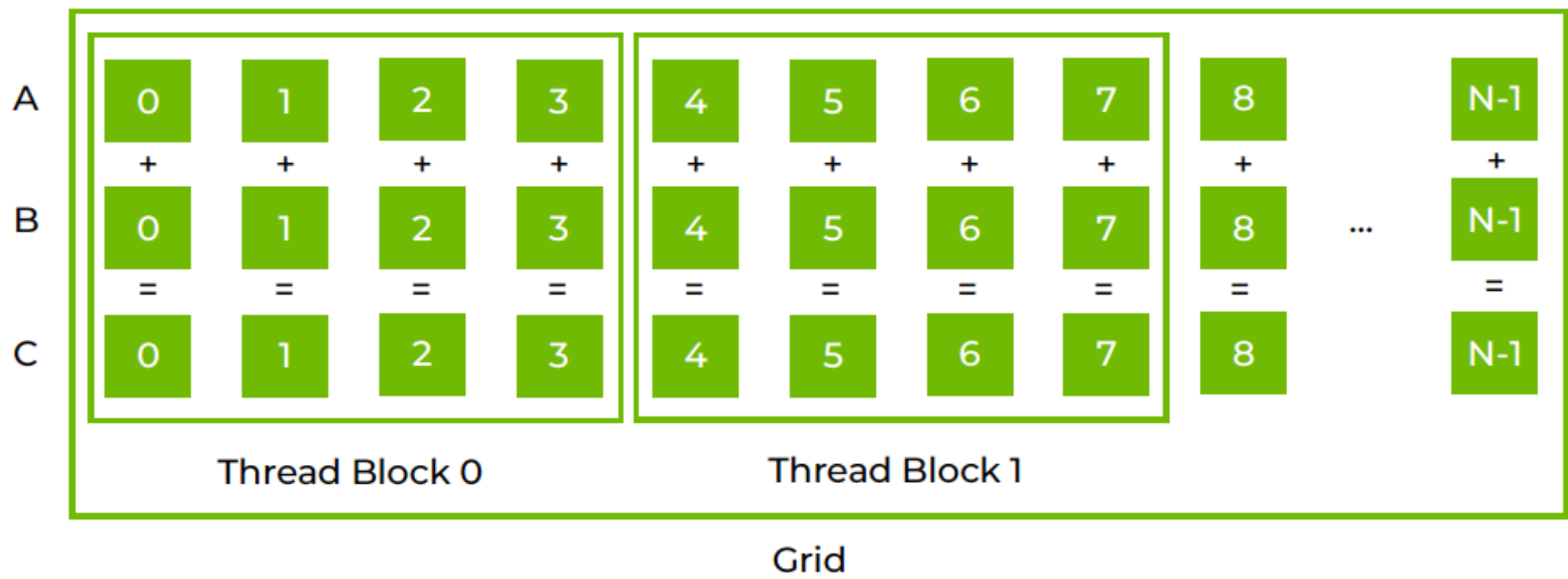
GPU memory structures

Example: Vector Addition

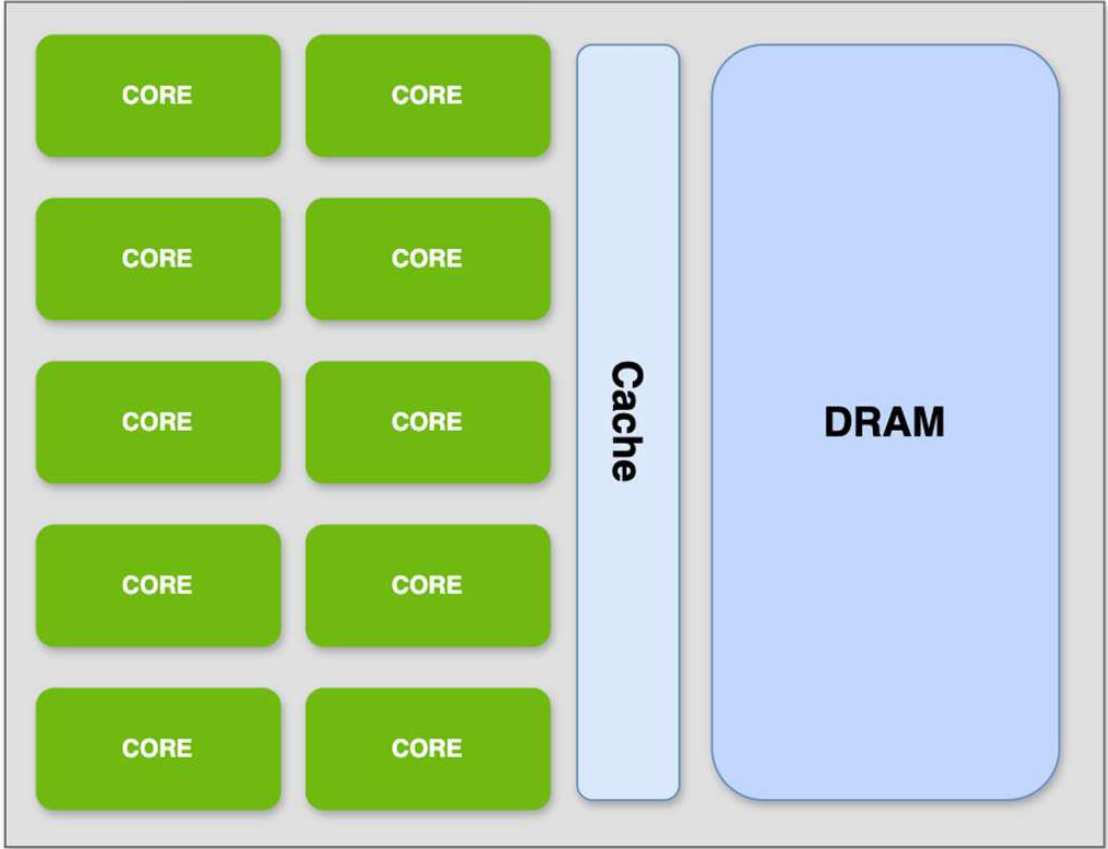


GPU memory structures

Example: Vector Addition



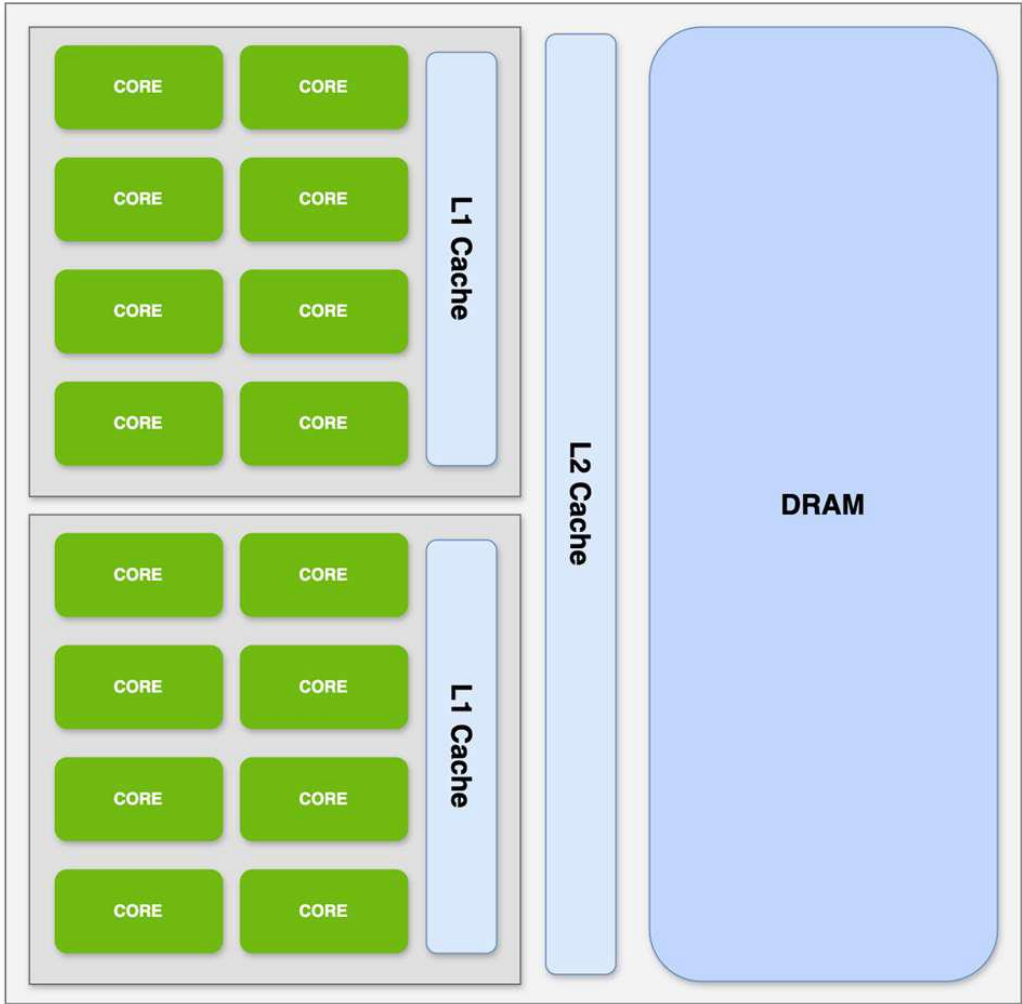
Memory Hierarchy in GPU



- *Add a cache between GPU and DRAM*
- *Spatial Locality & Temporal Locality*



Memory Hierarchy in GPU



Two-Level Cache:

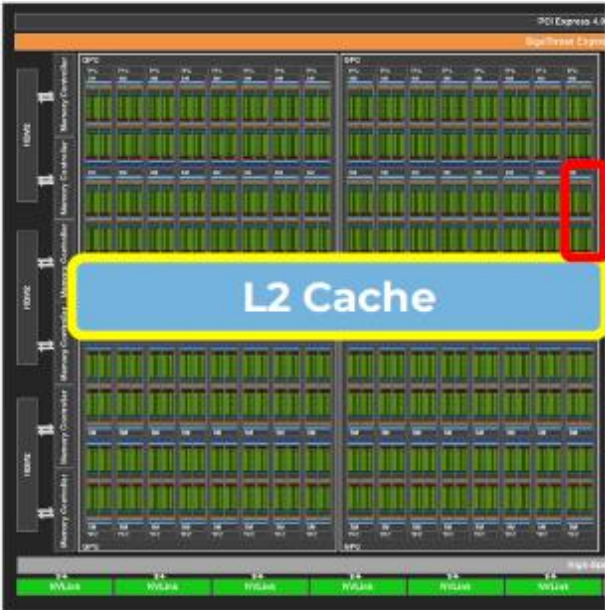
- *Utilize the spatial locality in SM*
- *Fit Core - SM - GPU architecture*
- *L1 Cache within SM, L2 Cache among SMs*



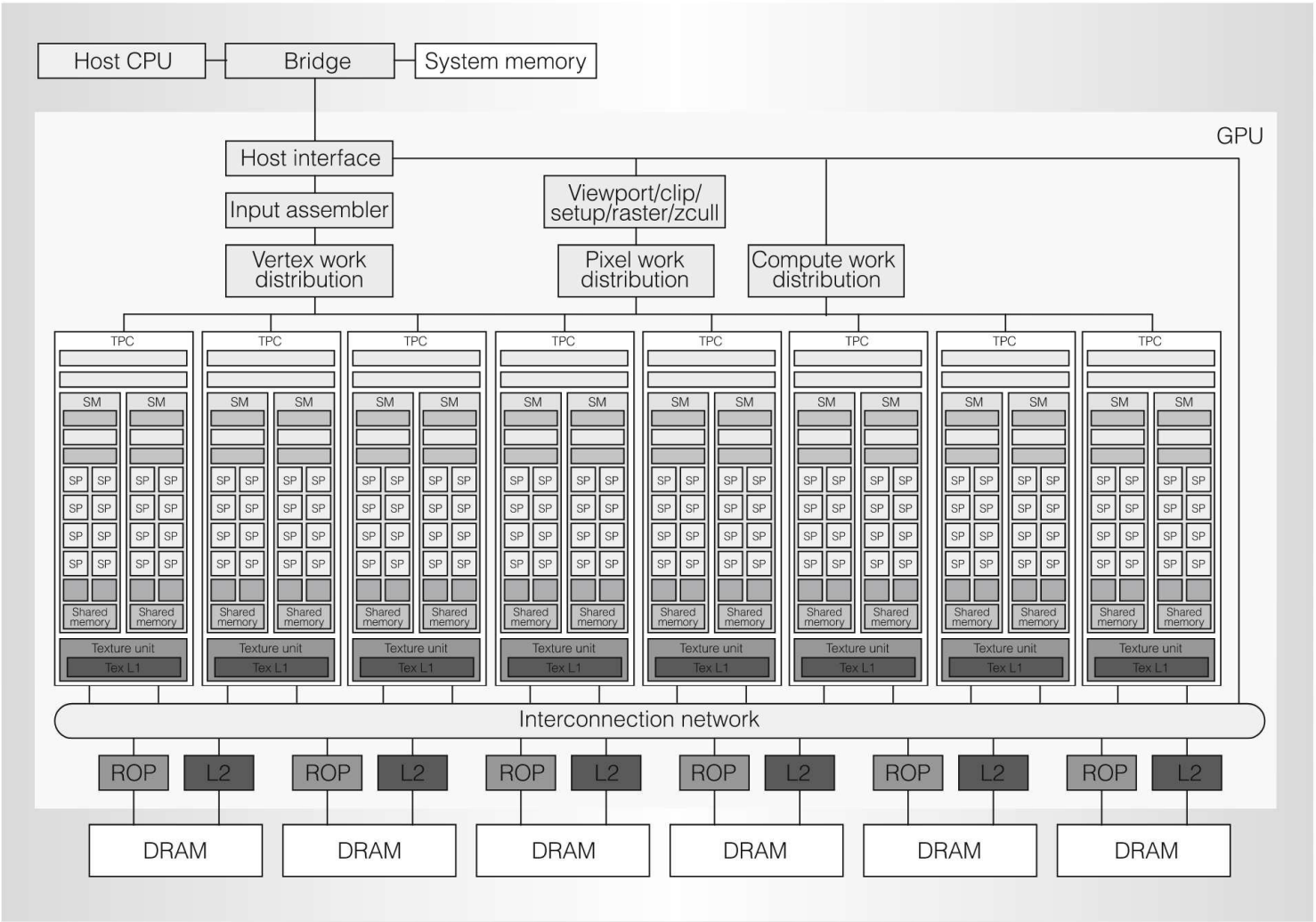
Memory Hierarchy in GPU

Memory Hierarchy in A100

- Additional Instruction Cache
 - Shared Memory
- fully controlled by programmer



GPU Organization and an example (NVIDIA Tesla)

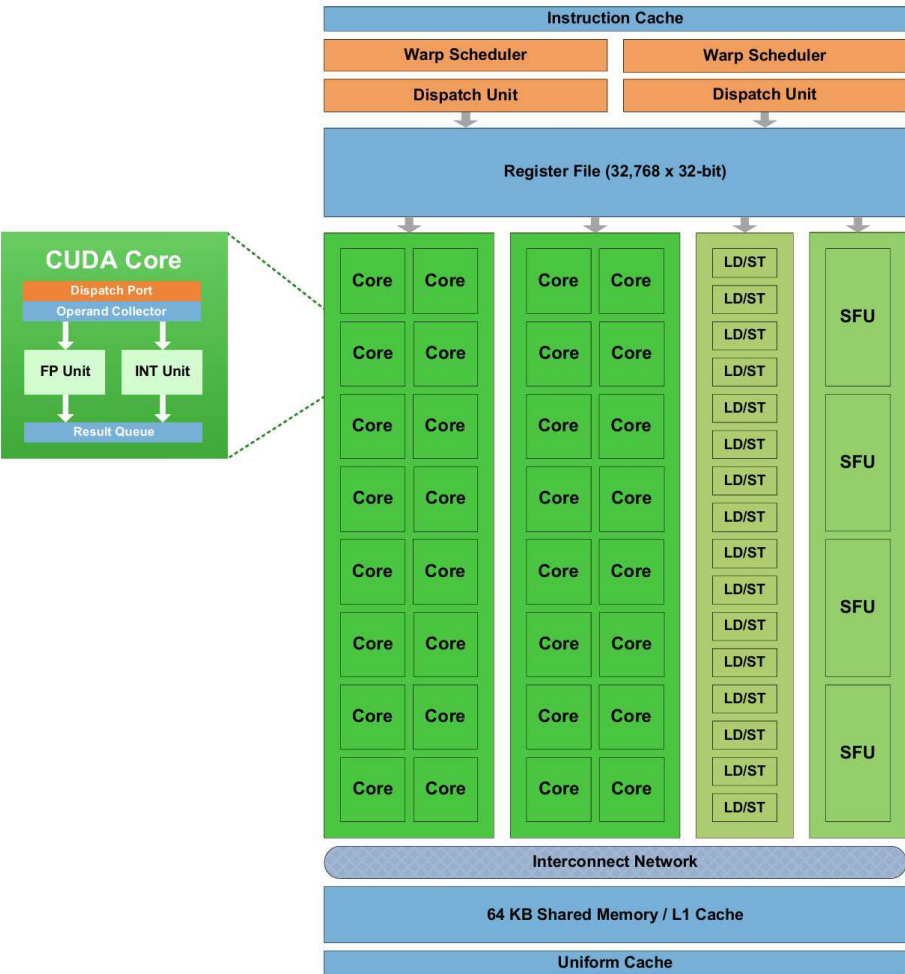


- Core
- SM(Streaming Multiprocessor)
- GPU

Figure 1. Tesla unified graphics and computing GPU architecture. TPC: texture/processor cluster; SM: streaming multiprocessor; SP: streaming processor; Tex: texture, ROP: raster operation processor.



GPU Organization and an example (Fermi)

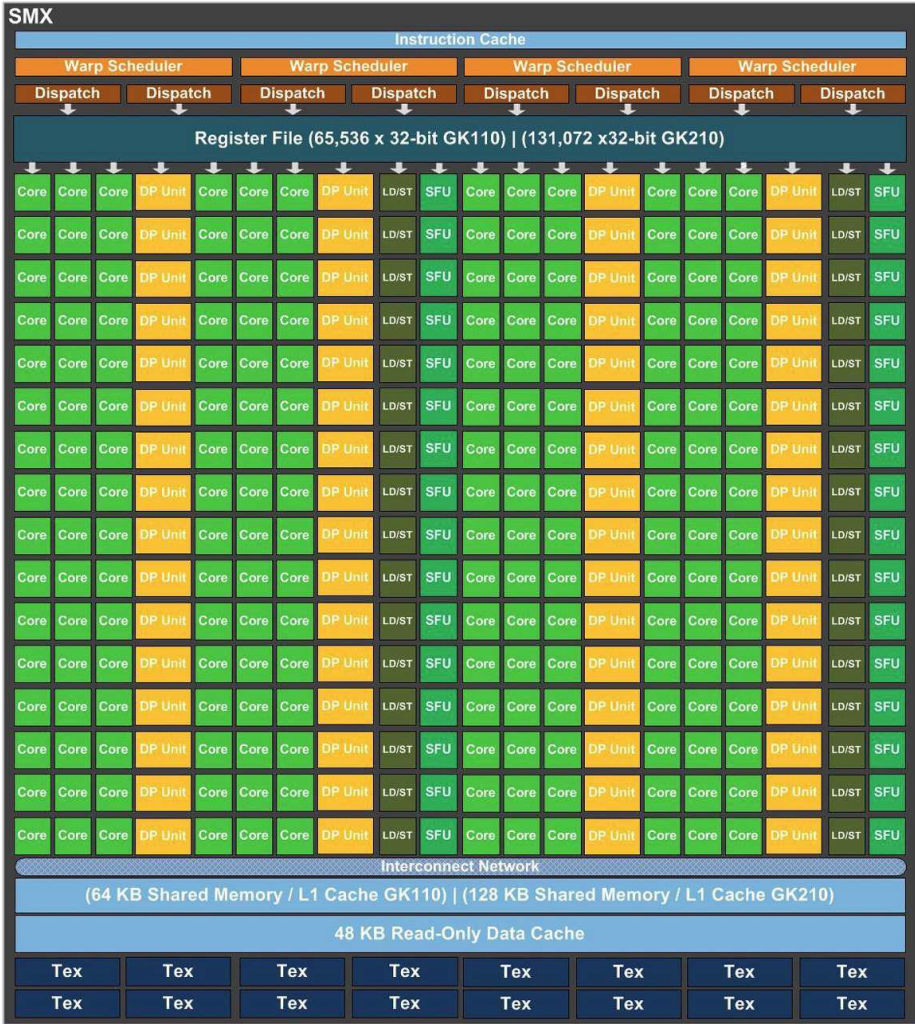


- *Integrated L1 and Shared Memory*
- *More cores in SM*

Figure 6. Each Fermi SM includes 32 cores, 16 load/store units, four special-function units, a 32K-word register file, 64K of configurable RAM, and thread control logic. Each core has both floating-point and integer execution units. (Source: NVIDIA)



GPU Organization and an example (Kepler)

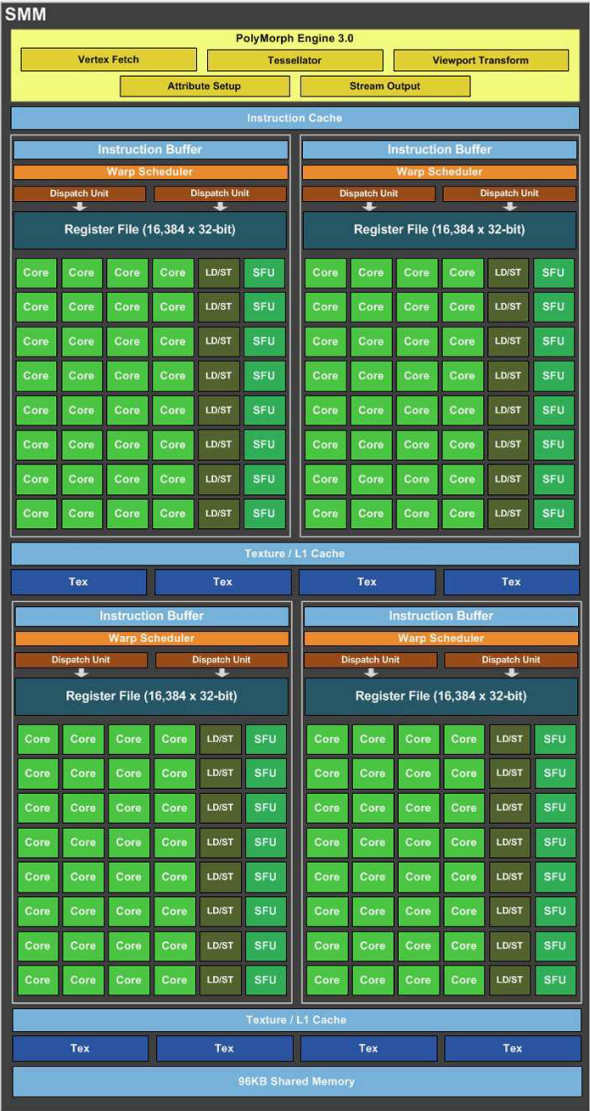


- *Giant SM: with 192 cores*
- *But larger SM means better?*

SMX: 192 single-precision CUDA cores, 64 double-precision units, 32 special function units (SFU), and 32 load/store units (LD/ST).



GPU Organization and an example (Maxwell)

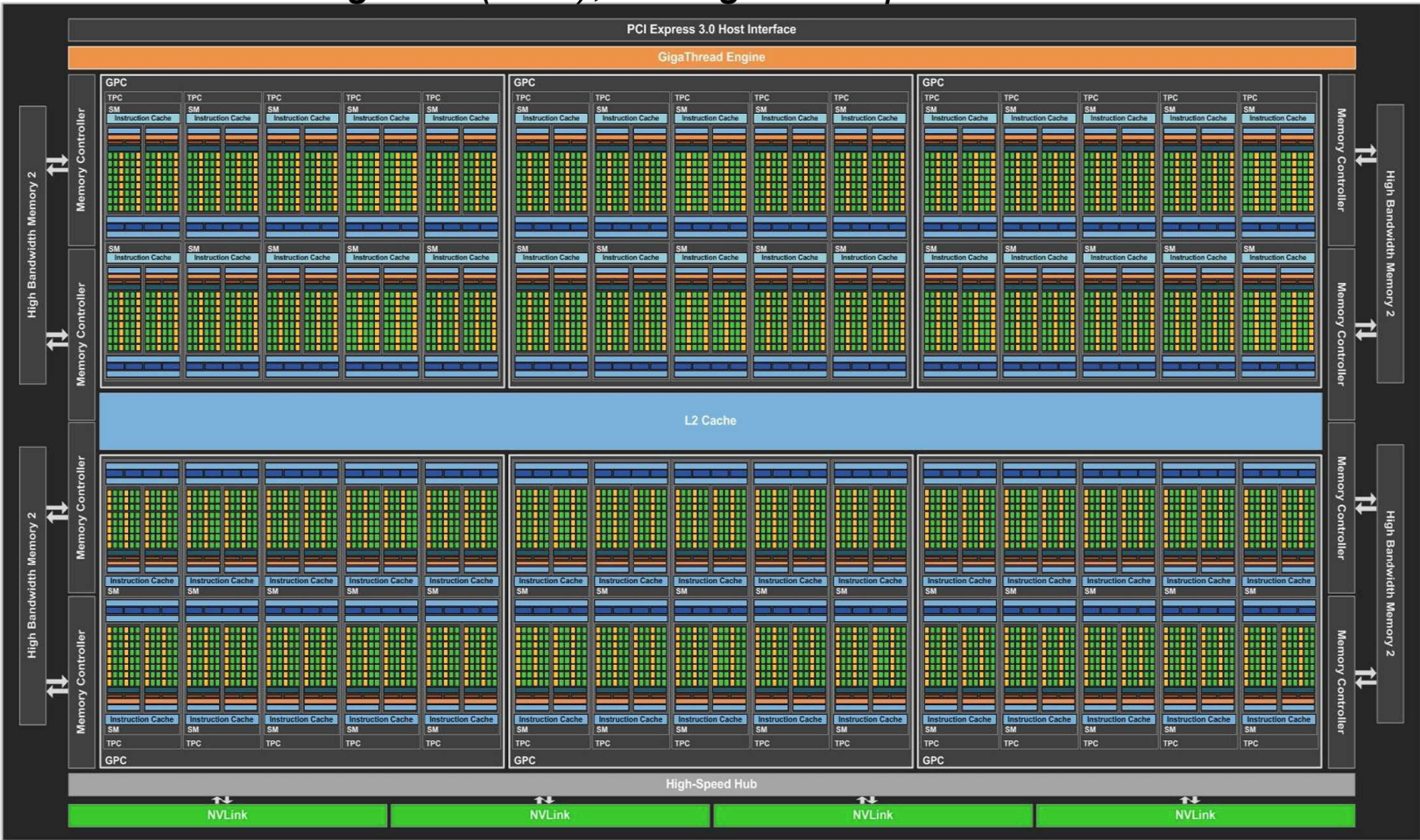


- *Split SM into 4 blocks*
- *Flexible scheduling and power control*
- *Split L1 and Shared Memory*

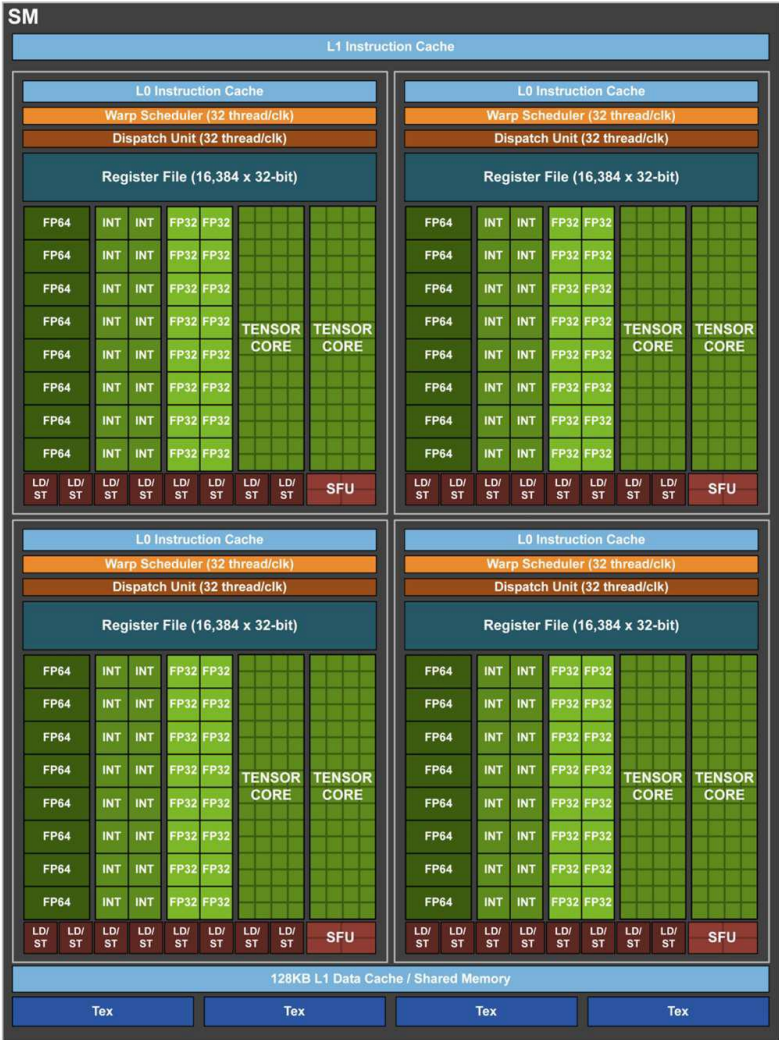


GPU Organization and an example (Pascal GPU)

Larger L2 (4MB), 7x larger than previous one



GPU Organization and an example (Volt)



- *Integrate L1 and Shared Memory again*
- *Instruction buffer -> L0 Instruction Cache*

Figure 5. Volta GV100 Streaming Multiprocessor (SM)



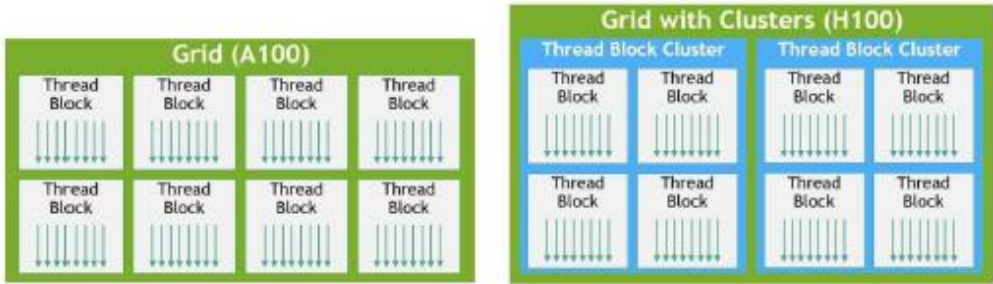
GPU Organization and an example (Ampere)

- *Larger L2 again: 40MB*
- *Additional data path between global and shared memory*



GPU Organization and an example (Hopper)

- Distributed Shared Memory
- Connection between Shared Memory in neighbor SMF



A Grid is composed of Thread Blocks in the legacy CUDA programming model as in A100, shown in the left half of the above diagram. The Hopper architecture adds an optional Cluster hierarchy, shown in the right half of the diagram.

Figure 14. Thread Block Clusters and Grids with Clusters



Figure 15. Thread Block to Thread Block data exchange (A100 vs H100 with Clusters)



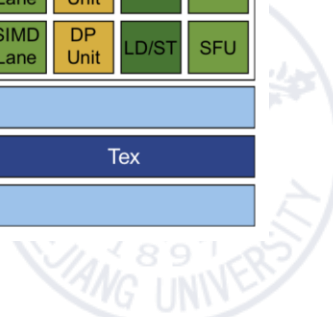
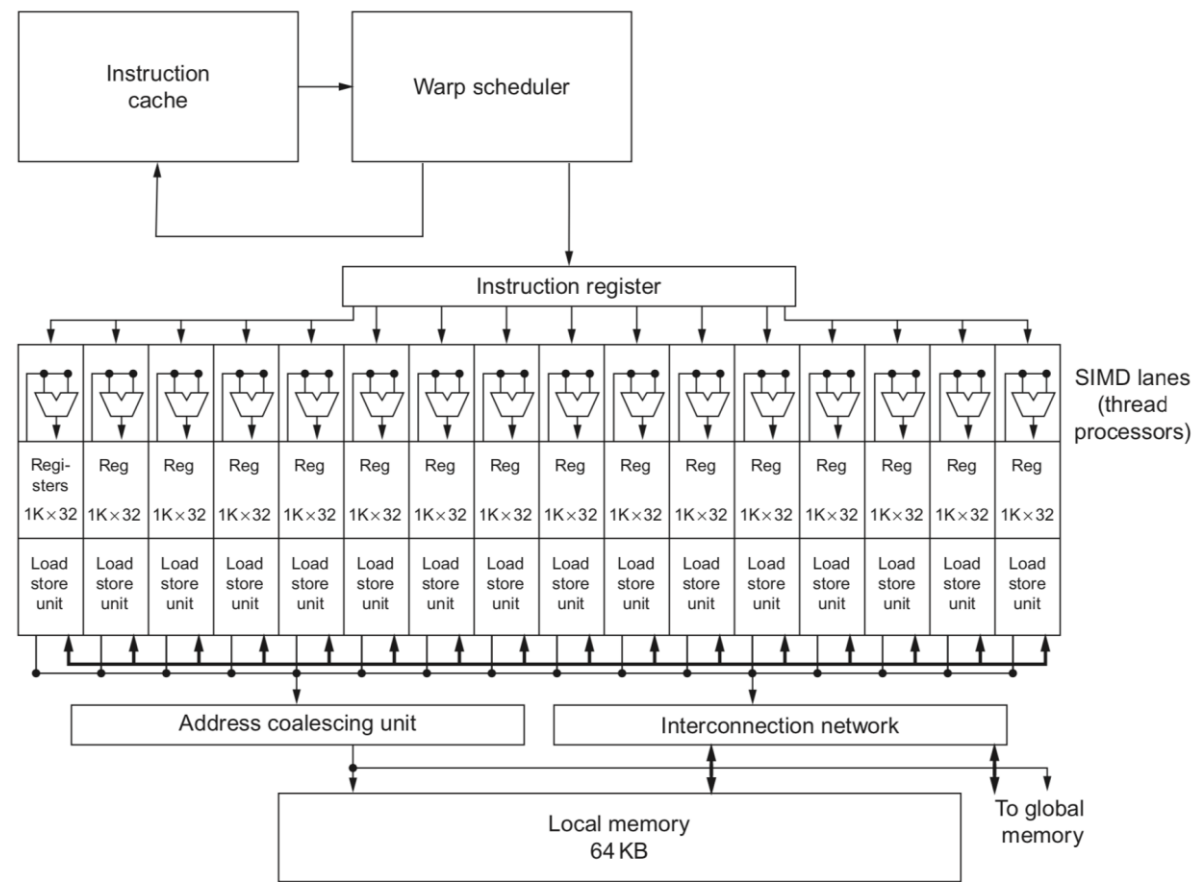
GPU Organization and summary

- Stable framework, evolving details.
- Expanding units, growing cache.
- Computation – Memory – Programming co-design.

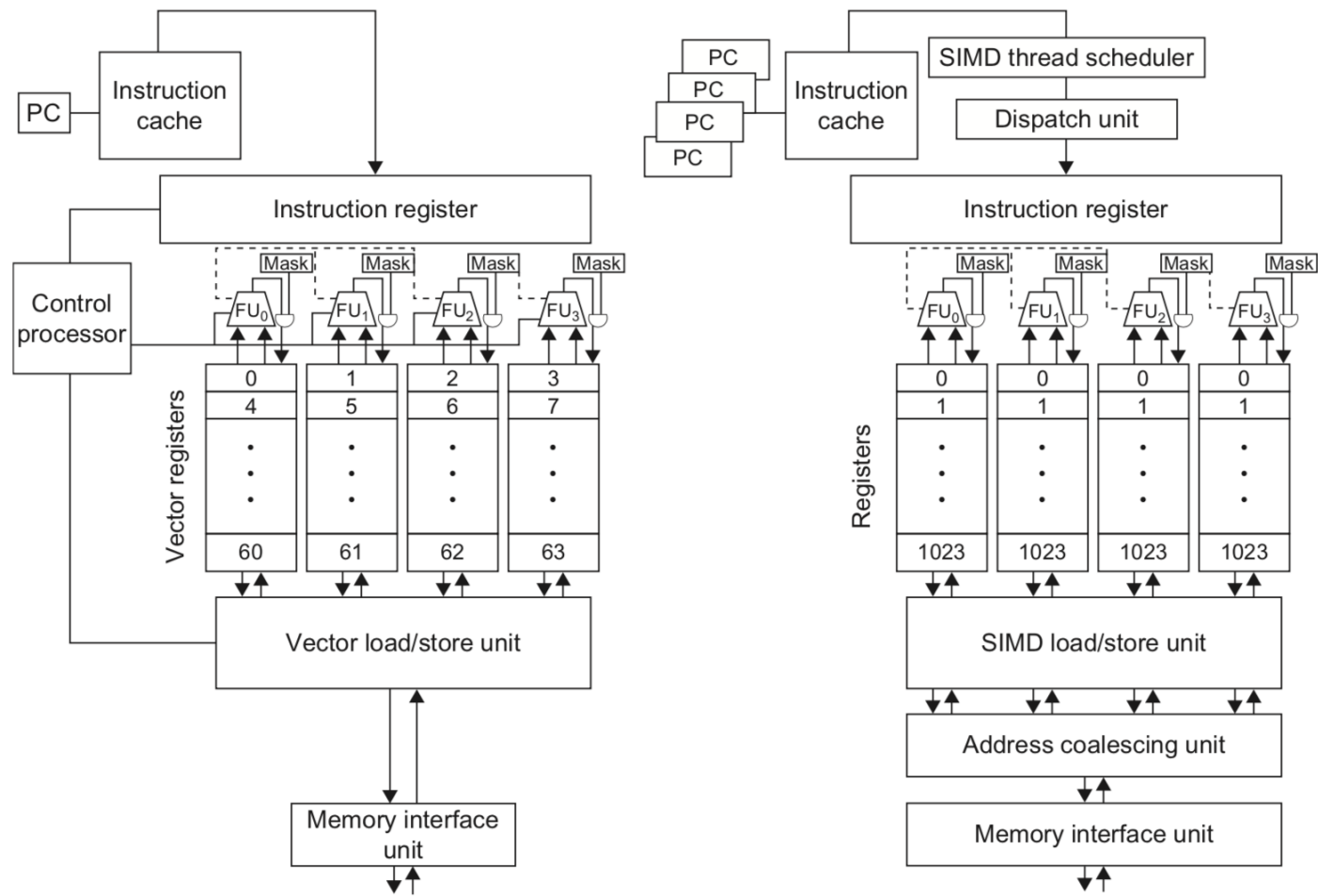
Architecture	Product	CUDA cores	L1 Cache	L2 Cache	Shared Memory
Maxwell	GTX TITAN X	3072	48KB	3MB	96KB
Volt	V100	5120	128KB	6MB	up to 96KB
Ampere	A100	6192	192KB	40MB	up to 164KB
Hopper	H100	16896	192KB	50MB	up to 164KB



GPU Organization and an example (Pascal GPU)



NVIDIA GPU (right) and vector machine (left)



NVIDIA GPU and vector machines

- Similarities to vector machines:
 - Works well with data-level parallel problems
 - Scatter-gather transfers
 - Mask registers
 - Large register files
- Differences:
 - No scalar processor
 - Uses multithreading to hide memory latency
 - Has many functional units, as opposed to a few deeply pipelined units like a vector processor



Loop-Level Parallelism

- Loops in programs are the fountainhead of many of the types of parallelism
- Finding and manipulating loop-level parallelism is critical to exploiting both DLP and TLP, as well as the more aggressive static ILP approaches (e.g., VLIW) .



Loop-Level Parallelism

- Focuses on determining whether data accesses in later iterations are dependent on data values produced in earlier iterations
 - Loop-carried dependence

- Example 1:

```
for (i=999; i>=0; i=i-1)
```

```
    x[i] = x[i] + s;
```

- No loop-carried dependence



Loop-Level Parallelism

- Example 2:

```
for (i=0; i<100; i=i+1) {  
    A[i+1] = A[i] + C[i];      /* S1 */  
    B[i+1] = B[i] + A[i+1];    /* S2 */  
}
```

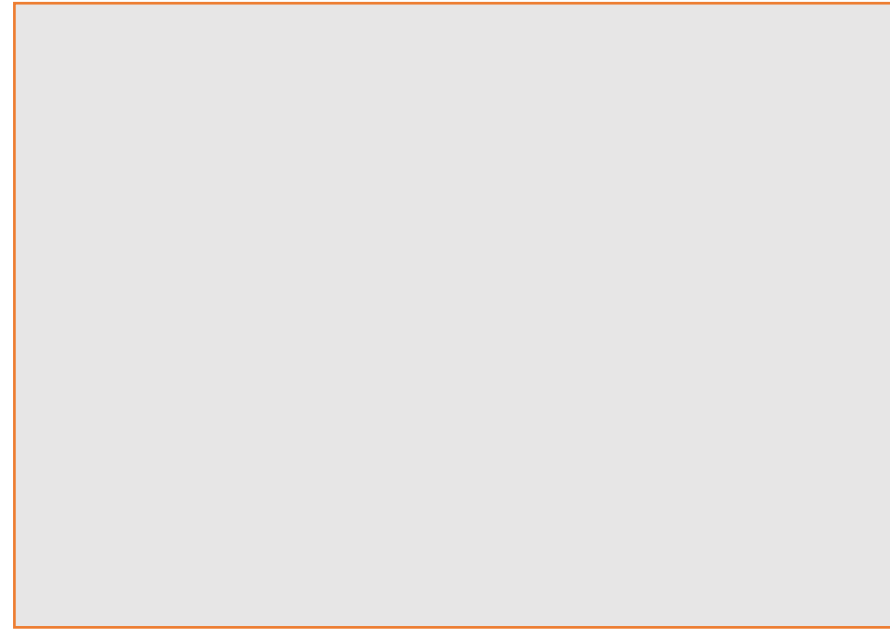
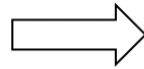
- S1 and S2 use values computed by S1 in previous iteration
- S2 uses value computed by S1 in same iteration



Loop-Level Parallelism

- Example 3:

```
for (i=0; i<100; i=i+1) {  
    A[i] = A[i] + B[i];    /* S1 */  
    B[i+1] = C[i] + D[i]; /* S2 */  
}
```



- S1 uses value computed by S2 in previous iteration but dependence is not circular so loop is parallel.



Loop-Level Parallelism

- Practice:

```

for (i=0; i<100; i=i+1) {
    Y[i] = X[i]/c;          /* S1 */
    X[i] = X[i] + c;        /* S2 */
    Z[i] = Y[i] + c;        /* S3 */
    Y[i] = c - Y[i];        /* S4 */
}

```

```

for (i=0; i<100; i=i+1) {
    T[i] = X[i]/c;          /* S1 */
    P[i] = X[i] + c;        /* S2 */
    Z[i] = T[i] + c;        /* S3 */
    Y[i] = c - T[i];        /* S4 */
}

```


Vector Chaining Technology

- Practice:

Assuming there is a Vector machine, having 2 load/store units with 10 clock cycles function unit time, 1 multiplier with 7 clock cycles function unit time, and 1 adder with 4 clock cycles function unit time. All the vector function units are fully pipelined that can start a new operation on every clock cycle. There is a code sequence as following, the vector length is 64.

- Vld v0, x5 ; load vector X
 - Vmul v1, v0, f0 ; vector –scalar multiply
 - Vld v2, x6 ; Load vector Y
 - Vadd v3, v1, v2 ; vector –vector add
 - Vst v3, x6 ; store the sum
- How many clock cycles to execute the code on a vector machine without chaining technique?
 - If the above code running on a vector machine with chaining technique, how many conveys will this vector sequence take? Please showing the layout of the conveys.
 - If an element of a result vector going from a function unit to a chaining register need 1 clock cycle, and vice versa, how many clock cycles to execute the code using chaining technique?